Algorithms

James Goudy

March 18, 2026

Contents

0.1 CSCI 232 Data Structures and Algorithms . . . 1
0.1.1 Advanced Data Structures and Programming Techniques - Course Introduction 1
0.1.2 Course Learning Outcomes . . . 1
0.2 Terms and Concepts . . . 2
0.2.1 Radix - Number Bases . . . 3
0.2.2 Serializable . . . 4
0.2.3 Iterator . . . 5
0.3 Binary Search Tree - Intro . . . 7
0.3.1 Background Information . . . 7
0.3.2 Applications That Use Binary Trees . . . 8
0.3.3 Binary Tree Comparisons . . . 10
0.3.4 Tree Traversal . . . 11
0.3.5 Demonstrate BST using iterative looping and recursion . . . 12
0.3.6 Binary Search Tree . . . 17
0.3.7 AVL Tree . . . 31
0.3.8 Red Black Tree . . . 51
0.3.9 BST AVL Red Black Tree Comparisons . . . 65
0.3.10 Splay Trees . . . 68
0.4 Chapter: Hash Maps, Open Addressing, and Chaining . . . 79
0.4.1 Chaining . . . 82
0.4.2 Open Addressing . . . 83
0.5 Skip List . . . 104
0.5.1 Introduction . . . 104
0.5.2 Skiplist Diagram . . . 105
0.5.3 Lecture code explaination . . . 105
0.5.4 Lecture Code . . . 109
0.6 Graphs . . . 117
0.6.1 A Comparative Analysis of Breadth-First Search and Depth-First Search Algorithms . . . 118
0.6.2 Understanding Breadth-First Search (BFS) . . . 126
0.6.3 Understanding Depth-First Search (DFS) . . . 131
0.6.4 Seven Bridges of Königsberg . . . 136
0.6.5 Depth-First Search (DFS) . . . 137
0.6.6 Breadth First . . . 140
0.6.7 DFS - Stored Paths . . . 143
0.6.8 DSF - Maze Solving . . . 152
0.6.9 Dijkstra's Algorithm . . . 156
0.7 Huffman Code . . . 162
0.7.1 What is the Huffman Algorithm? . . . 162
0.7.2 Example Code . . . 165
0.7.3 Code Explanation . . . 169
0.8 Serialization . . . 177
0.8.1 **Definition of Serialization** . . . 177
0.8.2 **Uses of Serialization** . . . 177

0.8.3 **Example in Java** . . . 177
0.8.4 **Equivalent Example in Python** . . . 178
0.8.5 **Equivalent Example in C#** . . . 179
0.8.6 **Summary** . . . 180
0.9 JAVA Techniques . . . 181
0.9.1 Examples of Java's HashMap, TreeMap, and LinkedHashMap . . . 182
0.9.2 HashSet, LinkedHashSet, and TreeSet Comparision . . . 185
0.9.3 Programming To The Interface, Not The Implementation" . . . 190
0.10 End . . . 192

0.1 CSCI 232 Data Structures and Algorithms

0.1.1 Advanced Data Structures and Programming Techniques - Course Introduction

Welcome to the continuation of CSC 132! This course builds upon the foundational knowledge acquired in its prerequisite and dives deeper into advanced data structures and programming techniques, equipping students with the tools to tackle complex computational problems. Whether your goal is to enhance your programming expertise, refine algorithmic thinking, or understand the intricate mechanics behind software systems, this course provides the necessary framework to achieve those objectives.

The curriculum introduces a variety of data structures such as trees, balanced trees, graphs, dictionaries, hash tables, and heaps, exploring both their theoretical underpinnings and practical applications. Emphasis is placed on understanding the efficiency and correctness of algorithms, ensuring students can critically evaluate and implement solutions in real-world scenarios. A key component of the learning experience is the hands-on application of these concepts using Java, offering students the opportunity to reinforce their understanding through practical exercises and projects.

0.1.2 Course Learning Outcomes

By the end of the course, students will have developed the skills to confidently explain and implement recursive algorithms, perform time-complexity analyses, and compare various sorting techniques. The course also introduces the implementation of basic Abstract Data Types (ADTs) such as vectors, lists, sorted lists, stacks, and queues. Furthermore, students will gain in-depth knowledge of specialized structures and algorithms, including general trees, binary trees, balanced search trees, and hash tables.

A significant focus is placed on the evaluation and selection of appropriate data structures for solving specific problems. This critical thinking skill ensures that students are not only proficient in implementation but also capable of making informed design decisions in their programming endeavors.

This course sets the stage for advanced study and professional application in computer science, making it an essential step in your academic and career journey. Prepare to engage with challenging concepts, build robust programming skills, and expand your understanding of the dynamic field of computer science!

0.2 Terms and Concepts

0.2.1 Radix - Number Bases

In the context of coding, **radix** (also sometimes called **base**) refers to the **number of unique digits used to represent numbers** in a **positional numeral system**. Here's a breakdown:

Positional Numeral System:

This is a system in which the **value of a digit depends on its position within the number**. The most common example is the **decimal system** used in everyday life, where we have 10 digits (0-9) and the position of a digit determines its value by powers of 10. For example, in the number 123, the "1" represents 1 x $10\hat{2}$ (100), the "2" represents 2 x $10\hat{1}$ (20), and the "3" represents 3 x $10\hat{0}$ (3).

Radix and Digits:

The **radix** of a system defines the number of unique digits used. So, the decimal system has a radix of **10** because it uses 10 digits. Other common systems include:

- **Binary (base 2):** Uses only 2 digits (0 and 1).
- **Octal (base 8):** Uses 8 digits (0-7).
- **Hexadecimal (base 16):** Uses 16 digits (0-9, A-F).

Why Radix Matters in Coding:

Understanding radix is crucial in several coding aspects:

- **Converting between different numeral systems:** Programmers often need to convert between different bases, for example, from decimal to binary (common in computer hardware) or vice versa. Knowing the radix is essential for performing these conversions accurately.
- **Data representation:** Numbers in memory are often stored in binary format using a specific number of bits. Understanding radix allows programmers to interpret and manipulate these binary representations.
- **Error checking:** Some error checking techniques rely on calculating checksums or hash values based on specific radices. Understanding the radix used in these calculations is essential for interpreting the results.
- **Communication protocols:** Sometimes, data is exchanged between systems using different radices (e.g., ASCII uses base-10 for characters). Knowing the radices involved is crucial for proper data transmission and interpretation.

In summary, understanding radix is a fundamental concept in coding, as it forms the foundation for representing, manipulating, and interpreting numerical data in various contexts.

For example:

- In the **decimal system** (the most common system), the radix is **ten**, as it uses the ten digits from 0 through 9.
- In **hexadecimal** (base-16), the radix is **sixteen**, allowing digits from 0 to 15 (where A represents 10, B represents 11, and so on).
- When you perform operations like **parsing integers** or **converting numbers to strings**, the radix determines how the numbers are interpreted or displayed
  For instance:
  - `Integer.parseInt("11", 16)` interprets "11" as a base-16 number, resulting in **17** (1*16 + 1).
  - `Integer.toString(11, 16)` converts the decimal value 11 to hexadecimal, resulting in **B**.

0.2.2 Serializable

"Serializable" refers to a crucial concept related to data storage and transmission. Let me break it down for you:

1. **Serialization**:
   - **Definition**: Serialization is the process of converting a data structure or object into a sequence of bits. These bits can then be stored in a file, memory buffer, or transmitted across a network connection.
   - **Purpose**: Serialization allows data to be saved persistently (e.g., onto a disk) or transmitted between different computer environments.
   - Use Cases:
     - Saving an object's state to a file.
     - Sending data over a network.
     - Storing data in a database.

2. **Deserialization**:
   - **Definition**: Deserialization is the reverse process. It involves reading data from a serialized format (e.g., from a file) and reconstructing an object with the same state.
   - **Purpose**: Deserialization allows us to recreate an object from its serialized form.
   - **Example**: Imagine saving a complex data structure (like an intricate graph or a user's session data) by serializing the root object. Later, you can deserialize it to restore the original object.

0.2.3 Iterator

In Java, an iterator is an interface that allows you to traverse elements in a collection one at a time. It provides a standardized way to loop through various collections like ArrayList, HashSet, and more, regardless of their underlying implementation.

Here are some key points about iterators in Java:

Functionality:

- **Traversing Collections:** Iterators act like a cursor that points to the current element in a collection. You can use the `hasNext()` method to check if there are more elements remaining, and `next()` to retrieve the next element in the iteration process.
- **Modification:** Unlike the older `Enumeration` interface, iterators allow you to remove elements from the collection you're iterating over using the `remove()` method. This maintains a safe and well-defined way to modify the collection during iteration.

Benefits of Using Iterators:

- **Universality:** Since most collection classes in Java implement the `Iterator` interface, you can use the same looping pattern to iterate through different collection types with consistent behavior.
- **Safe Removal:** The `remove()` method allows for safe modification of the collection during iteration. This prevents issues like concurrent modification exceptions that can occur with other looping mechanisms.
- **Efficiency:** Iterators can be efficient, especially when dealing with large collections, as they retrieve elements on demand.

Using an Iterator:

1. **Import the Interface:** You'll typically need to import the `java.util.Iterator` interface at the beginning of your code.
2. **Get an Iterator:** Most collection classes provide an `iterator()` method that returns an iterator object specific to that collection.
3. **Looping:** Use a `while` loop to iterate through the collection. Inside the loop, call `hasNext()` to check for elements, and `next()` to access the current element.
4. **Optional Removal:** If needed, you can call `remove()` to remove the element you just accessed using `next()`. However, keep in mind that removing elements while iterating might have specific restrictions depending on the collection type.

By understanding iterators, you can effectively process and manipulate elements within various collections in your Java programs.

Example Code

```
import java.util.ArrayList;
import java.util.Iterator;

public class IteratorExample {

  public static void main(String[] args) {
    // Create an ArrayList of names
    ArrayList<String> names = new ArrayList<String>();
    names.add("Alice");
    names.add("Bob");
    names.add("Charlie");

    // Get an iterator for the ArrayList
```

```
    Iterator<String> it = names.iterator();

    // Loop through the elements using hasNext() and next()
    System.out.println("Names in the ArrayList:");
    while (it.hasNext()) {
      String name = it.next();
      System.out.println(name);
    }
  }
}
```

Iterator vs For statement

Both iterators and for statements are used for looping through collections in Java, but they have some key differences:

Focus:

- **Iterator:** An iterator is an **object** that provides a way to access elements in a collection one at a time. It offers a more general-purpose approach to traversing various data structures.
- **For statement:** A for statement is a **syntactic construct** specifically designed for looping through collections. It offers a concise and readable way to iterate, especially for simple forward iteration.

Functionality:

- **Iterator:** Provides methods like `hasNext()` and `next()` for element access and checking. It also offers optional removal of elements with `remove()`.
- **For statement (for-each loop):** This type of for statement is specifically designed for iterating over collections. It automatically retrieves the next element and assigns it to a temporary variable within the loop. It doesn't provide direct access to the current element's index or removal functionality.

Use Cases:

- **Iterator:** Use iterators when you need more control over the iteration process, such as removing elements during iteration or working with custom data structures that don't have built-in for loop support.
- **For statement (for-each loop):** Use for-each loops for simple forward iteration through collections where you only need to access the elements themselves. They are generally more concise and readable for these scenarios.

Here's a table summarizing the key differences:

Feature	Iterator	For Statement (for-each loop)
Type	Interface	Syntactic construct
Focus	Traversing elements one by one	Looping through collections
Functionality	`hasNext()`, `next()`, `remove()`	Automatic element access
Index access	No direct access	No access
Element modification	Optional with `remove()`	Not possible
Use cases	More control, custom data structures	Simple forward iteration

Choosing Between Iterator and For Statement:

- In most cases, for simple iteration through collections, the for-each loop is preferred due to its conciseness and readability.
- If you need more control over the iteration process, like removing elements or working with custom data structures, then iterators are the way to go.

0.3 Binary Search Tree - Intro

0.3.1 Background Information

A binary search tree (BST) is a special type of binary tree data structure used to efficiently store and access sorted data. It's like a regular tree, but with additional rules that keep everything neatly organized:

Key features:

- **Ordering:** Each node in the tree has a value (key), and the key of a node is **always greater than** all the keys in its **left subtree** and **less than** all the keys in its **right subtree**. This creates a sorted order throughout the tree.
- **Efficient searching:** Because of this ordering, you can very quickly search for specific values in the tree. Imagine searching a phone book - you wouldn't start at the very beginning or end, but somewhere in the middle based on the last name. Similarly, in a BST, you can efficiently move left or right depending on the value you're searching for, narrowing down the possibilities with each comparison.
- **Dynamic insertions and deletions:** You can easily add new values (insert) or remove existing ones (delete) from the tree while maintaining the sorted order. The process involves comparing the new value with existing nodes and finding its appropriate place based on the ordering rule.

Benefits of using BSTs:

- **Fast search:** Searching for elements in a balanced BST takes **logarithmic time** on average, which is significantly faster than searching an unsorted list.
- **Ordered access:** You can easily traverse the tree in sorted order (e.g., for printing all elements in ascending order).
- **Efficient insertions and deletions:** These operations can also be done in logarithmic time in balanced BSTs.

However, BSTs also have some drawbacks:

- **Performance depends on balance:** The efficiency of BST operations relies heavily on the tree's balance. An unbalanced tree can perform much worse than a balanced one.
- **Not self-balancing:** By default, BSTs are not self-balancing, meaning insertions and deletions can sometimes lead to imbalances. Special techniques are needed to maintain balance and ensure optimal performance.

Overall, binary search trees are a powerful and versatile data structure for storing and managing sorted data efficiently. They offer fast search, insertion, and deletion operations, making them suitable for various applications like symbol tables, priority queues, and sorting algorithms.

0.3.2 Applications That Use Binary Trees

1. Database Systems:

- **Indexing:** B-trees and B+ trees are fundamental to database indexing, enabling fast retrieval of data based on specific keys. They optimize search and storage for large datasets, making queries lightning-fast.
- **Query Optimization:** Binary trees assist in planning efficient query execution paths, minimizing read operations and boosting performance.
- **Transaction Management:** They play a role in managing concurrent transactions, ensuring data integrity and consistency.

2. File Systems:

- **Directory Structure:** Many file systems, including Unix-based ones, organize directories and files in a hierarchical structure using binary trees. This allows for efficient navigation and access to files based on their paths.
- **File Allocation:** B-trees can be used to manage disk block allocation, optimizing storage and access to file contents.

3. Networking:

- **Routing Protocols:** Routers often employ binary trees to store routing tables, enabling efficient path lookups for packet forwarding.
- **Network Management:** Binary trees can represent network topologies and assist in managing network resources effectively.

4. Artificial Intelligence:

- **Decision Trees:** These tree-like models are used for classification and prediction tasks, learning patterns from data to make decisions.
- **Search Algorithms:** Binary trees are essential for AI algorithms that explore large search spaces, like those used in game playing or problem-solving.
- **Machine Learning:** Some machine learning techniques, like decision trees and random forests, rely on binary trees for model construction and prediction.

5. Compilers and Interpreters:

- **Syntax Parsing:** Binary trees are used to parse code syntax, representing the structure of expressions and statements for analysis and code generation.
- **Abstract Syntax Trees (ASTs):** These trees represent the abstract structure of code, used for optimizations and code generation.

6. Data Compression:

- **Huffman Coding:** This compression technique relies on binary trees to assign variable-length codes to symbols based on their frequencies, reducing data size effectively.

7. Computer Graphics:

- **Scene Management:** Binary space partitioning (BSP) trees are used to efficiently divide 3D scenes into smaller regions for rendering, optimizing graphics performance.
- **Ray Tracing:** Binary trees can accelerate ray intersection calculations, essential for realistic image rendering.

8. Sorting and Searching:

- **Binary Search Trees (BSTs):** These trees enable efficient searching, insertion, and deletion of elements, making them valuable for various data organization tasks.
- **Heaps:** Used for implementing priority queues, which are essential for scheduling tasks, managing resources, and optimization algorithms.

9. Expression Evaluation:

- **Expression Trees:** Represent mathematical expressions, enabling their evaluation and simplification.

10. Game Development:

- **Game Trees:** Represent possible moves and game states, used in AI algorithms for strategy games like chess and checkers.

0.3.3 Binary Tree Comparisons

Comparison Chart

Feature	Binary Tree	AVL Tree	Red-Black Tree
Structure	Nodes have 0 or 2 children	Self-balancing, height difference between subtrees <= 1	Self-balancing, specific color properties to maintain balance
Data Organization	Can hold any type of data	Usually holds comparable data for efficient search	Can hold any type of data
Search Performance	O(n) in worst case, O(log n) on average	O(log n) guaranteed	O(log n) guaranteed
Insertion/Deletion Performance	O(n) in worst case, O(log n) on average	O(log n) guaranteed, more complex operations	O(log n) guaranteed, simpler operations than AVL
Memory Usage	Minimum, only 2 pointers per node	Slightly higher than binary tree, stores balance factor	Slightly higher than binary tree, stores color information
Applications	Simple data structures, expression trees	Efficient search and sorting, databases	General purpose, good balance between performance and complexity
Pros	Simple to implement, efficient for small datasets	Fast search and guaranteed balance	Faster insertions than AVL, simpler operations
Cons	Can become unbalanced, leading to poor performance	More complex to implement than binary tree	Not as strictly balanced as AVL trees, potentially impacting search performance in rare cases

0.3.4 Tree Traversal

Pre-order, post-order, and in-order are not "order types" in the general sense, but rather **tree traversal methods** used to visit all nodes in a tree structure. They define the order in which we visit the root, left subtree, and right subtree of each node. Here's a breakdown of each:

Pre-order:

- Visit the **root** of the node.
- Recursively visit the **left subtree**.
- Recursively visit the **right subtree**.

Think of it as visiting the parent first, then the children, like inspecting a family photo by focusing on the parents before moving to their individual pictures.

In-order:

- Recursively visit the **left subtree**.
- Visit the **root** of the node.
- Recursively visit the **right subtree**.

Imagine reading a sentence word by word: you start at the leftmost word, move to the next, and so on, reaching the root (middle word) last.

Post-order:

- Recursively visit the **left subtree**.
- Recursively visit the **right subtree**.
- Visit the **root** of the node.

This is like cleaning up after a plant's growth: you prune the left and right branches before finally taking care of the main trunk.

Here are some examples of how these traversals are used in real life:

- **Pre-order:** Building a file system where folders are processed before their contents, or printing the document outline with headings first.
- **In-order:** Sorting a list of numbers in ascending order using a binary search tree, or printing the words in a sentence in the correct order.
- **Post-order:** Evaluating an arithmetic expression where leaves (operands) are processed before the operators, or calculating the size of a directory by first traversing all subdirectories.

0.3.5 Demonstrate BST using iterative looping and recursion

This Java program implements a **Binary Search Tree (BST)** that supports insertion, searching, and in-order traversal. It generates unique random numbers, inserts them into the BST, and allows the user to search for a specific number using both recursive and iterative approaches.

The **`Node` class** defines a BST node with an integer value and left and right child references. The **`BST` class** manages the BST operations, including inserting new values, searching for values recursively and iteratively, and performing an in-order traversal to display elements in sorted order.

In the **main method (`BST_Recursion_Loops`)**, the program first generates an array of 50 unique random numbers using a `HashSet`, ensuring all values are distinct. These numbers are inserted into the BST, and an in-order traversal is performed to display the sorted tree contents. The program then prompts the user to enter a search key and performs both iterative and recursive searches to check for its presence, displaying whether the key was found or not.

This implementation highlights key BST operations and provides a practical demonstration of recursion, loops, and tree traversal techniques in Java.

HashSet

A HashSet in Java is a part of the java.util package and implements the Set interface. It is used to store a collection of unique elements, meaning it does not allow duplicate values. It is backed by a HashMap, which makes its operations fast, typically running in O(1) time complexity for basic operations like add, remove, and contains. Key Features of HashSet

```
No Duplicates          It does not allow duplicate elements.
Unordered Collection   It does not maintain the order of elements.
Allows Null Values     It permits a single null value.
Fast Operations        Add, remove, and search operations are
                        generally O(1) due to hashing.
No Indexing            Unlike lists,you cannot access elements by an index.
```

Demo Code

```
/*

Demonstrate iterative and recursive searching
Developer: James Goudy
 */
package bst_recursion_loops;

import java.util.HashSet;
import java.util.Random;
import java.util.Scanner;
import org.w3c.dom.css.Counter;

// Class for a Binary Search Tree Node
class Node {

    int value;
    Node left, right;

    public Node(int item) {
        value = item;
```

```
        left = right = null;
    }
}

class BST {

    int cntr = 1;
    Node root;

    // Constructor
    public BST() {
        root = null;
    }

    // Recursive Search Function
    public boolean searchRecursive(Node root, int key) {
        // Base case: root is null or key is found
        if (root == null) {
            return false;
        }
        if (root.value == key) {
            return true;
        }

        // If the key is smaller, search in the left subtree
        if (key < root.value) {
            return searchRecursive(root.left, key);
        } else {
            // If the key is larger, search in the right subtree
            return searchRecursive(root.right, key);
        }
    }

    // Iterative Search Function
    public boolean searchIterative(Node root, int key) {
        while (root != null) {
            if (root.value == key) {
                return true;
            } else if (key < root.value) {
                root = root.left;  // Move to left subtree
            } else {
                root = root.right; // Move to right subtree
            }
        }
        return false; // Key not found
    }

    // Insert function to add nodes to the BST
    public void insert(int key) {
        root = insertData(root, key);
    }

    private Node insertData(Node root, int key) {
```

```
        if (root == null) {
            root = new Node(key);
            return root;
        }

        if (key < root.value) {
            root.left = insertData(root.left, key);
        } else if (key > root.value) {
            root.right = insertData(root.right, key);
        }

        return root;
    }

    // Helper function to start recursive search from the root
    public boolean searchRecursive(int key) {
        return searchRecursive(root, key);
    }

    // Helper function to start iterative search from the root
    public boolean searchIterative(int key) {
        return searchIterative(root, key);
    }

    // Inorder Traversal to print the BST
    public void inorder() {
        cntr = 1;
        inorderData(root);
        System.out.println();
    }

    private void inorderData(Node root) {

        if (root != null) {
            cntr++;

            if (cntr % 10 == 0 && cntr > 1) {
                System.out.println("");
            }



            inorderData(root.left);
            System.out.print(root.value + " ");
            inorderData(root.right);

        }

    }

}

public class BST_Recursion_Loops {
```

```
public static int[] generateUniqueArray(int size) {

    // create new HashSet
    HashSet<Integer> uniqueNumbers = new HashSet<>();

    Random rng = new Random();

    // Fill the set with unique random numbers
    while (uniqueNumbers.size() < size) {
        // Generates numbers from 0 to 999
        uniqueNumbers.add(rng.nextInt(1000));
    }

    // Convert HashSet to an array
    int[] result = new int[size];
    int index = 0;
    for (int num : uniqueNumbers) {
        result[index++] = num;
    }

    return result;
}

public static void main(String[] args) {

    // create sample data
    int[] sampleKeys;
    int numOfKeys = 50;
    int searchKey = 0;
    String status;



    Scanner myScan = new Scanner(System.in);
    BST myBST = new BST();

    sampleKeys = generateUniqueArray(numOfKeys);

    System.out.println("\nInput Data\n");
    for (int i = 0; i < sampleKeys.length; i++) {
        myBST.insert(sampleKeys[i]);
        System.out.print(sampleKeys[i] + " ");

        if (i % 10 == 0 && i > 0) {
            System.out.println("");
        }
    }

    System.out.println("\n - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -\n");

    myBST.inorder();
```

```
        System.out.print(" Enter Search Key: ");
        searchKey = Integer.parseInt(myScan.nextLine());

        System.out.println("\nSearch Iteratively");
        status = myBST.searchIterative(searchKey)? "Found" : "Not Found";
        System.out.println("The items was - " + status);

        System.out.println("\nSearch Recursively");
        status = myBST.searchRecursive(searchKey)? "Found" : "Not Found";
        System.out.println("The items was - " + status);


    }

}
```

0.3.6 Binary Search Tree

Background Information

A binary search tree (BST) is a special type of binary tree data structure used to efficiently store and access sorted data. It's like a regular tree, but with additional rules that keep everything neatly organized:

Key features:

- **Ordering:** Each node in the tree has a value (key), and the key of a node is **always greater than** all the keys in its **left subtree** and **less than** all the keys in its **right subtree**. This creates a sorted order throughout the tree.
- **Efficient searching:** Because of this ordering, you can very quickly search for specific values in the tree. Imagine searching a phone book - you wouldn't start at the very beginning or end, but somewhere in the middle based on the last name. Similarly, in a BST, you can efficiently move left or right depending on the value you're searching for, narrowing down the possibilities with each comparison.
- **Dynamic insertions and deletions:** You can easily add new values (insert) or remove existing ones (delete) from the tree while maintaining the sorted order. The process involves comparing the new value with existing nodes and finding its appropriate place based on the ordering rule.
- **No Duplicates**: Typically, BSTs do **not** allow duplicate values, although variations exist that permit duplicates with specific rules (e.g., storing duplicates in the left or right subtree).

Benefits of using BSTs:

- **Fast search:** Searching for elements in a balanced BST takes **logarithmic time** on average, which is significantly faster than searching an unsorted list.
- **Ordered access:** You can easily traverse the tree in sorted order (e.g., for printing all elements in ascending order).
- **Efficient insertions and deletions:** These operations can also be done in logarithmic time in balanced BSTs.

However, BSTs also have some drawbacks:

- **Performance depends on balance:** The efficiency of BST operations relies heavily on the tree's balance. An unbalanced tree can perform much worse than a balanced one.
- **Not self-balancing:** By default, BSTs are not self-balancing, meaning insertions and deletions can sometimes lead to imbalances. Special techniques are needed to maintain balance and ensure optimal performance.

Overall, binary search trees are a powerful and versatile data structure for storing and managing sorted data efficiently. They offer fast search, insertion, and deletion operations, making them suitable for various applications like symbol tables, priority queues, and sorting algorithms.

Lecture Code

```
/*
 * binary tree
 *
 */
package btreedemoone;

import java.util.Scanner;
import java.util.Stack;

class Node {

    // key
    public int key;
```

```
    // data - that's a double
    public int data;

    // node characteristics
    public Node leftChild;
    public Node rightChild;

    public Node(int key, int data) {
        this.key = key;
        this.data = data;
    }

    // display node
    public void displayNode() {
        System.out.print("{" + key + "," + data + "}");
    }

}// end of node

class Tree {

    private Node root;
    static Scanner myScan = new Scanner(System.in);

    public Tree() {
        root = null;
    }

    public void insert(int key, int data) {
        // newNode
        Node newNode = new Node(key, data);
        boolean run = true;

        if (root == null) {
            root = newNode;
        } else {
            Node current = root;
            Node parent;

            while (run) {
                parent = current;

                if (key < current.key) //go left
                {
                    current = current.leftChild;

                    if (current == null) {
                        parent.leftChild = newNode;
                        run = false;
                    }
                } else // go right
                {
                    current = current.rightChild;
```

```
                if (current == null) {
                    parent.rightChild = newNode;
                    run = false;
                }
            }
        }

    }

}// end of insert

public Node find(int key) {

    Node current = root;

    // check if empty
    if (root == null) {
        System.out.println("** Tree is empty **");
        return null;
    }

    while (current.key != key) {

        if (key < current.key) // go left?
        {
            current = current.leftChild;
        } else // go right
        {
            current = current.rightChild;
        }

        if (current == null) {
            System.out.println("*** key not found ***");
            return null;
        }
    }

    System.out.println("Key was FOUND");
    return current;
}

public boolean delete(int key) {
    Node current = root;
    Node parent = root;
    boolean isLeftChild = true;

    //check if empty
    if (root == null) {
        System.out.println("*** Tree is empty ***");
        return false;
    }
```

```
// look for the key
while (current.key != key) {
    parent = current;

    if (key < current.key) // go left?
    {
        isLeftChild = true;
        current = current.leftChild;
    } else // go right
    {
        isLeftChild = false;
        current = current.rightChild;
    }

    if (current == null) {
        System.out.println("*** key not found ***");
        return false;
    }

}// end while

// found we are on the node to delete
// if there is no children - simply delete the node
if (current.leftChild == null && current.rightChild == null) {
    //check if node is the root (there is only one node in tree)
    if (current == root) {
        root = null;
    } else if (isLeftChild) {
        parent.leftChild = null;
    } else {
        parent.rightChild = null;
    }
} // if no right child, rplace with left subtree
else if (current.rightChild == null) {
    if (current == root) {
        root = current.leftChild;
    } else if (isLeftChild) {
        parent.leftChild = current.leftChild;
    } else {
        parent.rightChild = current.leftChild;
    }
} //if no left child, replace with right subtree
else if (current.leftChild == null) {
    if (current == root) {
        root = current.rightChild;
    } else if (isLeftChild) {
        parent.leftChild = current.rightChild;
    } else {
        parent.rightChild = current.rightChild;
    }
} // if there are two children, replace with inorder successor
else {
    //get successor of node to delete of current
```

```
            Node successor = getSuccessor(current);

            // connect parent of current to successor instead
            if (current == root) {
                root = successor;
            } else if (isLeftChild) {
                parent.leftChild = successor;
            } else {
                parent.rightChild = successor;
            }

            // connect successor to current's left child
            successor.leftChild = current.leftChild;

            // NOTE: successor cannot have a left child
        }

        return true;

    }

    // return node with the next highest value after the delete node
    // goes to right child, then right child's lef descendent's
    private Node getSuccessor(Node deleteNode) {
        Node successorParent = deleteNode;
        Node successor = deleteNode;
        Node current = deleteNode.rightChild;

        while (current != null) {
            successorParent = successor;
            successor = current;
            current = current.leftChild;
        }

        // if successor not successful
        if (successor != deleteNode.rightChild) {
            successorParent.leftChild = successor.rightChild;
            successor.rightChild = deleteNode.rightChild;
        }

        return successor;
    }

    public void traverse2() {

        System.out.print("\nPreorder Traversal: ");
        preorder(root);

        System.out.print("\nInorder Traversal: ");
        inorder(root);

        System.out.print("\nPostorder Traversal: ");
        postorder(root);
```

```
    }

    private void preorder(Node nodeStart) {
        if (nodeStart != null) {
            System.out.print(nodeStart.key + " ");
            preorder(nodeStart.leftChild);
            preorder(nodeStart.rightChild);
        }
    }

    private void inorder(Node nodeStart) {
        if (nodeStart != null) {
            inorder(nodeStart.leftChild);
            System.out.print(nodeStart.key + " ");
            inorder(nodeStart.rightChild);
        }
    }

    private void postorder(Node nodeStart) {
        if (nodeStart != null) {
            postorder(nodeStart.leftChild);
            postorder(nodeStart.rightChild);
            System.out.print(nodeStart.key + " ");
        }
    }

    public void displayTree() {
        Stack globalStack = new Stack();
        globalStack.push(root);

        int nBlanks = 32;

        boolean isRowEmpty = false;

        System.out.println(
                "\n.........         Display Tree     ........................");

        while (!isRowEmpty) {
            Stack localStack = new Stack();
            isRowEmpty = true;

            for (int j = 0; j < nBlanks; j++) {
                System.out.print(" ");
            }

            while (globalStack.isEmpty() == false) {
                Node temp = (Node) globalStack.pop();
                if (temp != null) {
                    System.out.print(temp.data);
                    localStack.push(temp.leftChild);
                    localStack.push(temp.rightChild);
```

```
                    if (temp.leftChild != null || temp.rightChild != null) {
                        isRowEmpty = false;
                    }
                } else {
                    System.out.print(" - -");
                    localStack.push(null);
                    localStack.push(null);
                }

                for (int j = 0; j < nBlanks * 2 - 2; j++) {
                    System.out.print(' ');
                }
            }//while

            System.out.println();
            nBlanks /= 2;
            while (localStack.isEmpty() == false) {
                globalStack.push(localStack.pop());
            }
        }
        // for separation
        System.out.println(" - - - - - - - - - - - - - - - - - - - - - - - - - - - - -");
    } // display tree

}

public class BTreeDemoOne {

    static Scanner myScan = new Scanner(System.in);

    public static void main(String[] args) {

        String quit = "n";

        int findValue = 0;
        int deleteValue = 0;
        int insertValue = 0;

        boolean result = false;

        Tree theTree = new Tree();

        while (!quit.equals("y")) {
            theTree.insert(50, 50);
            theTree.insert(25, 25);
            theTree.insert(75, 75);
            theTree.insert(12, 12);
            theTree.insert(37, 37);
            theTree.insert(43, 43);
            theTree.insert(30, 30);
            theTree.insert(33, 33);
            theTree.insert(87, 87);
            theTree.insert(93, 93);
```

```
            theTree.insert(97, 97);
            theTree.insert(70, 70);
            theTree.insert(60, 60);

            theTree.displayTree();

            theTree.insert(85, 85);
            theTree.insert(47, 47);


            theTree.displayTree();

            try {
                System.out.print("Enter an integer to insert "
                        + "or letter to skip: ");
                insertValue = Integer.parseInt(myScan.nextLine());
                theTree.insert(insertValue, insertValue);
                theTree.displayTree();

            } catch (Exception e) {
            }


            theTree.traverse2();
            try {

                System.out.print("\nEnter Value To Find or letter to skip: ");
                findValue = Integer.parseInt(myScan.nextLine());
                theTree.find(findValue);


                System.out.println("\nEnter value to delete: ");
                deleteValue = Integer.parseInt(myScan.nextLine());

                if (theTree.delete(deleteValue)) {
                    theTree.displayTree();
                } else {
                    System.out.println("NOT FOUND");
                }
            } catch (Exception e) {
            }

            System.out.print("Would you like to quit y/n: ");
            quit = myScan.nextLine().toLowerCase();

        }

        System.out.println("\nbye\n");
    }

}
```

BST as Class

```
/*
```

```
    Change the package to your specific package
 */
package ds_redblacktree_rev2_separateclass;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

/**
 *
 * @author jgoudy
 */
public class BinaryTreeSearch
{

    class Node
    {

        int key;
        Node left;
        Node right;



        Node(int key)
        {
            this.key = key;
            right = null;
            left = null;
        }
    }

    Node root;
    public int trip = 0;

    public void add(int key)
    {
        root = addRecursive(root, key);
    }

    private Node addRecursive(Node current, int key)
    {

        if (current == null)
        {
            return new Node(key);
        }

        if (key < current.key)
        {
            current.left = addRecursive(current.left, key);
        } else if (key > current.key)
        {
```

```
            current.right = addRecursive(current.right, key);
        }

        return current;
    }

    public boolean isEmpty()
    {
        return root == null;
    }

    public int getSize()
    {
        return getSizeRecursive(root);
    }

    private int getSizeRecursive(Node current)
    {
        return current == null ? 0 : getSizeRecursive(current.left)
                + 1 + getSizeRecursive(current.right);
    }

    public boolean containsNode(int key)
    {
        trip = 0;
        return containsNodeRecursive(root, key);
    }

    private boolean containsNodeRecursive(Node current, int key)
    {
        if (current == null)
        {
            return false;
        }
        trip++;
        if (key == current.key)
        {
            return true;
        }

        return key < current.key
                ? containsNodeRecursive(current.left, key)
                : containsNodeRecursive(current.right, key);
    }

    public void delete(int key)
    {
        root = deleteRecursive(root, key);
    }

    private Node deleteRecursive(Node current, int key)
    {
        if (current == null)
```

```
        {
            return null;
        }

        if (key == current.key)
        {
            // Case 1: no children
            if (current.left == null && current.right == null)
            {
                return null;
            }

            // Case 2: only 1 child
            if (current.right == null)
            {
                return current.left;
            }

            if (current.left == null)
            {
                return current.right;
            }

            // Case 3: 2 children
            int smallestValue = findSmallestValue(current.right);
            current.key = smallestValue;
            current.right = deleteRecursive(current.right, smallestValue);
            return current;
        }
        if (key < current.key)
        {
            current.left = deleteRecursive(current.left, key);
            return current;
        }

        current.right = deleteRecursive(current.right, key);
        return current;
    }

    public int findSmallestValue(Node root)
    {
        return root.left == null ? root.key : findSmallestValue(root.left);
    }

    public int findLargestValue(Node root)
    {
        return root.left == null ? root.key : findLargestValue(root.right);
    }

    public void traverseInOrder(Node node)
    {
        if (node != null)
        {
```

```
            traverseInOrder(node.left);
            visit(node.key);
            traverseInOrder(node.right);
        }
    }

    public void traversePreOrder(Node node)
    {
        if (node != null)
        {
            visit(node.key);
            traversePreOrder(node.left);
            traversePreOrder(node.right);
        }
    }

    public void traversePostOrder(Node node)
    {
        if (node != null)
        {
            traversePostOrder(node.left);
            traversePostOrder(node.right);
            visit(node.key);
        }
    }

    public void traverseLevelOrder()
    {
        if (root == null)
        {
            return;
        }

        Queue<Node> nodes = new LinkedList<>();
        nodes.add(root);

        while (!nodes.isEmpty())
        {

            Node node = nodes.remove();

            System.out.print(" " + node.key);

            if (node.left != null)
            {
                nodes.add(node.left);
            }

            if (node.left != null)
            {
                nodes.add(node.right);
            }
        }
```

```
}

public void traverseInOrderWithoutRecursion()
{
    Stack<Node> stack = new Stack<Node>();
    Node current = root;
    stack.push(root);
    while (!stack.isEmpty())
    {
        while (current.left != null)
        {
            current = current.left;
            stack.push(current);
        }
        current = stack.pop();
        visit(current.key);
        if (current.right != null)
        {
            current = current.right;
            stack.push(current);
        }
    }
}

public void traversePreOrderWithoutRecursion()
{
    Stack<Node> stack = new Stack<Node>();
    Node current = root;
    stack.push(root);
    while (!stack.isEmpty())
    {
        current = stack.pop();
        visit(current.key);

        if (current.right != null)
        {
            stack.push(current.right);
        }

        if (current.left != null)
        {
            stack.push(current.left);
        }
    }
}

public void traversePostOrderWithoutRecursion()
{
    Stack<Node> stack = new Stack<Node>();
    Node prev = root;
    Node current = root;
    stack.push(root);
```

```
        while (!stack.isEmpty())
        {
            current = stack.peek();
            boolean hasChild = (current.left != null || current.right != null);
            boolean isPrevLastChild = (prev == current.right
                    || (prev == current.left && current.right == null));

            if (!hasChild || isPrevLastChild)
            {
                current = stack.pop();
                visit(current.key);
                prev = current;
            } else
            {
                if (current.right != null)
                {
                    stack.push(current.right);
                }
                if (current.left != null)
                {
                    stack.push(current.left);
                }
            }
        }
    }

    private void visit(int key)
    {
        System.out.print(" " + key);
    }

}
```

0.3.7 AVL Tree

AVL Tree Overview

An **AVL (Adelson-Velsky and Landis) Tree** is a type of self-balancing binary search tree. It is named after its inventors, **Georgy Adelson-Velsky** and **Evgenii Landis**, who introduced it in 1962.

In a typical binary search tree, the key in each node is larger than all keys in its left subtree and smaller than all keys in its right subtree. While a standard binary search tree can degrade to a linked list in the worst case, an AVL tree enforces additional balance properties that prevent extreme imbalance.

Key Properties and Balance Rules

1. **Balance Factor**: For each node, the AVL tree tracks a balance factor defined as:

   balance(node)=height(node.left)-height(node.right)\text{balance}(node) = \text{height}(node.left) - \text{height}(node.right)

   In an AVL tree, the balance factor for any node must always be **-1**, **0**, or **1**.

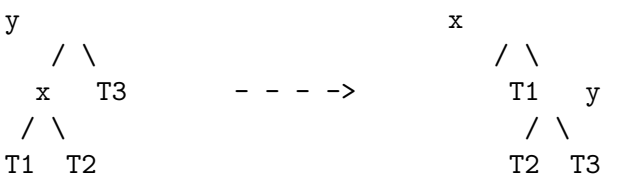
2. **Height Constraint**: The height of the left and right subtrees of any node differ by at most **1**. If the height difference exceeds 1, the tree performs rotations to restore balance.

3. **Rotations**: The four possible rotation cases occur after an insertion or deletion that breaks the balance:

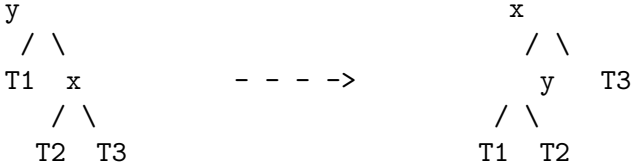
   - **Left-Left (LL) rotation**: Performed when a node is inserted into the left subtree of the left child. Fixed by a single right rotation.
   - **Right-Right (RR) rotation**: Performed when a node is inserted into the right subtree of the right child. Fixed by a single left rotation.
   - **Left-Right (LR) rotation**: Performed when a node is inserted into the right subtree of the left child. Fixed by a left rotation on the left child, then a right rotation on the node.
   - **Right-Left (RL) rotation**: Performed when a node is inserted into the left subtree of the right child. Fixed by a right rotation on the right child, then a left rotation on the node.

Single Rotations

- **Right Rotation** (for an unbalanced node leaning left):



- **Left Rotation** (for an unbalanced node leaning right):



Double Rotations When the subtree is not in a strictly left-left or right-right configuration but rather "zig-zag," two rotations are performed in sequence. For instance, **Left-Right** rotation is effectively a left rotation on the left child, followed by a right rotation on the original node.

Java Implementation of an AVL Tree

Below is a simple Java class that implements:

- **Insertion**
- **Deletion**
- **Search**
- **Traversals**: In-order, Pre-order, Post-order
- A **pretty print** method to display the tree structure

```
public class AVLTree<T extends Comparable<T>> {

    private class Node {
        T key;
        Node left, right;
        int height;

        Node(T key) {
            this.key = key;
            height = 1;
        }
    }

    private Node root;

    // - - - - - - - - - - - - - - - - - - - - - - -
    // Utility Functions
    // - - - - - - - - - - - - - - - - - - - - - - -

    // Return the height of a node, or 0 if null
    private int height(Node node) {
        return (node == null) ? 0 : node.height;
    }

    // Calculate the balance factor of a node
    private int getBalance(Node node) {
        if (node == null) {
            return 0;
        }
        return height(node.left) - height(node.right);
    }

    // Update the node's height based on children's heights
    private void updateHeight(Node node) {
        node.height = 1 + Math.max(height(node.left), height(node.right));
    }

    // - - - - - - - - - - - - - - - - - - - - - - -
```

```
    // Rotations
    // - - - - - - - - - - - - - - - - - - - - - - - - -

    // Right rotate subtree rooted with y
    private Node rotateRight(Node y) {
        Node x = y.left;
        Node T2 = x.right;

        // Perform rotation
        x.right = y;
        y.left = T2;

        // Update heights
        updateHeight(y);
        updateHeight(x);

        // x becomes new root
        return x;
    }

    // Left rotate subtree rooted with x
    private Node rotateLeft(Node x) {
        Node y = x.right;
        Node T2 = y.left;

        // Perform rotation
        y.left = x;
        x.right = T2;

        // Update heights
        updateHeight(x);
        updateHeight(y);

        // y becomes new root
        return y;
    }

    // - - - - - - - - - - - - - - - - - - - - - - - - -
    // Insertion
    // - - - - - - - - - - - - - - - - - - - - - - - - -
    public void insert(T key) {
        root = insertRec(root, key);
    }

    private Node insertRec(Node node, T key) {
        // 1. Perform the normal BST insertion
        if (node == null) {
            return new Node(key);
        }

        int cmp = key.compareTo(node.key);
        if (cmp < 0) {
            node.left = insertRec(node.left, key);
```

```
        } else if (cmp > 0) {
            node.right = insertRec(node.right, key);
        } else {
            // Duplicate keys not allowed or handle as desired
            return node;
        }

        // 2. Update the height of the ancestor node
        updateHeight(node);

        // 3. Get the balance factor
        int balance = getBalance(node);

        // 4. If the node becomes unbalanced, then check the four cases

        // Left Left Case
        if (balance > 1 && key.compareTo(node.left.key) < 0) {
            return rotateRight(node);
        }

        // Right Right Case
        if (balance < -1 && key.compareTo(node.right.key) > 0) {
            return rotateLeft(node);
        }

        // Left Right Case
        if (balance > 1 && key.compareTo(node.left.key) > 0) {
            node.left = rotateLeft(node.left);
            return rotateRight(node);
        }

        // Right Left Case
        if (balance < -1 && key.compareTo(node.right.key) < 0) {
            node.right = rotateRight(node.right);
            return rotateLeft(node);
        }

        // return the (unchanged) node pointer
        return node;
    }

    // - - - - - - - - - - - - - - - - - - - - - - - - -
    // Deletion
    // - - - - - - - - - - - - - - - - - - - - - - - - -
    public void delete(T key) {
        root = deleteRec(root, key);
    }

    private Node deleteRec(Node node, T key) {
        // 1. Perform standard BST delete
        if (node == null) {
            return node;
        }
```

```
int cmp = key.compareTo(node.key);
if (cmp < 0) {
    node.left = deleteRec(node.left, key);
} else if (cmp > 0) {
    node.right = deleteRec(node.right, key);
} else {
    // Node with only one child or no child
    if (node.left == null || node.right == null) {
        Node temp = (node.left != null) ? node.left : node.right;
        // No child
        if (temp == null) {
            node = null;
        } else {
            // One child
            node = temp;
        }
    } else {
        // Node with two children: get the inorder successor
        Node successor = getMinValueNode(node.right);
        // Copy successor's data to this node
        node.key = successor.key;
        // Delete the successor
        node.right = deleteRec(node.right, successor.key);
    }
}

// If the tree had only one node
if (node == null) {
    return node;
}

// 2. Update height of the current node
updateHeight(node);

// 3. Get balance factor
int balance = getBalance(node);

// 4. Rebalance if needed

// Left Left Case
if (balance > 1 && getBalance(node.left) >= 0) {
    return rotateRight(node);
}

// Left Right Case
if (balance > 1 && getBalance(node.left) < 0) {
    node.left = rotateLeft(node.left);
    return rotateRight(node);
}

// Right Right Case
if (balance < -1 && getBalance(node.right) <= 0) {
```

```
            return rotateLeft(node);
        }

        // Right Left Case
        if (balance < -1 && getBalance(node.right) > 0) {
            node.right = rotateRight(node.right);
            return rotateLeft(node);
        }

        return node;
    }

    // Helper function to find the node with the minimum key value
    private Node getMinValueNode(Node node) {
        Node current = node;
        while (current.left != null) {
            current = current.left;
        }
        return current;
    }

    // - - - - - - - - - - - - - - - - - - - - - - - -
    // Search
    // - - - - - - - - - - - - - - - - - - - - - - - -
    public boolean search(T key) {
        return searchRec(root, key);
    }

    private boolean searchRec(Node node, T key) {
        if (node == null) {
            return false;
        }
        int cmp = key.compareTo(node.key);
        if (cmp < 0) {
            return searchRec(node.left, key);
        } else if (cmp > 0) {
            return searchRec(node.right, key);
        } else {
            // cmp == 0, key found
            return true;
        }
    }

    // - - - - - - - - - - - - - - - - - - - - - - - -
    // Traversals
    // - - - - - - - - - - - - - - - - - - - - - - - -
    public void inOrder() {
        System.out.print("InOrder: ");
        inOrderRec(root);
        System.out.println();
    }

    private void inOrderRec(Node node) {
```

```
        if (node == null) {
            return;
        }
        inOrderRec(node.left);
        System.out.print(node.key + " ");
        inOrderRec(node.right);
    }

    public void preOrder() {
        System.out.print("PreOrder: ");
        preOrderRec(root);
        System.out.println();
    }

    private void preOrderRec(Node node) {
        if (node == null) {
            return;
        }
        System.out.print(node.key + " ");
        preOrderRec(node.left);
        preOrderRec(node.right);
    }

    public void postOrder() {
        System.out.print("PostOrder: ");
        postOrderRec(root);
        System.out.println();
    }

    private void postOrderRec(Node node) {
        if (node == null) {
            return;
        }
        postOrderRec(node.left);
        postOrderRec(node.right);
        System.out.print(node.key + " ");
    }

    // - - - - - - - - - - - - - - - - - - - - - - - - -
    // Pretty Print (Tree Display)
    // - - - - - - - - - - - - - - - - - - - - - - - - -
    public void printTree() {
        printTreeRec(root, "", true);
    }

    /*
     * Traverses the tree in a rotated manner and prints branches to visualize
     * the structure. The approach is to go right, then print current, then go left.
     */
    private void printTreeRec(Node node, String indent, boolean isRight) {
        if (node == null) {
            return;
        }
```

```
        // Print right subtree
        printTreeRec(node.right, indent + (isRight ? "     " : " |   "), false);

        // Print current node
        System.out.print(indent);
        if (isRight) {
            System.out.print("");
        } else {
            System.out.print("");
        }
        System.out.println(node.key);

        // Print left subtree
        printTreeRec(node.left, indent + (isRight ? " |   " : "     "), true);
    }

    // - - - - - - - - - - - - - - - - - - - - - - - -
    // Main (for demonstration)
    // - - - - - - - - - - - - - - - - - - - - - - - -
    public static void main(String[] args) {
        AVLTree<Integer> tree = new AVLTree<>();
        int[] values = { 10, 20, 30, 40, 50, 25 };
        for (int val : values) {
            tree.insert(val);
        }

        tree.printTree();
        tree.inOrder();
        tree.preOrder();
        tree.postOrder();

        System.out.println("Search 25: " + tree.search(25));
        System.out.println("Search 15: " + tree.search(15));

        System.out.println("\nDelete 20");
        tree.delete(20);
        tree.printTree();
    }
}
```

Explanation of the Key Methods

1. **Node Class** Each node stores:
   - The key (generic type `T`).
   - References to the left and right children.
   - An integer `height` to aid in maintaining balance.
2. **Height and Balance Factor**
   - `height(node)`: returns the height of the node (with `null` as height 0).
   - `getBalance(node)`: computes the difference in heights of the left and right subtrees.
3. **Insertion**
   - Insert the new key as in a regular BST.
   - Update the height of the ancestor nodes.

- Check the balance factor.
- Perform appropriate rotations if unbalanced.

4. **Deletion**
   - Perform the usual BST deletion (case of no child, one child, or two children).
   - Update the height and balance for nodes on the path back up.
   - Fix any imbalance with the correct rotations.
5. **Search** Standard BST search logic:
   - Compare, traverse left or right, or return true if the key is found.
6. **Traversals**
   - **In-order**: left, node, right
   - **Pre-order**: node, left, right
   - **Post-order**: left, right, node
7. **Pretty Print** A simple, recursive approach that prints the tree horizontally, showing child branches with ASCII symbols.

Time Complexity

- **Insertion**, **Deletion**, and **Search** each take **O(log n)** time on average because the AVL tree remains balanced (height is O(logn)\mathcal{O}(\log n)).

With this, you have a complete Java class demonstrating the core features of an AVL tree, along with helpful utility methods and a main routine for testing.

Supplemental Note

Let's break down the line of code in detail:

```
public class AVLTree<T extends Comparable<T>>
```

This declaration defines a generic class named `AVLTree` which has a single type parameter, `T`. The part that might look complex at first glance is:

```
T extends Comparable<T>
```

What does `T extends Comparable<T>` mean?

1. **Generic Type Parameter** When you write `class AVLTree<T>`, `T` is a generic type parameter. This allows the class `AVLTree` to work with **any** type without being tied to a specific class or interface.
2. **Type Bound (`extends Comparable<T>`)** By saying `T extends Comparable<T>`, we're adding a **constraint** on what `T` can be:
   - It must be a type that implements the `Comparable<T>` interface.
   - In other words, `T` must be able to compare itself to another object of the same type `T`.

   In Java, `extends` in a generic type context can mean either "extends (a superclass)" **or** "implements (an interface)." Since `Comparable` is an interface, this part actually means "implements `Comparable<T>`."

3. **Why do we need `Comparable<T>` in an AVL Tree?** An AVL tree is a **self-balancing binary search tree**. For a binary search tree, we need to **compare** elements to decide:

   - Where to insert a new node (go left if the new value is smaller, go right if it's larger).
   - How to traverse or search in the tree.

   Because we need to compare elements, the type `T` used in this tree must be **comparable**. That's why the code requires `T extends Comparable<T>`—so that each node's value can be compared to another node's value.

What does the `Comparable` interface do? `Comparable<T>` is a built-in Java interface (from the `java.lang` package) that has a single method:

```
public interface Comparable<T> {
    int compareTo(T o);
}
```

Any class that implements `Comparable<T>` is expected to provide its own definition of the `compareTo(T o)` method. The `compareTo` method returns:

- A **negative integer** if the current object is "less than" the argument.
- **Zero** if they are "equal."
- A **positive integer** if the current object is "greater than" the argument.

This enables sorting and ordering. In a data structure like an AVL tree, you rely on the `compareTo` method to decide how to place or find items.

Example For instance, Java's built-in `String` class implements `Comparable<String>`:

```
public final class String implements Comparable<String>, CharSequence, ...
{
    // Implementation details

    @Override
    public int compareTo(String anotherString) {
        // returns negative, 0, or positive
        // based on lexicographical comparison
    }
}
```

So if you create an `AVLTree<String>`, you know that you can compare `String` objects to decide their order in the tree.

Similarly, Java's built-in numeric types like `Integer` implement `Comparable<Integer>`, so `AVLTree<Integer>` would also work.

Summary of `T extends Comparable<T>`

- It **constrains** the generic parameter `T` to types that implement `Comparable` of themselves.
- It **ensures** that any instance of `AVLTree<T>` can compare its elements (`T` objects) with each other.
- It's essential for data structures or algorithms that rely on comparing objects (e.g., sorting, searching, or balancing a tree).

By using `T extends Comparable<T>`, the `AVLTree` class is guaranteed that `T` has a `compareTo` method, making it possible to implement comparison-based logic correctly and safely within the tree's operations.

Alternative

```
// AVLTreeDemo.java
// Compile: javac AVLTreeDemo.java
// Run:     java AVLTreeDemo



class AVLTree {

    // Node as a nested subclass (inner class).
    private class Node {

        int key;
        String data;
        int height;
        Node left, right;

        Node(int key, String data)
        {
            this.key = key;
            this.data = data;
            this.height = 1; // leaf height = 1
        }
    }

    private Node root;

    // - - - - - - - - - - - - - - - - - - - - - - -
    // Public API
    // - - - - - - - - - - - - - - - - - - - - - - -
    public void insert(int key, String data)
    {
        /*
         Rotation rules (AVL rebalancing after INSERT):
           Let balance = height(left) - height(right)

           If balance > 1 (left heavy):
             - Left -Left (LL): key inserted into left subtree of left child
                 => rotateRight(node)
             - Left -Right (LR): key inserted into right subtree of left child
                 => rotateLeft(node.left) then rotateRight(node)

           If balance < -1 (right heavy):
             - Right -Right (RR): key inserted into right subtree of right child
                 => rotateLeft(node)
             - Right -Left (RL): key inserted into left subtree of right child
                 => rotateRight(node.right) then rotateLeft(node)
         */
        root = insert(root, key, data);
    }

    public void delete(int key)
    {
```

```
/*
 Rotation rules (AVL rebalancing after DELETE):
   Deletion can reduce subtree heights, so after
   removing a key, walk back up:
   - Update height
   - Compute balance = height(left) - height(right)
   - If unbalanced, use the child's balance to pick the case:

   If balance > 1 (left heavy):
     - If balance(left) >= 0  => LL case => rotateRight(node)
     - If balance(left) <  0  => LR case => rotateLeft(node.left),
        rotateRight(node)

   If balance < -1 (right heavy):
     - If balance(right) <= 0 => RR case => rotateLeft(node)
     - If balance(right) >  0 => RL case => rotateRight(node.right),
        rotateLeft(node)
 */
    root = delete(root, key);
}

public String find(int key)
{
    Node n = find(root, key);
    return (n == null) ? null : n.data;
}

public void preOrder()
{
    preOrder(root);
    System.out.println();
}

public void inOrder()
{
    inOrder(root);
    System.out.println();
}

public void postOrder()
{
    postOrder(root);
    System.out.println();
}



// - - - - - - - - - - - - - - - - - - - - - - -
// Core AVL internals
// - - - - - - - - - - - - - - - - - - - - - - -
private Node insert(Node node, int key, String data)
{
    if (node == null) {
```

```
            return new Node(key, data);
        }

        if (key < node.key) {
            node.left = insert(node.left, key, data);
        } else if (key > node.key) {
            node.right = insert(node.right, key, data);
        } else {
            // Duplicate key: update data (common practical policy).
            node.data = data;
            return node;
        }

        updateHeight(node);
        int balance = getBalance(node);

        // Left heavy
        if (balance > 1) {
            if (key < node.left.key) {
                // LL
                return rotateRight(node);
            } else {
                // LR
                node.left = rotateLeft(node.left);
                return rotateRight(node);
            }
        }

        // Right heavy
        if (balance < -1) {
            if (key > node.right.key) {
                // RR
                return rotateLeft(node);
            } else {
                // RL
                node.right = rotateRight(node.right);
                return rotateLeft(node);
            }
        }

        return node;
    }

    private Node delete(Node node, int key)
    {
        if (node == null) {
            return null;
        }

        if (key < node.key) {
            node.left = delete(node.left, key);
        } else if (key > node.key) {
            node.right = delete(node.right, key);
```

```
        } else {
            // Found node to delete.
            if (node.left == null || node.right == null) {
                Node child = (node.left != null) ? node.left : node.right;
                node = child; // could become null if no children
            } else {
                // Two children: replace with inorder successor (min in right subtree)
                Node succ = minValueNode(node.right);
                node.key = succ.key;
                node.data = succ.data;
                node.right = delete(node.right, succ.key);
            }
        }

        // If the tree had one node and we deleted it.
        if (node == null) {
            return null;
        }

        updateHeight(node);
        int balance = getBalance(node);

        // Left heavy
        if (balance > 1) {
            int leftBal = getBalance(node.left);
            if (leftBal >= 0) {
                // LL
                return rotateRight(node);
            } else {
                // LR
                node.left = rotateLeft(node.left);
                return rotateRight(node);
            }
        }

        // Right heavy
        if (balance < -1) {
            int rightBal = getBalance(node.right);
            if (rightBal <= 0) {
                // RR
                return rotateLeft(node);
            } else {
                // RL
                node.right = rotateRight(node.right);
                return rotateLeft(node);
            }
        }

        return node;
    }

    private Node find(Node node, int key)
    {
```

```
        while (node != null) {
            if (key < node.key) {
                node = node.left;
            } else if (key > node.key) {
                node = node.right;
            } else {
                return node;
            }
        }
        return null;
    }

    private Node minValueNode(Node node)
    {
        Node cur = node;
        while (cur.left != null) {
            cur = cur.left;
        }
        return cur;
    }

    // - - - - - - - - - - - - - - - - - - - - - - -
    // Rotations + helpers
    // - - - - - - - - - - - - - - - - - - - - - - -
    private int height(Node n)
    {
        return (n == null) ? 0 : n.height;
    }

    private void updateHeight(Node n)
    {
        n.height = 1 + Math.max(height(n.left), height(n.right));
    }

    private int getBalance(Node n)
    {
        return (n == null) ? 0 : height(n.left) - height(n.right);
    }

    // Right rotation:
    //        y                 x
    //       / \               / \
    //      x   T3   =>       T1  y
    //     / \                   / \
    //    T1  T2                T2  T3
    private Node rotateRight(Node y)
    {
        Node x = y.left;
        Node t2 = x.right;

        x.right = y;
        y.left = t2;
```

```
        updateHeight(y);
        updateHeight(x);

        return x;
    }

    // Left rotation:
    //      x                    y
    //     / \                  / \
    //    T1  y      =>        x  T3
    //       / \              / \
    //      T2  T3           T1  T2
    private Node rotateLeft(Node x)
    {
        Node y = x.right;
        Node t2 = y.left;

        y.left = x;
        x.right = t2;

        updateHeight(x);
        updateHeight(y);

        return y;
    }

    // - - - - - - - - - - - - - - - - - - - - - -
    // Traversals (prints key:data)
    // - - - - - - - - - - - - - - - - - - - - - -
    private void preOrder(Node n)
    {
        if (n == null) {
            return;
        }
        System.out.print(n.key + ":" + n.data + "  ");
        preOrder(n.left);
        preOrder(n.right);
    }

    private void inOrder(Node n)
    {
        if (n == null) {
            return;
        }
        inOrder(n.left);
        System.out.print(n.key + ":" + n.data + "  ");
        inOrder(n.right);
    }

    private void postOrder(Node n)
    {
        if (n == null) {
            return;
```

```
        }
        postOrder(n.left);
        postOrder(n.right);
        System.out.print(n.key + ":" + n.data + "  ");
    }



    //  version (top -down).
    public void prettyPrint()
    {
        if (root == null) {
            System.out.println("(empty)");
            return;
        }

        int h = height(root);
        java.util.List<Node> level = new java.util.ArrayList<>();
        level.add(root);

        for (int depth = 1; depth <= h; depth++) {
            int leading = (1 << (h - depth + 1)) - 2;  // spaces before first node
            int between = (1 << (h - depth + 2)) - 2;  // spaces between nodes

            // Print node keys line
            printSpaces(leading);

            java.util.List<Node> next = new java.util.ArrayList<>();
            boolean anyNonNull = false;

            for (int i = 0; i < level.size(); i++) {
                Node n = level.get(i);

                if (n == null) {
                    System.out.print("  "); // placeholder width
                    next.add(null);
                    next.add(null);
                } else {
                    String s = String.valueOf(n.key);
                    // keep width roughly constant for small keys; pad to 2 chars
                    if (s.length() == 1) {
                        s = " " + s;
                    }
                    System.out.print(s);

                    next.add(n.left);
                    next.add(n.right);

                    if (n.left != null || n.right != null) {
                        anyNonNull = true;
                    }
                }
```

```
            if (i < level.size() - 1) {
                printSpaces(between);
            }
        }
        System.out.println();

        // Print connector line (slashes) except last level
        if (depth < h) {
            int slashLeading = leading - 1;
            int slashBetween = between - 2;

            if (slashLeading < 0) {
                slashLeading = 0;
            }
            if (slashBetween < 0) {
                slashBetween = 0;
            }

            printSpaces(slashLeading);

            for (int i = 0; i < level.size(); i++) {
                Node n = level.get(i);

                if (n == null) {
                    System.out.print("  ");
                } else {
                    char left = (n.left != null) ? '/' : ' ';
                    char right = (n.right != null) ? '\\' : ' ';
                    System.out.print(left);
                    System.out.print(right);
                }

                if (i < level.size() - 1) {
                    printSpaces(slashBetween);
                }
            }
            System.out.println();
        }

        level = next;

        // If the next level is entirely null, stop early.
        if (!anyNonNull && depth < h) {
            // still printed current level; no need to print empty lower levels
            break;
        }
    }
}

private void printSpaces(int count)
{
    for (int i = 0; i < count; i++) {
        System.out.print(' ');
```

```
        }
    }
}

public class AVLTreeDemo {

    public static void main(String[] args)
    {
        AVLTree tree = new AVLTree();

        // Insert test data (key, data).
        tree.insert(30, "thirty");
        tree.insert(20, "twenty");
        tree.insert(40, "forty");
        tree.insert(10, "ten");
        tree.insert(25, "twenty -five");
        tree.insert(35, "thirty -five");
        tree.insert(50, "fifty");
        tree.insert(5, "five");
        tree.insert(15, "fifteen");
        tree.insert(27, "twenty -seven");

        System.out.println("Pretty print after inserts:");
        tree.prettyPrint();

        System.out.println("\nInOrder (sorted by key):");
        tree.inOrder();

        System.out.println("PreOrder:");
        tree.preOrder();

        System.out.println("PostOrder:");
        tree.postOrder();

        int lookFor = 25;
        String found = tree.find(lookFor);
        System.out.println("\nFind " + lookFor + ": " + (found == null ? "(not found)" : found

        System.out.println("\nDelete 20 (has two children in this dataset):");
        tree.delete(20);
        tree.prettyPrint();

        System.out.println("\nDelete 5 (leaf):");
        tree.delete(5);
        tree.prettyPrint();

        System.out.println("\nDelete 40 (has children):");
        tree.delete(40);
        tree.prettyPrint();

        System.out.println("\nInOrder after deletions:");
        tree.inOrder();
```

```
        System.out.println("\n\nEnd\n");
    }
}
```

0.3.8 Red Black Tree

A **Red-Black Tree** is a type of **self-balancing binary search tree** in computer science. Let's delve into its fascinating properties:

1. **Structure and Color**:
   - Each node in a Red-Black Tree has an extra bit that represents its color (either **red** or **black**).
   - These color bits are used to ensure that the tree remains approximately balanced during insertions and deletions.
   - The tree's structure guarantees that operations like searching, insertion, and deletion complete within a known time.
2. **Properties of Red-Black Trees**:
   - **Root Property**: The root node is always black.
   - **External Property**: Every leaf (which is a NULL child of a node) is black.
   - **Internal Property**: The children of a red node are black. Thus, a red node's possible parent is a black node.
   - **Depth Property**: All leaves have the same black depth.
   - **Path Property**: Every simple path from the root to a descendant leaf node contains the same number of black nodes.
3. **Why Red-Black Trees?**:
   - Most Binary Search Tree (BST) operations (e.g., search, max, min, insert, delete) take O(h) time, where h is the height of the BST.
   - If we ensure that the tree's height remains O(log n) after every insertion and deletion, we guarantee an upper bound of O(log n) for all these operations.
   - The height of a Red-Black tree is always O(log n), where n is the number of nodes in the tree.

Insertion Process

1. **Initial Setup**:
   - When inserting a new node, it is initially colored **red**.
   - If the tree is empty, the new node becomes the **root** (colored **black**).
   - Otherwise, we insert the node as a **leaf** (colored **red**).
2. **Insertion Algorithm**:
   - Perform a standard **BST insertion** (similar to a regular binary search tree).
   - Color the newly inserted nodes as **RED**.
   - If the newly inserted node is the **root**, change its color to **BLACK** (increasing the black height of the tree by 1).
3. **Balancing Steps**:
   - If the parent of the newly inserted node is not BLACK and the node is not the root :
     - Case 1: If the uncle of the node is RED :
       - Change the color of the parent and uncle to **BLACK**.
       - Change the color of the **grandparent** to **RED**.
       - Repeat steps 2 and 3 for the new **grandparent**.
     - Case 2: If the uncle is BLACK :
       - There are four possible configurations (similar to AVL Tree rotations):
         - **Left Left Case (LL rotation)**
         - **Left Right Case (LR rotation)**
         - **Right Right Case (RR rotation)**

        - **Right Left Case (RL rotation)**
    - After these rotations, re-color according to the rotation case.

4. **Re-coloring after Rotations**:
    - For **Left Left Case** and **Right Right Case**, swap colors of **grandparent** and **parent** after rotations.
    - For **Left Right Case** and **Right Left Case**, swap colors of **grandparent** and **inserted node** after rotations.

Deletion Process

1. **Initial Setup**:
    - When deleting a node, it can fall into three cases:
        - The node has **no children**: Simply remove it and update the parent node.
        - The node has **one child**: Replace the node with its child.
        - The node has **two children**: Replace the node with its **in-order successor**, which is the leftmost node in the right subtree.

2. **Deletion Algorithm**:
    - Perform a standard **BST delete** operation (similar to regular binary search tree deletion).
    - Color the newly inserted nodes as **RED**.
    - If the deleted node was **BLACK**, we introduce the concept of **double black** (marking the replacement child as double black).

3. **Balancing Steps**:
    - After deletion, the Red-Black Tree properties might be violated.
    - To restore these properties, perform color changes and rotations similar to those during insertion, but with different conditions.
    - The main goal is to convert the **double black** node back to a **single black** node.
    - The cases for restructuring and recoloring are determined by the color of the **sibling** of the double black node.

RED BLACK Class

```
/*
 * To change this license header, choose License Headers in Project Properties.
 * To change this template file, choose Tools | Templates
 * and open the template in the editor.
 */


import java.util.Random;

public class RedBlackTree
{

    //An inner nested class can be declared private and only accessed
    //within the class
    //A inner nested class can be declared static and can be accessed
    //from outside the class.
    //Example in main or other function outside of class
    // RedBlackTree.Node tempNode = new RedBlackTree.Node();

    static class Node
```

```
{

    int key; // holds the key
    Node parent; // pointer to the parent
    Node left; // pointer to left child
    Node right; // pointer to right child
    int color; // 1 . Red, 0 . Black

    //some data - in this case string
    String sdata;
}

private Node root;
private Node TNULL;

private void preOrderHelper(Node node)
{
    if (node != TNULL)
    {
        System.out.print(node.key + " ");
        preOrderHelper(node.left);
        preOrderHelper(node.right);
    }
}

private void inOrderHelper(Node node)
{
    if (node != TNULL)
    {
        inOrderHelper(node.left);
        System.out.print(node.key + " ");
        inOrderHelper(node.right);
    }
}

private void postOrderHelper(Node node)
{
    if (node != TNULL)
    {
        postOrderHelper(node.left);
        postOrderHelper(node.right);
        System.out.print(node.key + " ");
    }
}

void preOrder()
{
    System.out.println("\nPreOder");
    Node node = root;
    if (node != TNULL)
    {
        System.out.print(" **" + node.key + "** ");
        preOrderHelper(node.left);
```

```
            preOrderHelper(node.right);
        }
        System.out.println("\n - - - - - - - - - - - - - - - - - -\n");
    }

    // In -Order traversal
    // Left Subtree . Node . Right Subtree
    public void inOrder()
    {
        System.out.println("\ninOder");
        Node node = root;
        if (node != TNULL)
        {
            inOrderHelper(node.left);
            System.out.print(" **" + node.key + "** ");
            inOrderHelper(node.right);
        }
        System.out.println("\n - - - - - - - - - - - - - - - - - -\n");
    }

    // Post -Order traversal
    // Left Subtree . Right Subtree . Node
    public void postOrder()
    {
        System.out.println("\npostOder");
        Node node = root;
        if (node != TNULL)
        {
            postOrderHelper(node.left);
            postOrderHelper(node.right);
            System.out.print(" **" + node.key + "** ");
        }
        System.out.println("\n - - - - - - - - - - - - - - - - - -\n");
    }

    // search the tree for the key k
    // and return the corresponding node
    public Node searchTree(int k)
    {
        return searchTreeHelper(this.root, k);
    }

    private Node searchTreeHelper(Node node, int key)
    {
        if (node == TNULL || key == node.key)
        {
            //if(node == null)

            return node;
        }

        if (key < node.key)
        {
```

```
            return searchTreeHelper(node.left, key);
        }
        return searchTreeHelper(node.right, key);
    }

    // fix the rb tree modified by the delete operation
    private void fixDelete(Node x)
    {
        Node s;
        while (x != root && x.color == 0)
        {
            if (x == x.parent.left)
            {
                s = x.parent.right;
                if (s.color == 1)
                {
                    // case 3.1
                    s.color = 0;
                    x.parent.color = 1;
                    leftRotate(x.parent);
                    s = x.parent.right;
                }

                if (s.left.color == 0 && s.right.color == 0)
                {
                    // case 3.2
                    s.color = 1;
                    x = x.parent;
                } else
                {
                    if (s.right.color == 0)
                    {
                        // case 3.3
                        s.left.color = 0;
                        s.color = 1;
                        rightRotate(s);
                        s = x.parent.right;
                    }

                    // case 3.4
                    s.color = x.parent.color;
                    x.parent.color = 0;
                    s.right.color = 0;
                    leftRotate(x.parent);
                    x = root;
                }
            } else
            {
                s = x.parent.left;
                if (s.color == 1)
                {
                    // case 3.1
                    s.color = 0;
```

```
                x.parent.color = 1;
                rightRotate(x.parent);
                s = x.parent.left;
            }

            if (s.right.color == 0 && s.right.color == 0)
            {
                // case 3.2
                s.color = 1;
                x = x.parent;
            } else
            {
                if (s.left.color == 0)
                {
                    // case 3.3
                    s.right.color = 0;
                    s.color = 1;
                    leftRotate(s);
                    s = x.parent.left;
                }

                // case 3.4
                s.color = x.parent.color;
                x.parent.color = 0;
                s.left.color = 0;
                rightRotate(x.parent);
                x = root;
            }
        }
    }
    x.color = 0;
}

private void rbTransplant(Node u, Node v)
{
    if (u.parent == null)
    {
        root = v;
    } else if (u == u.parent.left)
    {
        u.parent.left = v;
    } else
    {
        u.parent.right = v;
    }
    v.parent = u.parent;
}

private void deleteNodeHelper(Node node, int key)
{
    // find the node containing key
    Node z = TNULL;
    Node x, y;
```

```
while (node != TNULL)
{
    if (node.key == key)
    {
        z = node;
    }

    if (node.key <= key)
    {
        node = node.right;
    } else
    {
        node = node.left;
    }
}

if (z == TNULL)
{
    System.out.println("Couldn't find key in the tree");
    return;
}

y = z;
int yOriginalColor = y.color;
if (z.left == TNULL)
{
    x = z.right;
    rbTransplant(z, z.right);
} else if (z.right == TNULL)
{
    x = z.left;
    rbTransplant(z, z.left);
} else
{
    y = minimum(z.right);
    yOriginalColor = y.color;
    x = y.right;
    if (y.parent == z)
    {
        x.parent = y;
    } else
    {
        rbTransplant(y, y.right);
        y.right = z.right;
        y.right.parent = y;
    }

    rbTransplant(z, y);
    y.left = z.left;
    y.left.parent = y;
    y.color = z.color;
}
if (yOriginalColor == 0)
```

```
        {
            fixDelete(x);
        }
    }

    // fix the red -black tree
    private void fixInsert(Node k)
    {
        Node u;
        while (k.parent.color == 1)
        {
            if (k.parent == k.parent.parent.right)
            {
                u = k.parent.parent.left; // uncle
                if (u.color == 1)
                {
                    // case 3.1
                    u.color = 0;
                    k.parent.color = 0;
                    k.parent.parent.color = 1;
                    k = k.parent.parent;
                } else
                {
                    if (k == k.parent.left)
                    {
                        // case 3.2.2
                        k = k.parent;
                        rightRotate(k);
                    }
                    // case 3.2.1
                    k.parent.color = 0;
                    k.parent.parent.color = 1;
                    leftRotate(k.parent.parent);
                }
            } else
            {
                u = k.parent.parent.right; // uncle

                if (u.color == 1)
                {
                    // mirror case 3.1
                    u.color = 0;
                    k.parent.color = 0;
                    k.parent.parent.color = 1;
                    k = k.parent.parent;
                } else
                {
                    if (k == k.parent.right)
                    {
                        // mirror case 3.2.2
                        k = k.parent;
                        leftRotate(k);
                    }
```

```
                // mirror case 3.2.1
                k.parent.color = 0;
                k.parent.parent.color = 1;
                rightRotate(k.parent.parent);
            }
        }
        if (k == root)
        {
            break;
        }
    }
    root.color = 0;
}

private void printHelper(Node root, String indent, boolean last)
{
    // print the tree structure on the screen
    if (root != TNULL)
    {
        System.out.print(indent);
        if (last)
        {
            System.out.print("R - - - -");
            indent += "     ";
        } else
        {
            System.out.print("L - - - -");
            indent += "|    ";
        }

        String sColor = root.color == 1 ? "RED" : "BLACK";
        System.out.println(root.key + " -" + root.sdata
                + "(" + sColor + ")");
        printHelper(root.left, indent, false);
        printHelper(root.right, indent, true);
    }
}

public RedBlackTree()
{
    TNULL = new Node();
    TNULL.color = 0;
    TNULL.left = null;
    TNULL.right = null;
    root = TNULL;
}

// find the node with the minimum key
public Node minimum(Node node)
{
    while (node.left != TNULL)
    {
        node = node.left;
```

```
        }
        return node;
    }

    // find the node with the maximum key
    public Node maximum(Node node)
    {
        while (node.right != TNULL)
        {
            node = node.right;
        }
        return node;
    }

    // find the successor of a given node
    public Node successor(Node x)
    {
        // if the right subtree is not null,
        // the successor is the leftmost node in the
        // right subtree
        if (x.right != TNULL)
        {
            return minimum(x.right);
        }

        // else it is the lowest ancestor of x whose
        // left child is also an ancestor of x.
        Node y = x.parent;
        while (y != TNULL && x == y.right)
        {
            x = y;
            y = y.parent;
        }
        return y;
    }

    // find the predecessor of a given node
    public Node predecessor(Node x)
    {
        // if the left subtree is not null,
        // the predecessor is the rightmost node in the
        // left subtree
        if (x.left != TNULL)
        {
            return maximum(x.left);
        }

        Node y = x.parent;
        while (y != TNULL && x == y.left)
        {
            x = y;
            y = y.parent;
        }
```

```
    return y;
}

// rotate left at node x
public void leftRotate(Node x)
{
    Node y = x.right;
    x.right = y.left;
    if (y.left != TNULL)
    {
        y.left.parent = x;
    }
    y.parent = x.parent;
    if (x.parent == null)
    {
        this.root = y;
    } else if (x == x.parent.left)
    {
        x.parent.left = y;
    } else
    {
        x.parent.right = y;
    }
    y.left = x;
    x.parent = y;
}

// rotate right at node x
public void rightRotate(Node x)
{
    Node y = x.left;
    x.left = y.right;
    if (y.right != TNULL)
    {
        y.right.parent = x;
    }
    y.parent = x.parent;
    if (x.parent == null)
    {
        this.root = y;
    } else if (x == x.parent.right)
    {
        x.parent.right = y;
    } else
    {
        x.parent.left = y;
    }
    y.right = x;
    x.parent = y;
}

// insert the key to the tree in its appropriate position
```

```
// and fix the tree
public void insert(int key, String sdata)
{
    // Ordinary Binary Search Insertion
    Node node = new Node();
    node.parent = null;
    node.key = key;

    node.sdata = sdata;

    node.left = TNULL;
    node.right = TNULL;
    node.color = 1; // new node must be red

    Node y = null;
    Node x = this.root;

    while (x != TNULL)
    {
        y = x;
        if (node.key < x.key)
        {
            x = x.left;
        } else
        {
            x = x.right;
        }
    }

    // y is parent of x
    node.parent = y;
    if (y == null)
    {
        root = node;
    } else if (node.key < y.key)
    {
        y.left = node;
    } else
    {
        y.right = node;
    }

    // if new node is a root node, simply return
    if (node.parent == null)
    {
        node.color = 0;
        return;
    }

    // if the grandparent is null, simply return
    if (node.parent.parent == null)
    {
        return;
```

```
        }

        // Fix the tree
        fixInsert(node);
    }

    public Node getRoot()
    {
        return this.root;
    }

    // delete the node from the tree
    public void deleteNode(int key)
    {
        deleteNodeHelper(this.root, key);
    }

    // print the tree structure on the screen
    public void prettyPrint()
    {
        printHelper(this.root, "", true);
    }

   // static RedBlackTree bst = new RedBlackTree();

    void insertRandom()
    {
        int amt = 12;
        int amts = 5;
        int amtNodes, amtLetters;
        int[] keys;

        int selectKey = -1;
        int selectIndex = -1;
        boolean run = true;

        String numberOrder = "";
        int numberOrderCount = 0;

        Random RNG = new Random();
        amtNodes = RNG.nextInt(amt);
        //amtNodes = 15; //hard code the amount
        amtLetters = 3;

        System.out.println(amtNodes);
        keys = new int[amtNodes];

        //load keys
        for (int i = 0; i < keys.length; i++)
        {
            keys[i] = i;
        }
```

```
        for (int i = 0; i < amtNodes; i++)
        {
            //create random letter patterns
            String ld = "";
            for (int j = 0; j < amtLetters; j++)
            {
                ld = ld + (char) (RNG.nextInt(25) + 65);
            }

            //pick random key
            int cntr = 0;
            selectIndex = RNG.nextInt(amtNodes);

            selectKey = keys[selectIndex];
            run = true;
            while (run)
            {

                if (selectKey > -1)
                {
                    run = false;
                } else
                {
                    selectIndex = selectIndex + 1;
                    if (selectIndex >= amtNodes)
                    {
                        selectIndex = 0;
                    }

                }
                selectKey = keys[selectIndex];

            }
            //System.out.print(selectKey + " ");

            numberOrder = numberOrder + selectKey + " ";
            numberOrderCount++;
            if (numberOrderCount > 20)
            {
                numberOrder = numberOrder + "\n";
                numberOrderCount = 0;

            }

            insert(selectKey, ld);
            keys[selectKey] = -1;
        }
        System.out.println("\n" + numberOrder + "\n");
        System.out.println("");

    }

}
```

0.3.9 BST AVL Red Black Tree Comparisons

Here's a comparative table of **Binary Search Trees (BST)**, **AVL Trees**, and **Red-Black Trees**, highlighting their differences and similarities:

Feature	Binary Search Tree (BST)	AVL Tree	Red-Black Tree
Definition	A hierarchical data structure where each node has at most two children, and the left child is smaller while the right child is greater.	A self-balancing BST that maintains a strict height balance using rotations and balance factors.	A self-balancing BST that enforces balance using a color property (red or black nodes) and specific rebalancing rules.
Balance	Not guaranteed to be balanced; can become skewed (degraded to linked list).	Strictly balanced: the height difference between left and right subtrees is at most 1.	Loosely balanced: ensures logarithmic height but allows more imbalance than AVL.
Height	Can be as bad as **O(n)** in the worst case (skewed tree).	Always **O(log n)** due to strict balancing.	Always **O(log n)** but allows slightly more imbalance than AVL.
Rotations	No rotations are performed unless explicitly balanced.	Rotations occur frequently due to strict balance enforcement.	Rotations occur less frequently than AVL but are used to maintain red-black properties.
Insertion Complexity	**O(n)** in the worst case due to possible skewness.	**O(log n)** due to rebalancing with rotations.	**O(log n)** with fewer rotations compared to AVL.
Deletion Complexity	**O(n)** worst case.	**O(log n)** with rebalancing after deletion.	**O(log n)** with rebalancing but fewer rotations than AVL.
Search Complexity	**O(n)** in the worst case (if unbalanced).	**O(log n)** since tree remains strictly balanced.	**O(log n)** due to maintained balance.
Memory Overhead	Minimal, just pointers to child nodes.	Higher than BST due to storing balance factors at each node.	More than BST but less than AVL due to storing color bits.
Use Cases	Simple applications where balancing is not required.	Used in applications requiring fast lookups and strict balance (e.g., database indexing).	Common in memory allocators, operating systems, and complex data structures (e.g., Linux kernel, TreeMap in Java).
Preferred When	Insertions/deletions are rare, and tree balance is not a concern.	Lookups and searches need to be very fast, even at the cost of expensive insertions.	A balance between insertion speed and search efficiency is required.

Summary:

- **BSTs** are simple but can degrade into a linear structure, making operations inefficient.
- **AVL Trees** maintain a strict balance, ensuring fast lookups but with higher rotation costs.

- **Red-Black Trees** provide a good balance between insertion efficiency and search performance, making them widely used in practical applications like databases and OS kernels.

Choosing between a **Binary Search Tree (BST)**, **AVL Tree**, and **Red-Black Tree** depends on the specific requirements of the application. Below is a guideline on when to use each:

1. Binary Search Tree (BST)

Use when:

- **Data is inserted in a random manner**, reducing the likelihood of a skewed tree.
- **Balancing is not critical**, and you prefer a simple implementation.
- **Read and write operations are infrequent**, so the risk of degeneration to **O(n)** performance is low.
- **You need a tree structure for educational or conceptual purposes** rather than performance-oriented applications.

Example Use Cases:

- **Small datasets** where tree height isn't a major concern.
- **Simple dictionary or symbol table implementations** (if the data remains balanced).
- **Compiler design** (e.g., abstract syntax trees in compilers).
- **Basic educational tools** to demonstrate tree traversal and recursion.

---

2. AVL Tree

Use when:

- **Search operations are frequent** and must be as fast as possible.
- **Strict balancing is necessary** to guarantee **O(log n)** time complexity for all operations.
- **You can afford extra space** (due to balance factor storage) and a slightly higher insertion/deletion cost.
- **Fast lookups are more critical than insertions/deletions** (e.g., read-heavy workloads).

Example Use Cases:

- **Databases that require many searches** but relatively fewer inserts (e.g., in-memory key-value stores).
- **Applications that require consistent log-time lookups** (e.g., indexing large datasets).
- **Geo-spatial applications** where precise balancing affects query performance.
- **Memory-constrained embedded systems** where lookup efficiency is crucial.

---

3. Red-Black Tree

Use when:

- **Insertions and deletions occur frequently**, and you need a balance between speed and efficiency.
- **You need a self-balancing tree** but can tolerate a slightly less strict balance than AVL trees.
- **Performance is needed in general-purpose programming**, where a slight imbalance can be acceptable for faster insertions.
- **Memory overhead should be lower** than AVL trees (RB trees store only one extra bit per node for color).

Example Use Cases:

- **Operating system kernels (e.g., Linux scheduler, memory management)**.
- **Database indexing** (e.g., Red-Black Trees are used in PostgreSQL for some indexing structures).
- **Balanced associative containers in programming languages** (e.g., **TreeMap in Java**, **std::map in C++**).
- **Networking applications**, where frequent insertions and deletions happen (e.g., routing table lookups).

Decision Summary:

Scenario	Best Choice
Simple implementation, small dataset, or learning purposes	BST
Fast lookups with rare insertions/deletions (read-heavy workload)	AVL Tree
Frequent insertions/deletions with a balance of efficiency	Red-Black Tree
OS kernels, memory allocators, databases, or language libraries	Red-Black Tree
If data insertion order is random and balanced naturally	BST (unless performance degrades)
If data is likely to be inserted in sorted order (worst case for BSTs)	AVL or Red-Black

0.3.10 Splay Trees

Splay Trees

A **splay tree** is a type of self-adjusting binary search tree (BST) where recently accessed elements are moved to the root of the tree using a process called *splaying*. This helps ensure that frequently accessed elements are quicker to access.

Who Invented It:

- Splay trees were invented by **Daniel Sleator** and **Robert Tarjan** in 1985. The invention was part of their research on self-adjusting data structures.

When Was It Invented:

- The splay tree was introduced in **1985**.

Major Uses:

- **Dynamic sets**: Splay trees are used to maintain a dynamic set of elements that can support a variety of search and update operations.
- **Memory-efficient**: Since splay trees do not require additional space for balancing data like AVL trees or Red-Black trees, they can be more memory-efficient.
- **Efficient in practice for certain workloads**: Splay trees perform well when there are repeated accesses to a small subset of elements (i.e., the access pattern exhibits locality).

Typical Applications:

- **Cache systems**: Frequently accessed items bubble to the front.
- **Text editors**: Recently opened files or commands can be organized in a splay list.
- **Autocomplete lists**: Recently or frequently used search terms are moved to the front.

Insert Function:

- **Insertion in a splay tree** is similar to that in a regular binary search tree. The node is inserted following the normal BST rules. However, after insertion, the tree is "splayed" to bring the newly inserted node to the root.

```
public void insert(int value) {
    root = insertRec(root, value);
    splay(root, value);
}

private Node insertRec(Node root, int value) {
    if (root == null) {
        root = new Node(value);
        return root;
    }

    if (value < root.data) {
        root.left = insertRec(root.left, value);
    } else if (value > root.data) {
        root.right = insertRec(root.right, value);
    }

    return root;
}
```

Delete Function:

- **Deletion** is performed in the same way as in a normal binary search tree. After finding the node to delete, it is removed, and then the tree is splayed to restore the tree's structure.

```
public void delete(int value) {
    root = deleteRec(root, value);
}

private Node deleteRec(Node root, int value) {
    if (root == null) return root;

    if (value < root.data) {
        root.left = deleteRec(root.left, value);
    } else if (value > root.data) {
        root.right = deleteRec(root.right, value);
    } else {
        // node with value found
        if (root.left == null) return root.right;
        else if (root.right == null) return root.left;
    }

    return root;
}
```

Find Function:

- The **find function** searches for a value in the tree, and if the value is found, it performs a splay operation to bring that value to the root. If the value is not found, the tree remains unchanged.

```
public boolean find(int value) {
    root = splay(root, value);
    return (root != null && root.data == value);
}
```

Rules for Rotations:

- Splay trees use three types of rotations to adjust the tree during splaying:
  1. **Zig**: When the node is the child of the root.
  2. **Zig-Zig**: When the node is the left or right child of a left or right child (two-level deep).
  3. **Zig-Zag**: When the node is the left child of the right child or the right child of the left child.

Rotations:

- **Single Rotation (Zig)**: If the node is the child of the root, perform a single rotation (right or left depending on the situation).
- **Double Rotation (Zig-Zig or Zig-Zag)**: In these cases, two rotations are needed. They help in cases where the tree has more than one level to adjust.

```
private Node splay(Node root, int value) {
    if (root == null || root.data == value) return root;

    if (value < root.data) {
        if (root.left == null) return root;
```

```
        if (value < root.left.data) {
            root.left.left = splay(root.left.left, value);
            root = rotateRight(root);
        } else if (value > root.left.data) {
            root.left.right = splay(root.left.right, value);
            if (root.left.right != null) {
                root.left = rotateLeft(root.left);
            }
        }
        return (root.left == null) ? root : rotateRight(root);
    } else {
        if (root.right == null) return root;
        if (value > root.right.data) {
            root.right.right = splay(root.right.right, value);
            root = rotateLeft(root);
        } else if (value < root.right.data) {
            root.right.left = splay(root.right.left, value);
            if (root.right.left != null) {
                root.right = rotateRight(root.right);
            }
        }
        return (root.right == null) ? root : rotateLeft(root);
    }
}
```

Java Program Demonstrating Insert, Find, Delete, and Pretty Print:

```
package inst_splay_2025;

class SplayTree {

    private Node root;

    // Inner class representing a node in the splay tree
    class Node {

        int data;
        Node left, right;

        public Node(int item)
        {
            data = item;
            left = right = null;
        }
    }

    // Inserts a value into the tree
    public void insert(int value)
    {
        if (root == null) {
            root = new Node(value);
            return;
        }
```

```
        // Splay the tree to bring the value (or closest) to the root
        root = splay(root, value);

        // If the value already exists, no need to insert
        if (root.data == value) {
            return;
        }

        Node newNode = new Node(value);

        // If the new value is smaller, make the new node root and adjust pointers
        if (value < root.data) {
            newNode.right = root;
            newNode.left = root.left;
            root.left = null;
        } else { // If the new value is greater
            newNode.left = root;
            newNode.right = root.right;
            root.right = null;
        }

        root = newNode; // Update root
    }

    // Searches for a value in the tree
    public boolean find(int value)
    {
        root = splay(root, value);
        return (root != null && root.data == value);
    }

    // Deletes a node with the given value
    public void delete(int value)
    {
        if (root == null) {
            return;
        }

        // Splay the tree so that the value (or closest) is at the root
        root = splay(root, value);

        if (root.data != value) {
            return; // If not found, do nothing
        }
        if (root.left == null) {
            root = root.right;
        } else {
            Node temp = root;
            // Splay the largest node in the left subtree to make it new root
            root = splay(root.left, value);
            root.right = temp.right;
        }
    }
```

```
// Right rotation (Zig rotation)
private Node rotateRight(Node root)
{
    if (root == null || root.left == null) {
        return root;
    }
    Node newRoot = root.left;
    root.left = newRoot.right;
    newRoot.right = root;
    return newRoot;
}

// Left rotation (Zag rotation)
private Node rotateLeft(Node root)
{
    if (root == null || root.right == null) {
        return root;
    }
    Node newRoot = root.right;
    root.right = newRoot.left;
    newRoot.left = root;
    return newRoot;
}

/*
 * Splay Operation: Moves the given value to the root if present, or the closest value.
 *
 * Splay tree follows these rules for rotations:
 *
 * - Zig Rotation (Single Right): When the value is in the left child of the root.
 * - Zag Rotation (Single Left): When the value is in the right child of the root.
 * - Zig -Zig (Double Right Rotation): When the value is in the left child of the left chi
 * - Zag -Zag (Double Left Rotation): When the value is in the right child of the right ch
 * - Zig -Zag (Left -Right Rotation): When the value is in the right child of the left chi
 * - Zag -Zig (Right -Left Rotation): When the value is in the left child of the right chi
 */
private Node splay(Node root, int value)
{
    if (root == null || root.data == value) {
        return root;
    }

    if (value < root.data) {
        if (root.left == null) {
            return root;
        }

        // Zig -Zig case (left -left)
        if (value < root.left.data) {
            root.left.left = splay(root.left.left, value);
            root = rotateRight(root);
        } // Zig -Zag case (left -right)
```

```
            else if (value > root.left.data) {
                root.left.right = splay(root.left.right, value);
                if (root.left.right != null) {
                    root.left = rotateLeft(root.left);
                }
            }

            // Perform a final Zig rotation
            return (root.left == null) ? root : rotateRight(root);
        } else {
            if (root.right == null) {
                return root;
            }

            // Zag -Zag case (right -right)
            if (value > root.right.data) {
                root.right.right = splay(root.right.right, value);
                root = rotateLeft(root);
            } // Zag -Zig case (right -left)
            else if (value < root.right.data) {
                root.right.left = splay(root.right.left, value);
                if (root.right.left != null) {
                    root.right = rotateRight(root.right);
                }
            }

            // Perform a final Zag rotation
            return (root.right == null) ? root : rotateLeft(root);
        }
    }

    // Prints the tree structure
    public void prettyPrint(Node root, String indent)
    {
        if (root == null) {
            return;
        }
        prettyPrint(root.right, indent + "   ");
        System.out.println(indent + root.data);
        prettyPrint(root.left, indent + "   ");
    }

    // Initiates tree printing
    public void printTree()
    {
        prettyPrint(root, "");
    }
}

// Driver class to test the Splay Tree implementation
public class Inst_splay_2025 {

    public static void main(String[] args)
```

```
    {
        SplayTree tree = new SplayTree();

        tree.insert(10);
        tree.insert(20);
        tree.insert(5);
        tree.insert(55);
        tree.insert(60);
        tree.insert(65);
        tree.insert(70);
        tree.insert(75);
        tree.insert(30);
        tree.insert(15);
        tree.insert(25);

        tree.printTree();
        System.out.println("\n - - - - - - - - - - - - - - - -\n");

        tree.find(20);
        tree.printTree();
        System.out.println("\n - - - - - - - - - - - - - - - -\n");

        tree.find(5);
        tree.printTree();
        System.out.println("\n - - - - - - - - - - - - - - - -\n");

        tree.find(65);
        tree.printTree();
        System.out.println("\n - - - - - - - - - - - - - - - -\n");

        tree.delete(10);
        tree.printTree();
        System.out.println("\n - - - - - - - - - - - - - - - -\n");

        tree.delete(30);
        tree.printTree();
        System.out.println("\n - - - - - - - - - - - - - - - -\n");

        tree.delete(15);
        tree.printTree();
        System.out.println("\n - - - - - - - - - - - - - - - -\n");

        tree.delete(5);
        tree.printTree();
    }
}
```

Explanation of Rotation Rules:

1. **Zig (Single Rotation)**: If the node to be accessed is the left or right child of the root, perform a single rotation (either left or right).

2. **Zig-Zig (Double Rotation)**: If the node is the left or right child of a left or right child (two levels deep), perform two rotations. This optimizes the tree by rotating both the node and its

parent.

3. **Zig-Zag (Double Rotation)**: If the node is the left child of the right child, or the right child of the left child (mixed child directions), perform a rotation on the child followed by a rotation on the node itself.

This program demonstrates how to use splay tree operations and includes helpful comments and explanations of the rotation rules. Each rotation helps ensure the tree remains balanced by moving frequently accessed nodes towards the root.

Splay using for loops

This version avoids the recursive calls and uses loops to handle the tree traversal and adjustments.

```
class SplayTree {
    private Node root;

    // Node class to represent each node in the tree
    class Node {
        int data;  // Data stored in the node
        Node left, right;  // Left and right child nodes

        // Constructor to create a new node with the given value
        public Node(int item) {
            data = item;
            left = right = null;
        }
    }

    // Insert function to add a new value into the splay tree
    public void insert(int value) {
        if (root == null) {
            root = new Node(value);  // If tree is empty, create a new node as root
            return;
        }

        Node node = root;
        Node parent = null;

        // Loop to find the correct position for insertion
        while (node != null) {
            parent = node;
            if (value < node.data) {
                node = node.left;
            } else if (value > node.data) {
                node = node.right;
            } else {
                return;  // If the value is already present, do nothing
            }
        }

        // Create a new node and attach it to the parent
```

```
        node = new Node(value);
        if (value < parent.data) {
            parent.left = node;
        } else {
            parent.right = node;
        }

        // After insertion, splay the tree to bring the inserted node to the root
        root = splay(root, value);
    }

    // Delete function to remove a node from the splay tree
    public void delete(int value) {
        if (root == null) return;

        // Splay the tree to bring the node to be deleted to the root
        root = splay(root, value);

        if (root == null || root.data != value) return;  // Node not found

        // Node to be deleted is at the root
        if (root.left == null) {
            root = root.right;  // If left child is null, replace root with right child
        } else {
            Node rightSubtree = root.right;
            root = root.left;
            root = splay(root, value);  // Splay the left subtree to bring the max element to
            root.right = rightSubtree;  // Attach the right subtree to the new root
        }
    }

    // Find function to search for a value in the splay tree
    public boolean find(int value) {
        if (root == null) return false;

        // Splay the tree to bring the searched node to the root
        root = splay(root, value);

        return (root != null && root.data == value);
    }

    // Function to perform a right rotation on the tree (used during splaying)
    private Node rotateRight(Node root) {
        Node newRoot = root.left;  // Make the left child the new root
        root.left = newRoot.right;  // Make the right child of the new root the left child of
        newRoot.right = root;  // Make the old root the right child of the new root
        return newRoot;  // Return the new root
    }

    // Function to perform a left rotation on the tree (used during splaying)
    private Node rotateLeft(Node root) {
        Node newRoot = root.right;  // Make the right child the new root
        root.right = newRoot.left;  // Make the left child of the new root the right child of
```

```
        newRoot.left = root;  // Make the old root the left child of the new root
        return newRoot;  // Return the new root
    }

    // Splay function to bring the node with the given value to the root
    private Node splay(Node root, int value) {
        while (root != null) {
            // Base case: if the node is found or we reach the leaf, break out of the loop
            if (root.data == value) return root;

            // Zig (single rotation): If the node is the left child or right child of the root
            if (value < root.data) {
                if (root.left == null) break;  // If there's no left child, stop

                // Zig -Zig (double rotation): If node's value is in the left subtree of left
                if (value < root.left.data) {
                    root = rotateRight(root);
                } else if (value > root.left.data) {
                    // Zig -Zag (double rotation): If node's value is in the right subtree of
                    root.left = rotateLeft(root.left);
                    root = rotateRight(root);
                }
            } else {
                if (root.right == null) break;  // If there's no right child, stop

                // Zig -Zig (double rotation): If node's value is in the right subtree of righ
                if (value > root.right.data) {
                    root = rotateLeft(root);
                } else if (value < root.right.data) {
                    // Zig -Zag (double rotation): If node's value is in the left subtree of r
                    root.right = rotateRight(root.right);
                    root = rotateLeft(root);
                }
            }
        }
        return root;
    }

    // Pretty print function to display the tree visually
    public void prettyPrint(Node root, String indent) {
        if (root == null) return;  // If the root is null, return

        // Print the right subtree first with increased indentation
        prettyPrint(root.right, indent + "   ");
        System.out.println(indent + root.data);  // Print the current node
        // Print the left subtree with increased indentation
        prettyPrint(root.left, indent + "   ");
    }

    // Wrapper function to print the tree
    public void printTree() {
        prettyPrint(root, "");  // Call the prettyPrint function starting from the root
    }
```

```
    public static void main(String[] args) {
        SplayTree tree = new SplayTree();

        // Inserting values into the tree
        tree.insert(10);
        tree.insert(20);
        tree.insert(30);
        tree.insert(15);

        // Pretty print the tree after insertion
        System.out.println("Tree after insertion:");
        tree.printTree();

        // Find a value in the tree and bring it to the root
        tree.find(20);
        System.out.println("\nTree after finding 20:");
        tree.printTree();

        // Delete a value from the tree
        tree.delete(10);
        System.out.println("\nTree after deleting 10:");
        tree.printTree();
    }
}
```

Explanation of Changes:

1. **Insert Method**: The insertion now uses a loop to traverse the tree and find the correct position to insert the new node. The insertion process is followed by a splay operation to move the newly inserted node to the root.

2. **Delete Method**: Similar to insertion, the delete method now uses a loop to traverse the tree and delete the node. It also includes a splay operation to move the necessary nodes after deletion.

3. **Splay Method**: The splay function has been rewritten to use a loop instead of recursion. It continuously adjusts the tree until the target node is either found or no further adjustments are necessary.

4. **Rotation Methods**: The left and right rotations remain the same, as these are simple tree manipulations that don't depend on recursion.

This approach eliminates recursion in favor of iterative methods using loops, which may help avoid potential stack overflow issues with deep trees, though it may still be less efficient compared to a more balanced tree like an AVL or Red-Black tree. However, the splay tree performs well with specific access patterns, especially when there is locality of reference.

0.4 Chapter: Hash Maps, Open Addressing, and Chaining

Introduction

Hash maps are one of the workhorses of computer science. They power symbol tables, caches, dictionaries, indexing structures, and lookup systems across almost every area of software. When they behave well, they are astonishingly fast. A key goes in, a value comes out, and the whole operation feels almost immediate.

That speed, however, is not free. A hash map works because it converts a key into a numeric form, uses that number to compute an array position, and then deals intelligently with the awkward fact that different keys may land in the same location. That last detail matters. Collisions are not a sign that something has gone wrong. They are an ordinary part of hashing. The real question is how the structure resolves them.

This chapter explains the foundations of hash maps and collision handling, with special attention to **chaining**, **linear probing**, **quadratic probing**, and **double hashing**. It also explains how indices are computed safely in Java, why deletion in open addressing is trickier than students first expect, and how design choices affect runtime and memory behavior. You will also study a complete Java demonstration program that includes all of the major topics discussed in this chapter.

Chapter at a Glance

This chapter introduces the structure and purpose of hash maps, explains how hash functions map keys to array positions, and examines why collisions are unavoidable. It then studies the two major families of collision handling: **chaining** and **open addressing**. Within open addressing, it explores three probing strategies in detail: **linear probing**, **quadratic probing**, and **double hashing**. The chapter compares their behavior, tradeoffs, and typical use cases, and concludes with a complete Java program that demonstrates all of them in one place.

Learning Goals

By the end of this chapter, you should be able to:

Explain what a hash map is and why it is useful.
Describe how a hash function helps compute a table position.
Explain why collisions occur and why they must be handled deliberately.
Distinguish between **chaining** and **open addressing**.
Explain in detail how **linear probing**, **quadratic probing**, and **double hashing** work.
Describe when each probing strategy is appropriate.
Explain why deletion in open addressing usually requires tombstones.
Interpret and explain the Java expression

```
int index = (hash & 0x7fffffff) % table.length;
```

Trace insertion, search, and deletion behavior in hash-based structures.
Evaluate the performance tradeoffs among different collision strategies.

Foundations

A **map** is an abstract data type that stores associations between keys and values. If you know the key, you can retrieve the value. A student ID may map to a student record. A course code may map to a course title. A username may map to an account object.

A **hash map** is a map implementation that uses a **hash function** to convert a key into an integer, then uses that integer to determine where the key-value pair should be stored in an array-like structure called a **hash table**.

The overall idea is straightforward:

Take a key.

Compute a hash code from that key.

Convert the hash code into a legal array index.

Store or search for the entry at that index.

The hope is that this process gives very fast access, often close to constant time on average.

A hash map is therefore an example of a data structure that uses computation to reduce search cost. Instead of scanning through all elements, it uses the key to jump to a likely location immediately.

Why This Topic Matters

Hashing sits at the center of practical computing. If you write compilers, you use symbol tables. If you write web applications, you use caches and session maps. If you write search systems, databases, language runtimes, or operating systems, you will meet hash-based lookup again and again.

More importantly, hash maps are one of the first data structures that force students to face an uncomfortable but valuable truth: fast average performance depends on design assumptions. A hash map is not "always O(1)." That slogan is a half-truth repeated with the confidence of a bad witness. Hash maps are often extremely efficient, but only if the hash function distributes keys well, the collision policy is sensible, and the structure is resized before the table becomes overcrowded.

This topic matters because it teaches not only a useful data structure, but also a more mature habit of thought: performance claims depend on conditions.

Core Concepts

Keys and Values A hash map stores pairs:

- key
- value

For example:

- `"CS101"` → `"Introduction to Programming"`
- `"apple"` → `17`
- `42` → `"student record"`

Each key identifies one associated value.

Hash Code A key is transformed into an integer called a **hash code**. In Java, this often comes from the `hashCode()` method.

```
String s = "CS101";
int hash = s.hashCode();
```

The raw hash code is not automatically a valid array index. It may be negative, and it may be much larger than the table size. That means it must be converted.

Index Computation A common pattern in Java hash-table implementations is:

```
int index = (hash & 0x7fffffff) % table.length;
```

This expression deserves careful attention because students often use it without understanding it. Let us break it apart in detail.

Suppose `hash` is an `int`. In Java, an `int` is a 32-bit signed integer. That means the leftmost bit is the **sign bit**. If that bit is `1`, the number is negative.

The constant `0x7fffffff` is a hexadecimal number. In binary, it looks like this:

```
01111111111111111111111111111111
```

Notice what that means:
The first bit is `0`.
The remaining 31 bits are `1`.
Now consider the bitwise AND operation:

```
hash & 0x7fffffff
```

This clears the sign bit of `hash` and keeps the remaining 31 bits. The result is guaranteed to be nonnegative.

Then:

```
(hash & 0x7fffffff) % table.length
```

takes that nonnegative number and reduces it into the range:

```
0 to table.length - 1
```

So the full expression means:
Take the hash code.
Force it to be nonnegative by clearing its sign bit.
Reduce it modulo the table size so it becomes a valid array index.

Why not just use `Math.abs(hash)`? Because `Math.abs()` has a famous edge case:

```
Math.abs(Integer.MIN_VALUE)
```

still returns a negative number. That happens because the negative range of two's complement integers is slightly larger than the positive range. In other words, one value does not have a positive counterpart representable as an `int`.

That means this can fail:

```
int index = Math.abs(hash) % table.length;
```

if `hash` is `Integer.MIN_VALUE`.
The bit-mask approach avoids that particular problem.

Key Point The expression

```
int index = (hash & 0x7fffffff) % table.length;
```

is a practical way to convert a possibly negative Java hash code into a safe, nonnegative array index.

Debugging Note If your indexing logic sometimes produces a negative index, do not blame the compiler, the phase of the moon, or Java in general. Check your hash-to-index computation first.

Terminology

A **hash function** transforms a key into an integer hash code.
A **hash table** is the array-like structure that stores entries.
A **bucket** is one location in that table.
A **collision** occurs when two different keys map to the same bucket.
A **load factor** measures how full the table is.
A **probe** is an attempt to examine a slot during open addressing.
A **probe sequence** is the order of slots examined during probing.
A **tombstone** is a deletion marker used in open addressing.
A **rehash** moves entries into a new table, usually after resizing.

Hash Map Architecture

At a conceptual level, a hash map needs four things:
A way to represent entries.
A hash-to-index computation.
A collision-resolution strategy.
A resizing policy.
A simple entry representation in Java might be:

```
static class Entry<K, V> {
    K key;
    V value;
    boolean deleted;

    Entry(K key, V value) {
        this.key = key;
        this.value = value;
        this.deleted = false;
    }
}
```

For chaining, the `deleted` flag is not usually needed. For open addressing, it is useful because deletion is more delicate.

0.4.1 Chaining

How Chaining Works

In **chaining**, each table position stores a collection of entries rather than only one entry. If multiple keys hash to the same index, all of them are stored in that bucket's chain.

That chain is often implemented as a linked list in introductory treatments, though dynamic arrays or balanced trees may also be used in more advanced implementations.

If a collision occurs, chaining does not search for a different array position. It simply adds the new entry to the collection stored at that bucket.

Example Idea If keys `10`, `15`, and `20` all map to bucket `2`, then bucket `2` might hold a linked list containing:

```
[10] -> [15] -> [20]
```

The array position remains the same. The chain grows.

Why Chaining Is Attractive

Chaining is conceptually straightforward. Collisions are stored together. Deletion is simpler than in open addressing. The table can even handle a load factor greater than 1.0, because one bucket can store multiple entries.

Chaining Runtime

Average-case behavior is often:
Insert: O(1)
Search: O(1)
Delete: O(1)
Worst-case behavior is O(n) if many keys pile into the same bucket.

Performance Note Chaining is often more forgiving than open addressing when the load factor rises. That does not mean overcrowding is fine. It means the collapse is usually less dramatic.

0.4.2 Open Addressing

How Open Addressing Works

In **open addressing**, every entry is stored directly in the table array itself. If a collision occurs, the algorithm probes other locations until it finds a suitable slot.

This means there are no external lists. Everything stays in the table.

That sounds elegant, and it is. It is also more fragile.

Search and insertion must follow the same probing rules. Deletion must preserve the integrity of those probe sequences. If you break the sequence, later searches can fail even when the desired key is still in the table.

The three probing strategies covered here are:
Linear probing
Quadratic probing
Double hashing

Linear Probing

What Linear Probing Is **Linear probing** resolves a collision by checking the next slot, then the next, then the next, wrapping around the table if necessary.

If the original hash index is `h(k)`, then the probe sequence is:

```
h(k), h(k)+1, h(k)+2, h(k)+3, ...
```

taken modulo the table size.

Mathematically:

$$\text{index}_i = (h(k) + i) \bmod m \tag{1}$$

where:

- `h(k)` is the starting index
- `i` is the probe number
- `m` is the table length

Step-by-Step Example of Linear Probing Suppose the table size is `7`, and keys `10`, `17`, and `24` all hash initially to index `3`.

Insert `10`:

- slot 3 is empty
- place `10` at slot 3

Insert `17`:

- slot 3 is occupied
- try slot 4
- slot 4 is empty
- place `17` at slot 4

Insert `24`:

- slot 3 occupied
- slot 4 occupied
- try slot 5
- place `24` at slot 5

So the cluster becomes:

```
3: 10
4: 17
5: 24
```

Strengths of Linear Probing It is simple.

It is easy to implement.

It often has good cache performance because nearby locations are examined in sequence.

Weaknesses of Linear Probing Its major weakness is **primary clustering**. Once a block of occupied cells forms, new collisions tend to enlarge that block. Clusters become magnets for more collisions. As the load factor rises, probe lengths can increase sharply.

When Should One Use Linear Probing? Use linear probing when:

You want the simplest open-addressing implementation.

You want a compact table with good cache locality.

You expect the load factor to be kept relatively low.

You want a teaching implementation or a performance-oriented design where simplicity matters.

Do not choose linear probing casually if the table will become crowded. It performs well when managed properly and degrades badly when neglected.

Quadratic Probing

What Quadratic Probing Is **Quadratic probing** resolves collisions by increasing the probe distance quadratically rather than linearly.

Instead of checking:

```
h, h+1, h+2, h+3, ...
```

it checks something like:

```
h, h+1^2, h+2^2, h+3^2, ...
```

modulo the table size.

A common form is:

$$\text{index}_i = (h(k) + i^2) \bmod m \tag{2}$$

Other variants use constants such as:

$$\text{index}_i = (h(k) + c_1 i + c_2 i^2) \bmod m \tag{3}$$

but the simpler square-based version is enough for learning the idea.

Step-by-Step Example of Quadratic Probing Suppose the table size is `11`, and a key hashes to index `5`.

If slot `5` is occupied, quadratic probing tries:

- `5 + 1^2 = 6`
- `5 + 2^2 = 9`
- `5 + 3^2 = 14 mod 11 = 3`
- `5 + 4^2 = 21 mod 11 = 10`

So the probe sequence is:

```
5, 6, 9, 3, 10, ...
```

Notice that the probes spread out more than linear probing. That reduces contiguous clustering.

Strengths of Quadratic Probing It reduces primary clustering compared with linear probing.

It still remains fairly easy to implement.

It spreads collisions out more broadly.

Weaknesses of Quadratic Probing It can still suffer from **secondary clustering**, meaning keys that start at the same initial hash index follow the same probe pattern.

Also, depending on the table size and probing formula, the sequence may fail to visit every slot. That means insertion can fail to find an open slot even though one exists, unless the table size and formula are chosen carefully.

Chapter Warning Quadratic probing is not "linear probing but smarter" in some vague universal sense. It helps with clustering, but it introduces mathematical constraints. If the table size and formula are chosen badly, the probe sequence may not cover the whole table.

When Should One Use Quadratic Probing? Use quadratic probing when:

You want to reduce clustering compared with linear probing.

You still want an open-addressing method that is simpler than double hashing.

You can control the table size and probing design carefully.

Quadratic probing is often a reasonable middle ground: less clustering than linear probing, less conceptual overhead than double hashing.

Double Hashing

What Double Hashing Is **Double hashing** is an open-addressing collision-resolution strategy that uses a second hash function to determine the step size between probes.

Instead of moving by `+1` each time, or by quadratic offsets, the algorithm computes a second value and uses that as the jump distance.

A common form is:

$$\text{index}_i = (h_1(k) + i \cdot h_2(k)) \bmod m \tag{4}$$

where:

- `h1(k)` is the primary hash and gives the starting position
- `h2(k)` is the secondary hash and gives the step size
- `i` is the probe number
- `m` is the table size

This means each key may follow its own probe path through the table rather than sharing the same fixed pattern used by linear or quadratic probing.

That is the central strength of double hashing. It reduces the tendency of colliding keys to travel together in the same sequence.

Why Double Hashing Matters The big weakness of linear probing is **primary clustering**. Once a run of occupied slots appears, new collisions tend to extend it.

Quadratic probing reduces that problem, but keys with the same starting index can still follow the same general probe pattern. That is **secondary clustering**.

Double hashing addresses both issues more effectively by making the probe increment depend on the key itself. Two keys that start at the same initial bucket may still move through the table differently if their second-hash values differ.

That usually produces better probe distribution, shorter average probe sequences, and more stable performance as collisions increase.

Key Point Double hashing works well not because it "uses two hashes" in some vague symbolic sense, but because the second hash gives each key a more individualized probe step.

Step-by-Step Example of Double Hashing Suppose:

```
m = 11
h1(k) = k % 11
h2(k) = 7 - (k % 7)
```

Now consider key `22`.

Primary hash:

```
h1(22) = 22 % 11 = 0
```

Secondary hash:

```
h2(22) = 7 - (22 % 7)
       = 7 - 1
       = 6
```

So the probe sequence is:

- `(0 + 0*6) % 11 = 0`
- `(0 + 1*6) % 11 = 6`
- `(0 + 2*6) % 11 = 1`
- `(0 + 3*6) % 11 = 7`

- `(0 + 4*6) % 11 = 2`
- `(0 + 5*6) % 11 = 8`

and so on.

This sequence jumps around the table rather than walking in a fixed pattern. That is exactly what makes double hashing effective.

Now compare that with another key, say `33`.

Primary hash:

```
h1(33) = 33 % 11 = 0
```

Secondary hash:

```
h2(33) = 7 - (33 % 7)
       = 7 - 5
       = 2
```

Its probe sequence becomes:

- `(0 + 0*2) % 11 = 0`
- `(0 + 1*2) % 11 = 2`
- `(0 + 2*2) % 11 = 4`
- `(0 + 3*2) % 11 = 6`
- `(0 + 4*2) % 11 = 8`

So even though `22` and `33` begin at the same primary index, they move through the table differently. That is the point.

The Real Design Question: How Should the Second Hash Be Chosen? This is the part many short explanations skip, and they should not.

The second hash function must be chosen carefully. A bad second hash can sabotage the entire design.

A good secondary hash should satisfy three conditions:

It should never return `0`.

It should produce step sizes that allow the probe sequence to cover the table effectively.

It should vary across keys so different keys are likely to follow different probe paths.

If the second hash returns zero, the probe formula becomes:

[(h_1(k) + i \cdot 0) \bmod m]

which means the algorithm never moves beyond the original slot. That is not probing. That is being stuck with mathematical paperwork.

Chapter Warning In double hashing, a zero-valued second hash is a design failure. The probe sequence will never advance.

Common Secondary Hash Functions

1. `1 + (hash % (m - 1))` One of the most common textbook choices is:

```
int step = 1 + ((hash & 0x7fffffff) % (table.length - 1));
```

This is a strong general-purpose teaching example.

Why it works:

- `hash % (table.length - 1)` produces a value from `0` to `table.length - 2`
- adding `1` shifts the range to `1` through `table.length - 1`
- therefore the step size is never zero

This choice works especially well when the table length is **prime**. If `m` is prime, then every value from `1` through `m - 1` is relatively prime to `m`, which helps the probe sequence visit the entire table.

This is the cleanest secondary hash to teach first.

2. `R - (hash % R)` Another classic form is:

```
int R = 7; // prime smaller than table length
int step = R - ((hash & 0x7fffffff) % R);
```

Here, `R` is usually chosen as a prime number smaller than the table size.

Why it works:

- the result is nonzero
- it often gives a good spread of step sizes
- it is a traditional and widely taught form

Example:

If `R = 7` and `hash % 7 = 3`, then:

```
step = 7 - 3; // step = 4
```

This form is simple, memorable, and common in algorithm texts.

3. Odd-valued step for power-of-two table sizes If the table size is a power of two, a common strategy is to force the step size to be odd.

For example:

```
int step = ((hash & 0x7fffffff) % (table.length - 1)) | 1;
```

or more explicitly:

```
int step = 1 + 2 * ((hash & 0x7fffffff) % ((table.length - 1) / 2));
```

Why this works:

If the table size is a power of two, then any odd step is relatively prime to that table size. That helps the probe sequence cover the table rather than cycling through a smaller subset of positions.

This is especially useful in implementations that deliberately use power-of-two capacities for performance reasons.

4. Simple classroom examples for integer keys For classroom tracing, you may also see examples like:

```
int step = 7 - (key % 7);
```

These are fine for hand computation and lecture demonstrations. They are easy to trace on a whiteboard.

But for a more general implementation, it is better to compute the step from the key's hash code rather than the raw integer key value. That makes the design work more cleanly for arbitrary object keys.

Choosing a Good Secondary Hash in Practice

The best advice is:

If the table size is prime, use a prime-friendly form such as:

```
1 + (hash % (m - 1))
```

or:

```
R - (hash % R)
```

where `R` is a smaller prime.

If the table size is a power of two, force the step size to be odd.

Performance Note Double hashing is often one of the strongest standard open-addressing strategies because it reduces both primary and secondary clustering more effectively than linear or quadratic probing.

Strengths of Double Hashing It greatly reduces clustering.

Keys with the same initial hash can still follow different probe sequences.

It often gives better probe distribution than linear probing.

It often gives better probe distribution than quadratic probing.

It usually performs well when the table is moderately full, provided the secondary hash and table size are chosen intelligently.

Weaknesses of Double Hashing It is more complicated to implement.

The second hash function must be chosen carefully.

The table size and step size must be compatible enough to let the probe sequence cover the table well.

It is harder to explain quickly than linear probing, which is probably why people explain it quickly and badly.

When Should One Use Double Hashing? Use double hashing when:

You want better probe distribution than linear or quadratic probing.

You want to reduce both primary and secondary clustering.

You are willing to accept a more careful implementation.

You want open addressing, but you do not want all colliding keys marching in lockstep through the table.

Double hashing is often the best open-addressing choice when collision quality matters enough to justify the extra complexity.

Comparing Linear, Quadratic, and Double Hashing

Comparison Box Linear probing

- simplest implementation
- very cache-friendly
- suffers from primary clustering
- good when load factor stays low and simplicity matters

Quadratic probing

- reduces primary clustering
- still simpler than double hashing
- can suffer from secondary clustering
- requires careful table-size design

Double hashing

- usually best probe distribution of the three
- reduces both primary and secondary clustering effectively
- depends on a well-designed second hash function
- more complex to implement
- strong choice when collision behavior and probe quality matter

---

Deletion in Open Addressing

Deletion in open addressing is more complicated than in chaining.

Suppose several keys collide and are placed later in the probe sequence. If you remove one entry by setting its slot to `null`, a later search may stop too early, incorrectly concluding that the desired key is not present.

For that reason, open addressing usually uses a **tombstone** marker. A tombstone means:

"There used to be something here. Keep probing."

Example Suppose linear probing placed these entries:

```
index 3 -> A
index 4 -> B
index 5 -> C
```

If `A` is deleted and slot `3` becomes `null`, then a search for `C` starting at `3` may incorrectly stop immediately.

If slot `3` becomes a tombstone instead, probing continues and `C` can still be found.

Common Mistake Students often write open-address deletion by assigning `null` to the slot and then wonder why later searches mysteriously fail. This is not mysterious. This is a broken probe chain.

Behind-the-Scenes Behavior

Hash maps are shaped by more than their visible interface.

A good implementation depends on:

a usable hash function

a safe index computation

a table size policy

a collision-resolution strategy

a resizing threshold

a deletion policy

In Java, custom key classes must also obey the relationship between `equals()` and `hashCode()`. If two objects are considered equal by `equals()`, they must produce compatible hash behavior. If they do not, a hash-based structure can fail to retrieve a key that is already in the table.

That is not a minor technicality. It is structural.

Best Practices

Use a good hash function.

Keep the load factor under control.

Resize before performance collapses.

Use chaining if you want simpler deletion behavior and tolerance for fuller tables.

Use linear probing when simplicity and locality matter.

Use quadratic probing when you want to reduce clustering but keep the design moderately simple.

Use double hashing when you want better probe distribution and can manage the extra design complexity.

For Java keys, implement `equals()` and `hashCode()` correctly.

Do not treat hash maps as magical boxes. They are engineered compromises, and every compromise has consequences.

Sample Program Covering All Topics

The following Java program demonstrates:
chaining
linear probing
quadratic probing
double hashing
safe index computation using

```
(hash & 0x7fffffff) % table.length
```

tombstone-based deletion in open addressing
This is an educational demonstration, not a production replacement for `java.util.HashMap`.

```
import java.util.LinkedList;
import java.util.Iterator;

public class HashingChapterDemo {

    // - - - - - - - - - - - - - - - - - Entry - - - - - - - - - - - - - - - -
    static class Entry<K, V> {
        K key;
        V value;
        boolean deleted;

        Entry(K key, V value) {
            this.key = key;
            this.value = value;
            this.deleted = false;
        }

        @Override
        public String toString() {
            return deleted ? "[DELETED]" : key + "=" + value;
        }
    }

    // - - - - - - - - - - - - - - - - - Base utility - - - - - - - - - - - - - - - -
    static abstract class BaseHashTable<K, V> {
        protected Entry<K, V>[] table;
        protected int size;

        @SuppressWarnings("unchecked")
        public BaseHashTable(int capacity) {
            table = new Entry[capacity];
            size = 0;
        }

        protected int primaryIndex(K key) {
```

```
        int hash = key.hashCode();
        return (hash & 0x7fffffff) % table.length;
    }

    public void printTable(String title) {
        System.out.println(title);
        for (int i = 0; i < table.length; i++) {
            System.out.println(i + ": " + (table[i] == null ? "empty" : table[i]));
        }
        System.out.println();
    }
}

// - - - - - - - - - - - - - - - - - Chaining - - - - - - - - - - - - - - - - -
static class ChainedHashMap<K, V> {
    private LinkedList<Entry<K, V>>[] table;
    private int size;

    @SuppressWarnings("unchecked")
    public ChainedHashMap(int capacity) {
        table = new LinkedList[capacity];
        size = 0;
    }

    private int indexFor(K key) {
        int hash = key.hashCode();
        return (hash & 0x7fffffff) % table.length;
    }

    public void put(K key, V value) {
        int index = indexFor(key);

        if (table[index] == null) {
            table[index] = new LinkedList<>();
        }

        for (Entry<K, V> e : table[index]) {
            if (e.key.equals(key)) {
                e.value = value;
                return;
            }
        }

        table[index].add(new Entry<>(key, value));
        size++;
    }

    public V get(K key) {
        int index = indexFor(key);

        if (table[index] == null) {
            return null;
        }
```

```
            for (Entry<K, V> e : table[index]) {
                if (e.key.equals(key)) {
                    return e.value;
                }
            }
            return null;
        }

        public boolean remove(K key) {
            int index = indexFor(key);

            if (table[index] == null) {
                return false;
            }

            Iterator<Entry<K, V>> it = table[index].iterator();
            while (it.hasNext()) {
                Entry<K, V> e = it.next();
                if (e.key.equals(key)) {
                    it.remove();
                    size - -;
                    return true;
                }
            }
            return false;
        }

        public void printTable(String title) {
            System.out.println(title);
            for (int i = 0; i < table.length; i++) {
                System.out.print(i + ": ");
                if (table[i] == null || table[i].isEmpty()) {
                    System.out.println("empty");
                } else {
                    System.out.println(table[i]);
                }
            }
            System.out.println();
        }
    }

    // - - - - - - - - - - - - - - - - - - Linear Probing - - - - - - - - - - - - - - - - -
    static class LinearProbingMap<K, V> extends BaseHashTable<K, V> {

        public LinearProbingMap(int capacity) {
            super(capacity);
        }

        public void put(K key, V value) {
            int index = primaryIndex(key);
            int firstDeleted = -1;
```

```
            while (table[index] != null) {
                if (table[index].deleted) {
                    if (firstDeleted == -1) {
                        firstDeleted = index;
                    }
                } else if (table[index].key.equals(key)) {
                    table[index].value = value;
                    return;
                }

                index = (index + 1) % table.length;
            }

            if (firstDeleted != -1) {
                table[firstDeleted] = new Entry<>(key, value);
            } else {
                table[index] = new Entry<>(key, value);
            }
            size++;
        }

        public V get(K key) {
            int index = primaryIndex(key);
            int start = index;

            while (table[index] != null) {
                if (!table[index].deleted && table[index].key.equals(key)) {
                    return table[index].value;
                }
                index = (index + 1) % table.length;
                if (index == start) break;
            }
            return null;
        }

        public boolean remove(K key) {
            int index = primaryIndex(key);
            int start = index;

            while (table[index] != null) {
                if (!table[index].deleted && table[index].key.equals(key)) {
                    table[index].deleted = true;
                    size - -;
                    return true;
                }
                index = (index + 1) % table.length;
                if (index == start) break;
            }
            return false;
        }
    }

    // - - - - - - - - - - - - - - - - - - Quadratic Probing - - - - - - - - - - - - - - - - -
```

```
    static class QuadraticProbingMap<K, V> extends BaseHashTable<K, V> {

        public QuadraticProbingMap(int capacity) {
            super(capacity);
        }

        public void put(K key, V value) {
            int base = primaryIndex(key);
            int firstDeleted = -1;

            for (int i = 0; i < table.length; i++) {
                int index = (base + i * i) % table.length;

                if (table[index] == null) {
                    if (firstDeleted != -1) {
                        table[firstDeleted] = new Entry<>(key, value);
                    } else {
                        table[index] = new Entry<>(key, value);
                    }
                    size++;
                    return;
                }

                if (table[index].deleted) {
                    if (firstDeleted == -1) {
                        firstDeleted = index;
                    }
                } else if (table[index].key.equals(key)) {
                    table[index].value = value;
                    return;
                }
            }

            System.out.println("Quadratic probing insert failed for key " + key + ". Table may
        }

        public V get(K key) {
            int base = primaryIndex(key);

            for (int i = 0; i < table.length; i++) {
                int index = (base + i * i) % table.length;

                if (table[index] == null) {
                    return null;
                }

                if (!table[index].deleted && table[index].key.equals(key)) {
                    return table[index].value;
                }
            }
            return null;
        }
```

```
    public boolean remove(K key) {
        int base = primaryIndex(key);

        for (int i = 0; i < table.length; i++) {
            int index = (base + i * i) % table.length;

            if (table[index] == null) {
                return false;
            }

            if (!table[index].deleted && table[index].key.equals(key)) {
                table[index].deleted = true;
                size - -;
                return true;
            }
        }
        return false;
    }
}

// - - - - - - - - - - - - - - - - - Double Hashing - - - - - - - - - - - - - - - - -
static class DoubleHashingMap<K, V> extends BaseHashTable<K, V> {

    public DoubleHashingMap(int capacity) {
        super(capacity);
    }

    // Option 1: common choice when table length is prime
    private int stepHashOption1(K key) {
        int hash = key.hashCode() & 0x7fffffff;
        return 1 + (hash % (table.length - 1));
    }

    // Option 2: classic textbook form using a smaller prime R
    private int stepHashOption2(K key) {
        int hash = key.hashCode() & 0x7fffffff;
        int R = 7; // should be prime and smaller than table length
        return R - (hash % R);
    }

    // Option 3: odd -valued step, useful for power -of -two style thinking
    private int stepHashOption3(K key) {
        int hash = key.hashCode() & 0x7fffffff;
        return ((hash % (table.length - 1)) | 1);
    }

    // Pick one strategy for actual use
    private int stepHash(K key) {
        return stepHashOption1(key);
        // return stepHashOption2(key);
        // return stepHashOption3(key);
    }
```

```
    public void put(K key, V value) {
        int base = primaryIndex(key);
        int step = stepHash(key);
        int firstDeleted = -1;

        for (int i = 0; i < table.length; i++) {
            int index = (base + i * step) % table.length;

            if (table[index] == null) {
                if (firstDeleted != -1) {
                    table[firstDeleted] = new Entry<>(key, value);
                } else {
                    table[index] = new Entry<>(key, value);
                }
                size++;
                return;
            }

            if (table[index].deleted) {
                if (firstDeleted == -1) {
                    firstDeleted = index;
                }
            } else if (table[index].key.equals(key)) {
                table[index].value = value;
                return;
            }
        }

        System.out.println("Double hashing insert failed for key " + key + ". Table may be
    }

    public V get(K key) {
        int base = primaryIndex(key);
        int step = stepHash(key);

        for (int i = 0; i < table.length; i++) {
            int index = (base + i * step) % table.length;

            if (table[index] == null) {
                return null;
            }

            if (!table[index].deleted && table[index].key.equals(key)) {
                return table[index].value;
            }
        }
        return null;
    }

    public boolean remove(K key) {
        int base = primaryIndex(key);
        int step = stepHash(key);
```

```
        for (int i = 0; i < table.length; i++) {
            int index = (base + i * step) % table.length;

            if (table[index] == null) {
                return false;
            }

            if (!table[index].deleted && table[index].key.equals(key)) {
                table[index].deleted = true;
                size - -;
                return true;
            }
        }
        return false;
    }
}

// - - - - - - - - - - - - - - - - - Helper to show hash/index computation - - - - - - - - -
public static void showIndexComputation(Integer key, int tableLength) {
    int hash = key.hashCode();
    int masked = hash & 0x7fffffff;
    int index = masked % tableLength;

    System.out.println("Key: " + key);
    System.out.println("hashCode(): " + hash);
    System.out.println("hash & 0x7fffffff: " + masked);
    System.out.println("index = (hash & 0x7fffffff) % " + tableLength + " = " + index);
    System.out.println();
}

// - - - - - - - - - - - - - - - - - Demo - - - - - - - - - - - - - - - - -
public static void main(String[] args) {

    System.out.println("=== Safe Index Computation Demo ===");
    showIndexComputation(10, 11);
    showIndexComputation( -25, 11);

    System.out.println("=== Chaining Demo ===");
    ChainedHashMap<Integer, String> chained = new ChainedHashMap<>(5);
    chained.put(10, "Ten");
    chained.put(15, "Fifteen");
    chained.put(20, "Twenty");
    chained.printTable("Chaining table after inserts:");
    System.out.println("Get 15: " + chained.get(15));
    chained.remove(15);
    chained.printTable("Chaining table after removing 15:");

    System.out.println("=== Linear Probing Demo ===");
    LinearProbingMap<Integer, String> linear = new LinearProbingMap<>(7);
    linear.put(10, "Ten");
    linear.put(17, "Seventeen");
    linear.put(24, "Twenty -Four");
    linear.printTable("Linear probing after inserts:");
```

```
        System.out.println("Get 24: " + linear.get(24));
        linear.remove(17);
        linear.printTable("Linear probing after removing 17:");
        linear.put(31, "Thirty -One");
        linear.printTable("Linear probing after inserting 31:");

        System.out.println("=== Quadratic Probing Demo ===");
        QuadraticProbingMap<Integer, String> quadratic = new QuadraticProbingMap<>(11);
        quadratic.put(10, "Ten");
        quadratic.put(21, "Twenty -One");
        quadratic.put(32, "Thirty -Two");
        quadratic.put(43, "Forty -Three");
        quadratic.printTable("Quadratic probing after inserts:");
        System.out.println("Get 32: " + quadratic.get(32));
        quadratic.remove(21);
        quadratic.printTable("Quadratic probing after removing 21:");
        quadratic.put(54, "Fifty -Four");
        quadratic.printTable("Quadratic probing after inserting 54:");

        System.out.println("=== Double Hashing Secondary Hash Demo ===");
        int sampleKey = 22;
        int hash = Integer.hashCode(sampleKey) & 0x7fffffff;

        System.out.println("Sample key: " + sampleKey);
        System.out.println("Primary index: " + (hash % 11));
        System.out.println("Option 1 step = 1 + (hash % (m - 1)) = " + (1 + (hash % (11 - 1))))
        System.out.println("Option 2 step = R - (hash % R), R = 7 => " + (7 - (hash % 7)));
        System.out.println("Option 3 odd -valued step = ((hash % (m - 1)) | 1) = " + ((hash %
        System.out.println();

        System.out.println("=== Double Hashing Demo ===");
        DoubleHashingMap<Integer, String> doubleHash = new DoubleHashingMap<>(11);
        doubleHash.put(22, "Twenty -Two");
        doubleHash.put(33, "Thirty -Three");
        doubleHash.put(44, "Forty -Four");
        doubleHash.put(55, "Fifty -Five");
        doubleHash.printTable("Double hashing after inserts:");
        System.out.println("Get 44: " + doubleHash.get(44));
        doubleHash.remove(33);
        doubleHash.printTable("Double hashing after removing 33:");
        doubleHash.put(66, "Sixty -Six");
        doubleHash.printTable("Double hashing after inserting 66:");
    }
}
```

Walkthrough of the Sample Program

The program begins with an `Entry` class used by all implementations. Each entry stores a key, a value, and a `deleted` flag. That flag matters for open addressing because tombstones preserve the probe sequence after deletion.

The `BaseHashTable` class provides a shared table and a shared `primaryIndex()` method. That method uses the expression:

```
(hash & 0x7fffffff) % table.length
```

to convert a key's hash code into a safe index.

The `ChainedHashMap` class uses an array of `LinkedList<Entry<K, V»`. If several keys land in the same bucket, they are stored together in the linked list for that bucket.

The `LinearProbingMap` class places entries directly into the array and resolves collisions by moving one step at a time.

The `QuadraticProbingMap` class uses square offsets such as `i * i` to spread collisions out.

The `DoubleHashingMap` class computes both a primary hash index and a secondary hash step size. Unlike linear probing and quadratic probing, which follow fixed probe patterns, double hashing uses a second hash function so that different keys can follow different probe sequences. The sample code includes several common secondary-hash strategies, including `1 + (hash % (m - 1))`, `R - (hash % R)` for a smaller prime `R`, and an odd-valued step suitable for power-of-two table sizes. This makes the probe pattern more individualized and usually reduces clustering more effectively than the other two open-addressing methods.

The `showIndexComputation()` method prints the internal stages of the safe index formula so students can see the hash code, the masked nonnegative value, and the final modulo reduction.

Finally, the `main()` method demonstrates all of the structures and probing styles in action.

Common Mistakes and Misconceptions

A hash map is not automatically O(1) in all cases. That is average-case behavior under good conditions.

A good hash function does not eliminate collisions. It reduces harmful clustering.

Linear probing is not always the best choice simply because it is easy.

Quadratic probing is not automatically safe under all table sizes and formulas.

Double hashing is not "too advanced to matter." It is often the most effective probing strategy among the three.

Deletion in open addressing is not ordinary deletion. Probe sequences must be preserved.

Using a raw hash code as an array index is wrong unless it has been converted safely into the valid table range.

Checkpoint Can you explain, in your own words, why the bit mask `0x7fffffff` is used before taking modulo table length?

If not, revisit that section. It matters.

Key Terms

hash map
- hash table
- hash function
- hash code
- bucket
- collision
- load factor
- chaining

open addressing
linear probing
quadratic probing
double hashing
secondary hash function
probe
probe sequence
tombstone
rehash
primary clustering
secondary clustering

Chapter Summary

A hash map stores key-value pairs by transforming keys into numeric hash codes and then converting those hash codes into table indices. Because different keys can map to the same index, collisions are unavoidable and must be handled carefully.

Two major strategies exist for collision handling. **Chaining** stores multiple entries in the same bucket, usually using a linked structure. It is conceptually simple and handles deletion gracefully. **Open addressing** keeps all entries inside the table itself and uses probing to find alternate positions.

Three important probing strategies are **linear probing**, **quadratic probing**, and **double hashing**. Linear probing is simple and cache-friendly but suffers from primary clustering. Quadratic probing reduces primary clustering but can still suffer from secondary clustering and requires careful design. Double hashing typically provides the best probe distribution of the three strategies because it uses a second hash function to determine the probe step size. Common second-hash choices include `1 + (hash % (m - 1))`, `R - (hash % R)` for a smaller prime `R`, and odd-valued step functions for power-of-two table sizes. It is more complex to implement, but it often reduces clustering more effectively than linear or quadratic probing.

The expression

```
int index = (hash & 0x7fffffff) % table.length;
```

is a practical way in Java to convert a possibly negative hash code into a safe, nonnegative array index.

The real lesson of hash maps is not merely that they are fast. It is that they are fast when their assumptions are respected.

Final Takeaway Hashing is where elegant theory meets rude reality. The array is finite, collisions are inevitable, and performance is conditional. The art is in how you recover from the collision without losing the speed that made hashing attractive in the first place.

Multiple-Choice Review Questions

1. What is the purpose of a hash function in a hash map? A. To sort keys B. To convert a key into an integer useful for locating a table position C. To remove collisions D. To compress values only

2. What does the expression below mainly accomplish?

```
(hash & 0x7fffffff) % table.length
```

A. It sorts the table B. It forces the hash to be odd C. It converts a possibly negative hash into a valid nonnegative index D. It doubles the table length

3. In chaining, a collision is handled by: A. jumping to the next empty slot B. storing multiple entries in the same bucket structure C. deleting the older entry D. using two hash tables at once

4. Linear probing resolves collisions by: A. using linked lists B. using the next slot, then the next, and so on C. squaring the hash code D. reversing the key

5. Quadratic probing typically uses which kind of offset pattern? A. powers of two only B. consecutive negative offsets C. square-based offsets such as `i * i` D. a second table

6. Double hashing differs from linear and quadratic probing because it: A. stores collisions in chains B. uses a second hash function to determine step size C. never has collisions D. avoids modular arithmetic

7. Which probing method is most likely to suffer from primary clustering? A. chaining B. linear probing C. double hashing D. separate trees

8. Why are tombstones often needed in open addressing? A. to increase table size B. to make hash codes positive C. to preserve probe sequences after deletion D. to sort the entries

9. When might double hashing be a strong choice? A. when you want good probe distribution and can accept extra implementation complexity B. when you need the simplest code possible C. when collisions never happen D. when deletion does not matter

10. Which statement is most accurate? A. Hash maps are always O(1) B. A good hash function prevents all collisions C. Open addressing and chaining are identical except for syntax D. Hash-map performance depends on hashing quality, load factor, and collision strategy

---

Answer Key

1. B
2. C
3. B
4. B
5. C
6. B
7. B
8. C
9. A

10. D

End-of-Chapter Glossary

Bucket A location in the table array. In chaining, one bucket may hold multiple entries.

Chaining A collision-resolution method in which each bucket stores a collection of entries.

Collision The event in which two different keys map to the same hash-table location.

Double Hashing An open-addressing strategy that uses a second hash function to determine the probe step size, allowing different keys to follow different probe sequences.

Entry A key-value pair stored in a hash-based structure.

Hash Code An integer derived from a key and used to compute a table position.

Hash Function A function that transforms a key into an integer hash code.

Hash Map A map implementation based on hashing for fast average-case lookup, insertion, and deletion.

Hash Table The underlying array-based storage structure used in a hash map.

Linear Probing An open-addressing method that checks the next slot, then the next, in sequence.

Load Factor A measure of how full the table is, usually the number of stored entries divided by table size.

Open Addressing A collision-resolution strategy in which all entries remain inside the table array and alternate locations are found through probing.

Primary Clustering A phenomenon in linear probing in which contiguous blocks of occupied cells grow and attract more collisions.

Probe One attempt to inspect a slot during insertion or search in open addressing.

Probe Sequence The sequence of positions examined during probing.

Quadratic Probing An open-addressing strategy that uses square-based offsets to resolve collisions.

Rehashing Reinserting entries into a new table after resizing.

Secondary Clustering A collision pattern in which keys with the same initial hash follow the same probe sequence.

Secondary Hash Function In double hashing, the second function used to compute the step size between probes.

Tombstone A marker indicating that a slot once held an entry and must still be treated as part of a valid probe path.

0.5 Skip List

0.5.1 Introduction

Skip lists are a powerful data structure that offer efficient search and insertion operations on sorted data. They achieve this by combining the simplicity of linked lists with the efficiency of jump searches. Here's a deeper dive into their inner workings:

Structure:

- **Levels:** A skip list is made up of multiple levels, with Level 0 being the bottommost level and acting as a standard sorted linked list. Higher levels contain fewer elements and serve as shortcuts to navigate the data faster.
- Nodes:
  Each element in the skip list is stored in a node. A node typically contains the element's value and pointers:
  - **Forward Pointers:** These point to the next element in the same level, maintaining the sorted order within each level.
  - **Shortcut Pointers (Optional):** These pointers "jump" over some elements in the level below, allowing for faster traversal during search and insertion.

Random Height Assignment:

- A crucial aspect of skip lists is the random height assigned to each element. This randomness is what gives them their probabilistic nature.
- A coin-flipping approach (or any other random number generation) is used to determine the height. Higher levels have fewer elements because the chance of a successful coin flip (reaching a higher level) decreases with each level.

Searching in a Skip List:

1. Start at the highest level (Level 3 in the 4-level example).
2. Compare the search value with the current element's value.
3. If the search value is less, move to the previous element using the forward pointer in the current level.
4. If the search value is greater and there's a shortcut pointer available, use it to jump several elements ahead in the level below (Level 2).
5. Repeat steps 2-4 until you either find the element or reach the end of the level.
6. If you haven't found it yet, move down to Level 1 and repeat the process (compare, move, jump using shortcuts if available).
7. Finally, search the bottom level (Level 0) using the standard linked list traversal.

Benefits:

- **Average Search Time (O(log n)):** Due to the shortcut pointers, searching a skip list on average takes logarithmic time (O(log n)), where n is the number of elements. This is significantly faster than a linear search (O(n)) in a standard linked list.
- **Efficient Insertions:** Similar to searching, insertions also benefit from the layered structure. By efficiently navigating through the levels, the element can be inserted in its correct sorted position.
- **Simpler than Balanced Trees:** Compared to balanced search trees (e.g., AVL trees), skip lists are easier to implement and have lower memory overhead.

Drawbacks:

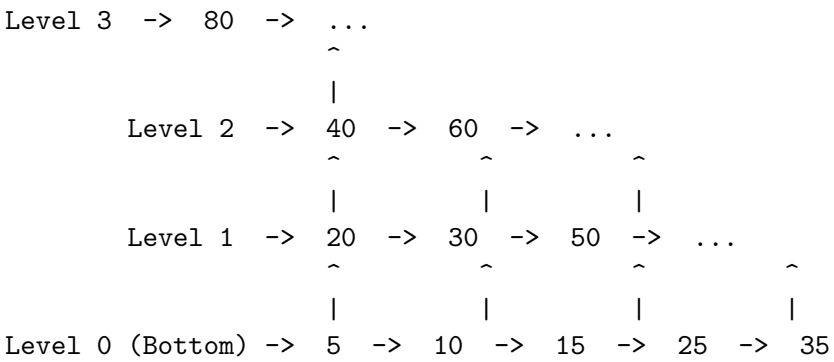
- **Probabilistic:** The random height assignment makes skip lists probabilistic. While the average performance is excellent, there's a small chance of the worst-case scenario (O(n)) happening in search or insertion.

- **Space Overhead:** The shortcut pointers add some extra space compared to a regular linked list.

Overall, skip lists offer a compelling balance between simplicity and efficiency for maintaining sorted data. They are a powerful alternative to balanced search trees in situations where the slight possibility of a worst-case scenario is acceptable.

0.5.2 Skiplist Diagram

Here's a diagram of a skip list with 4 levels:



Explanation:

- This diagram shows a skip list with four levels (Level 0, Level 1, Level 2, Level 3).
- Similar to the previous diagram, Level 0 is the bottommost sorted linked list.
- Elements have values and pointers. Solid arrows show pointers to the next element in the same level. Dashed arrows represent shortcut pointers that jump over elements in the lower level.
- In this example:
  - Elements 5, 10, 15, 25, and 35 only appear in Level 0.
  - Elements 20, 30, and 50 have a level of 1 (one shortcut pointer).
  - Elements 40 and 60 have a level of 2 (two shortcut pointers).
  - Element 80 has the highest level (3) with three shortcut pointers, skipping all elements in the lower levels.

0.5.3 Lecture code explaination

Skip Lists are a probabilistic alternative to traditional sorted linked lists. They achieve faster search times by utilizing a linked list structure with multiple layers (levels).

Here's a breakdown of the code:

Classes:

- `Node`: This class represents a single node in the SkipList. It has two properties:
  - `key`: An integer value representing the data stored in the node.
  - `forward`: An array of pointers to other nodes. The length of this array determines the level of the node in the SkipList.
- `SkipList`: This class implements the SkipList data structure. It has several properties and methods:
  - `maxLevel`: An integer specifying the maximum level allowed in the SkipList.
  - `P`: A double value between 0 and 1 representing the probability of adding a new level when inserting a node.
  - `level`: An integer representing the current highest level in the SkipList.
  - `head`: A pointer to the head node of the SkipList. This node has a special key value (usually -1) and forward pointers for all possible levels.

Methods:

Helper Methods

- `randomLevel()`: This function generates a random integer value between 1 and `maxLevel` that determines the level of a new node being inserted. It uses the `P` value to simulate a coin toss. With a higher `P`, there's a greater chance of getting a higher level.
- `createNode(int key, int level)`: This function creates a new `Node` object with the specified `key` and `level`.

Insert Element

`insertElement(int key)`: This function inserts a new element with the provided `key` into the SkipList. Here's a simplified explanation of the process:

The provided function `insertElement(int key)` inserts a new element with the specified `key` into the SkipList data structure. Here's a breakdown of how it works:

1. Initialization:

- `Node current = head;`: This line sets a pointer `current` to the head node of the SkipList. This will be used to traverse the list during insertion.
- `Node update[] = new Node[maxLevel + 1];`: This line creates an array of `Node` pointers named `update`. This array will hold references to nodes at each level that will be used to update the forward pointers during insertion.

2. Traversing Levels and Finding Insertion Point:

- `for (int i = level; i >= 0; i-)` : This loop iterates through each level of the SkipList, starting from the highest level (`level`) down to level 0.
  - `while (current.forward[i] != null && current.forward[i].key <key)`: This inner loop iterates within a specific level, moving the `current` pointer forward until it reaches the end of the level (null) or finds a node with a key greater than the `key` being inserted. This ensures we find the last node before the insertion point for the new element.
  - `update[i] = current;`: After finding the appropriate position in the current level, the `update` array at index `i` is assigned the value of the `current` pointer. This essentially stores references to nodes at each level that will be used to link the new node into the SkipList.

3. Checking for Existing Key:

- `current = current.forward[0]`: This line moves the `current` pointer to the node pointed to by the head node's forward pointer at level 0 (the bottommost level). This effectively checks if a node with the same `key` already exists in the list.
- `if (current == null || current.key != key)`: This condition checks if the `current` pointer is null (meaning the key is not present) or if the key of the current node (if found) doesn't match the `key` being inserted. This indicates that we need to insert a new node.

4. Determining Random Level for New Node:

- `int rlevel = randomLevel();`: This line calls the `randomLevel()` function (assumed to be implemented elsewhere) to generate a random level for the new node. The `randomLevel()` function likely uses a probability `P` to determine the level with a higher chance of getting a higher level for the new node.

5. Updating Head Pointers if Needed:

- `if (rlevel >level)` : This condition checks if the randomly generated level (`rlevel`) for the new node is higher than the current highest level (`level`) in the SkipList.
  - `for (int i = (level + 1); i <(rlevel + 1); i++)` : This loop iterates from the current highest level (`level`+ 1) up to the randomly generated level (`rlevel`).

    - `update[i] = head;`: Inside the loop, each element of the `update` array for these new levels is assigned the `head` node. This essentially updates the head node's forward pointers for these higher levels to point to the new node being inserted. This creates a new "tower" in the SkipList if the random level is higher.
  - `level = rlevel;`: This line updates the `level` variable to reflect the new highest level in the SkipList after potentially adding new head pointers.

6. Creating the New Node:

- `Node n = createNode(key, rlevel);`: This line calls the `createNode(key, rlevel)` function (assumed to be implemented elsewhere) to create a new `Node` object with the specified `key` and the randomly generated level (`rlevel`). This function likely allocates memory and sets the initial forward pointers of the new node to null.

7. Updating Forward Pointers for Insertion:

- `for (int i = 0; i <= rlevel; i++)` : This loop iterates through all levels, including the new ones if applicable (up to `rlevel`).
  - `n.forward[i] = update[i].forward[i];`: This line updates the forward pointer of the new node (`n`) at level `i` to point to the node currently stored in the `update` array at the same level `i`. This essentially connects the new node to the previous nodes at each level.
  - `update[i].forward[i] = n;`: This line updates the forward pointer of the node stored in the `update` array at level `i`

Search Element

`searchElement(int key)`: This function searches for an element with the provided `key` in the SkipList. It follows a similar process as `insertElement` to traverse down each level until it finds the key or reaches the end of the list. It then prints whether the key was found or not.

The provided function `searchElement(int key)` searches for a node with the specified `key` in the SkipList data structure. Here's a breakdown of how it works:

1. Initialization:

- `Node current = head;`: This line sets a pointer `current` to the head node of the SkipList. This will be used to traverse the list during the search.
- `System.out.println("\n----------\n");`: This line prints a line separator.

2. Traversing Levels:

- `for (int i = level; i >= 0; i-)`: This loop iterates through each level of the SkipList, starting from the highest level (`level`) down to level 0.

3. Searching Within Each Level:

- `while (current.forward[i] != null && current.forward[i].key <key)`: This loop iterates within a specific level, moving the `current` pointer forward until it reaches the end of the level (null) or finds a node with a key greater than or equal to the `key` being searched for. This ensures we find the last node before (or the node itself) that has a key less than the target `key`.

4. Search Result:

- `if (current.forward[0] == null)` : This condition checks if the loop in step 3 completed without finding a node with a key greater than or equal to the target `key` at level 0 (the bottommost level). This essentially means the target `key` is not present in the SkipList.
  - `System.out.println("Not Found"); return;`: If the key is not found, the function prints a message indicating "Not Found" and exits by returning from the function.

5. Checking the Found Node (if any):

- `current = current.forward[0]`: This line assumes we didn't exit in the previous step (i.e., the key might have been found). Here, the `current` pointer is moved to the node pointed to by the head node's forward pointer at level 0. This effectively moves to the first node in the list (remember, level 0 represents the bottommost level with all actual data).
- `if (current.key == key)` : This condition checks if the key of the current node (potentially the found node) matches the target `key` .
  - `System.out.println("Found key: " + key);`: If the keys match, the function prints a message indicating "Found key: " followed by the actual key value.
- `else` : If the keys don't match (i.e., we moved to a node with a higher key but not the exact target key), it implies the target key is not present in the list.
  - `System.out.println("Not Found");`: The function prints a message indicating "Not Found".

Overall, the `searchElement` function efficiently searches for a node with a specific `key` in the SkipList by utilizing the multi-level structure. It traverses down each level, stopping at the last node before the target key (or the node itself if the key exists). Finally, it checks if the key matches and prints the appropriate message.

Delete Element

`deleteElement(int key)`: This function searches for a node with the provided `key` and removes it from the SkipList. It adjusts the forward pointers of the surrounding nodes to skip over the deleted node. It also checks if the highest level can be reduced after a deletion.

The provided function `deleteElement(int key)` removes a node with the specified `key` from the SkipList data structure. Here's a breakdown of how it works:

1. Debugging Message (Commented Out):

Java

```
// Print a message indicating the key to be deleted (for debugging purposes)
System.out.println("\n" + key + " is to be deleted\n");
```

This line is commented out but serves a debugging purpose. It would print a message indicating the key being deleted.

2. Initialization:

- `Node current = head;`: This line sets a pointer `current` to the head node of the SkipList. This will be used to traverse the list during deletion.

3. Traversing Levels:

- `for (int i = level; i >= 0; i-)`: This loop iterates through each level of the SkipList, starting from the highest level (`level`) down to level 0.

4. Searching Within Each Level:

- `while (current.forward[i] != null && current.forward[i].key <= key)`: This loop iterates within a specific level, moving the `current` pointer forward until it reaches the end of the level (null) or finds a node with a key greater than the `key` being deleted. This ensures we find the node before or the node itself that has the `key` we want to delete.

5. Deletion Logic:

- `if (current.forward[i].key == key)` : This condition checks if the current node's forward pointer at the current level points to the node with the `key` to be deleted.
  - `System.out.println("delete found");`: This line (for debugging) prints a message indicating the deletion target is found.

  - `current.forward[i] = current.forward[i].forward[i]`: This is the core deletion step. It updates the forward pointer of the `current` node at the current level to skip over the node being deleted. It essentially points to the node that comes after the deleted node in that level.
  - `if (current.key == -1 && current.forward[i] == null)` : This condition checks if the deletion happened at level 0 (i.e., the head node) and the head node's forward pointer became null after the deletion. This means the list is now empty.
    * `level-;`: If the list is empty, the level is reduced to reflect that there are no elements anymore.
    * `System.out.println("level deleted");`: This line (for debugging) prints a message indicating the level has been reduced.

6. Moving to the Next Node (if not deleted):

- `else` : If the `key` is not found at the current level (i.e., the loop condition `current.forward[i].key <= key` is not met), the `else` block executes.
  - `current = current.forward[i]`: This line simply moves the `current` pointer to the next node in the current level, continuing the search for the target `key`.

Overall, the `deleteElement` function efficiently removes a node with the specified `key` from the SkipList by adjusting the forward pointers of surrounding nodes to skip over the deleted node. It also handles the case where the deletion empties the list, reducing the highest level (effectively making the list empty).

- `displayList()`: This function prints the contents of the SkipList. It iterates through each level, starting from the highest, and prints the keys of the nodes in that level.

Overall, Skip Lists provide a good balance between insertion, search, and deletion operations. They are well-suited for scenarios where search operations are frequent.

0.5.4 Lecture Code

```
/*
Project:  Skiplist
Programmer: James Goudy
 */
package skiplists_rev24;

class Node {

    // A single node in the SkipList
    public int key;
    public Node[] forward;

    public Node(int key, int level) {
        // Initializes a new Node with a key and an array of forward pointers
        this.key = key;
        forward = new Node[level + 1];

        for (int c = 0; c < level + 1; c++) {
            forward[c] = null;
        }
    }
}
```

```
class SkipList {

    // A SkipList data structure
    private int maxLevel;
    private double P;
    private int level = 0;
    private Node head;

    public SkipList(int maxLevel, double P) {
        // Constructor for SkipList, initializes head node with special key
        this.maxLevel = maxLevel;
        this.P = P;
        level = 0;
        head = new Node( -1, maxLevel);
    }

    int randomLevel() {
        // Generates a random level for a new node
        double r = Math.random();
        int lvl = 0;
        while (r < P && lvl < maxLevel) {
            lvl++;
            r = Math.random();
        }
        return lvl;
    }

    static Node createNode(int key, int level) {
        // Creates a new node with a key and specified level
        Node n = new Node(key, level);
        return n;
    }

    public void insertElement(int key) {
        // Inserts a new element with the provided key into the SkipList
        Node current = head;
        Node update[] = new Node[maxLevel + 1];

        for (int i = level; i >= 0; i - -) {
            // Traverse each level, find the last node before the key
            while (current.forward[i] != null
                    && current.forward[i].key < key) {
                current = current.forward[i];
            }
            update[i] = current;
        }

        current = current.forward[0];

        if (current == null || current.key != key) {
            // If key not found, insert a new node
            int rlevel = randomLevel();
```

```
            if (rlevel > level) {
                // If random level is higher than current level,
                // update head pointers
                for (int i = (level + 1); i < (rlevel + 1); i++) {
                    update[i] = head;
                }
                level = rlevel;
            }

            Node n = createNode(key, rlevel);

            for (int i = 0; i <= rlevel; i++) {
                // Update forward pointers of new node
                // and previous nodes at each level
                n.forward[i] = update[i].forward[i];
                update[i].forward[i] = n;
            }
            System.out.println("Successfully Inserted key " + key + "\n");
        }
    }

    void searchElement(int key) {
        // Searches for an element with the provided key in the SkipList
        Node current = head;
        System.out.println("\n - - - - - - - - - - - - - - - - - - - - -\n");
        for (int i = level; i >= 0; i - -) {
            // Traverse each level, find the last node before the key
            while (current.forward[i] != null
                    && current.forward[i].key < key) {
                current = current.forward[i];
            }
        }

        if (current.forward[0] == null) {
            // Key not found
            System.out.println("Not Found");
            return;
        }

        current = current.forward[0];

        if (current.key == key) {
            // Key found
            System.out.println("Found key: " + key);
        } else {
            // Key not found
            System.out.println("Not Found");
        }
    }

    // - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
    void deleteElement(int key) {
        // Print a message indicating the key
```

```
        // to be deleted (for debugging purposes)
        System.out.println("\n" + key + " is to be deleted\n");

        Node current = head;

        // Traverse down each level of the SkipList
        for (int i = level; i >= 0; i - -) {
            // Search for the node with the key or
            // the last node before the key at the current level
            while (current.forward[i] != null
                    && current.forward[i].key <= key) {
                // If the key is found in the current level
                if (current.forward[i].key == key) {
                    System.out.println("delete found");

                    // Update the forward pointer
                    // of the current node to skip the deleted node
                    current.forward[i] = current.forward[i].forward[i];

                    // If the head node's forward pointer
                    // becomes null after deletion
                    // and the current level is 0,
                    // it means the list is empty,
                    // so reduce the level
                    if (current.key == -1 && current.forward[i] == null) {
                        level - -;
                        System.out.println("level deleted");
                    }
                } else {
                    // If the key is not found at the current level,
                    // move to the next node
                    current = current.forward[i];
                }
            }
        }
    }

    void displayList() {
        // Prints the contents of the SkipList
        System.out.println("\n*****Skip List*****\n");
        for (int i = 0; i <= level; i++) {
            Node node = head.forward[i];
            System.out.print("Level " + i + ": ");
            // Traverse each level and print the keys of the nodes
            while (node != null) {
                System.out.print(node.key + " ");
                node = node.forward[i];
            }
            System.out.println("");
        }
    }
}
```

```
public class Skiplists_rev24 {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        // create SkipList object with MAXLVL and P > 0 and P < 1
        int MAXLVL = 4;

        // the higer the probability number,
        // the greaterchance of more levels.
        double prob = .75;

        SkipList lst = new SkipList(MAXLVL, prob);

        //lst.insertElement(35);
        lst.insertElement(3);
        lst.insertElement(6);
        lst.insertElement(7);
        lst.insertElement(9);
        lst.insertElement(12);
        lst.insertElement(19);

        lst.insertElement(17);

        lst.insertElement(26);
        lst.insertElement(21);
        lst.insertElement(25);
        lst.displayList();

        lst.searchElement(19);
        lst.searchElement(25);
        lst.searchElement(9);
        lst.searchElement(5);
        lst.searchElement(55);

        lst.deleteElement(25);
        lst.displayList();

        lst.deleteElement(7);
        lst.displayList();

        lst.deleteElement(3);
        lst.displayList();

        lst.deleteElement(17);
        lst.displayList();

        System.out.println("\nbye\n");
    }

}
```

Output

Note: Output will be different for the levels each time due to the randomness factor

```
Successfully Inserted key 3

Successfully Inserted key 6

Successfully Inserted key 7

Successfully Inserted key 9

Successfully Inserted key 12

Successfully Inserted key 19

Successfully Inserted key 17

Successfully Inserted key 26

Successfully Inserted key 21

Successfully Inserted key 25


*****Skip List*****

Level 0: 3 6 7 9 12 17 19 21 25 26
Level 1: 3 6 7 9 12 17 19 21 25 26
Level 2: 3 6 7 9 17 19 25 26
Level 3: 3 6 9 17 19 25
Level 4: 3 6 9 17 19

 - - - - - - - - - - - - - - - - - - - -

Found key: 19

 - - - - - - - - - - - - - - - - - - - -

Found key: 25

 - - - - - - - - - - - - - - - - - - - -

Found key: 9

 - - - - - - - - - - - - - - - - - - - -

Not Found

 - - - - - - - - - - - - - - - - - - - -

Not Found

25 is to be deleted
```

```
delete found
delete found
delete found
delete found

*****Skip List*****

Level 0: 3 6 7 9 12 17 19 21 26
Level 1: 3 6 7 9 12 17 19 21 26
Level 2: 3 6 7 9 17 19 26
Level 3: 3 6 9 17 19
Level 4: 3 6 9 17 19

7 is to be deleted

delete found
delete found
delete found

*****Skip List*****

Level 0: 3 6 9 12 17 19 21 26
Level 1: 3 6 9 12 17 19 21 26
Level 2: 3 6 9 17 19 26
Level 3: 3 6 9 17 19
Level 4: 3 6 9 17 19

3 is to be deleted

delete found
delete found
delete found
delete found
delete found

*****Skip List*****

Level 0: 6 9 12 17 19 21 26
Level 1: 6 9 12 17 19 21 26
Level 2: 6 9 17 19 26
Level 3: 6 9 17 19
Level 4: 6 9 17 19

17 is to be deleted

delete found
delete found
delete found
delete found
delete found

*****Skip List*****
```

```
Level 0: 6 9 12 19 21 26
Level 1: 6 9 12 19 21 26
Level 2: 6 9 19 26
Level 3: 6 9 19
Level 4: 6 9 19

bye
```

0.6 Graphs

In computer science, a **graph** is an abstract representation of a set of objects, known as **vertices** or **nodes**, connected by a set of **edges**. Graphs offer a flexible way to depict relationships and interactions between different entities. They can be either **directed** (where edges have a specific direction) or **undirected** (where edges have no specific direction) 12. Let's delve deeper into the world of graphs:

1. **Graph Data Structure**: A graph data structure consists of nodes (vertices) and edges. It's used to represent relationships between different entities. Graph algorithms manipulate and analyze graphs, solving various problems like finding the shortest path or detecting cycles 1.

2. **Types of Graphs**:
   - **Directed Graph (Digraph)**: Edges have a specific direction, forming an ordered pair (u, v).
   - **Undirected Graph**: Edges have no specific direction, forming an unordered pair {u, v}.
   - **Weighted Graph**: Edges have associated weights (e.g., distances, costs).
   - **Cyclic Graph**: Contains cycles (closed loops).
   - **Acyclic Graph**: Has no cycles.
   - **Connected Graph**: Every pair of vertices is connected by a path.
   - **Disconnected Graph**: Contains isolated components.
   - **Bipartite Graph**: Vertices can be divided into two disjoint sets such that no edge connects vertices within the same set.

3. **Graph Algorithms**:
   - **Breadth-First Search (BFS)**: Traverses the graph layer by layer.
   - **Depth-First Search (DFS)**: Explores as far as possible along each branch before backtracking.
   - **Dijkstra's Algorithm**: Finds the shortest path in weighted graphs.
   - **Minimum Spanning Tree (MST)**: Connects all vertices with minimum total edge weight.
   - **Topological Sorting**: Orders vertices based on dependencies.
   - **Articulation Points**: Critical vertices in a connected graph.
   - **Strongly Connected Components**: Maximal strongly connected subgraphs.
   - **Eulerian Path/Circuit**: Visits each edge exactly once.
   - **Bellman–Ford Algorithm**: Detects negative cycles.
   - **Transitive Closure**: Determines reachability between vertices.

Graphs are everywhere in computer science, from social networks to algorithms. They provide a conceptual and computational framework for solving complex problems. So next time you encounter a network, think of it as a graph connecting nodes of information!

0.6.1 A Comparative Analysis of Breadth-First Search and Depth-First Search Algorithms

1. Introduction to Graph Traversal: BFS and DFS

Graphs serve as fundamental data structures in computer science, representing entities (nodes or vertices) and the connections or relationships between them (edges). Many computational problems across diverse domains, from network routing and social network analysis to state-space exploration in artificial intelligence, can be modeled using graphs. A common requirement when working with graphs is the ability to systematically visit or process each node. This process is known as graph traversal. Efficient graph traversal algorithms are essential for exploring graph structures and solving problems like finding paths, checking connectivity, identifying cycles, and analyzing network properties.

Two of the most foundational and widely used graph traversal algorithms are Breadth-First Search (BFS) and Depth-First Search (DFS). While both aim to visit all reachable nodes from a starting point, they employ fundamentally different strategies, leading to distinct characteristics, performance trade-offs, and suitability for various applications. BFS explores the graph level by level, expanding outwards from the source, whereas DFS explores as deeply as possible along one path before backtracking to explore alternatives. This report provides a detailed technical explanation and comparative analysis of BFS and DFS, examining their underlying mechanisms, core data structures (queues and stacks/recursion), strengths, weaknesses, and practical real-world applications.

2. Breadth-First Search (BFS): The Level-by-Level Explorer

Fundamental Mechanism Breadth-First Search operates by exploring a graph in layers, systematically visiting all nodes at a given distance (in terms of number of edges) from a starting source node before moving on to nodes at the next distance level.7 This methodical, breadth-wise expansion can be visualized as ripples spreading outwards in concentric circles after a pebble is dropped into still water 8, or akin to exploring a multi-story building by examining every room on one floor before proceeding to the next. The algorithm starts at a designated source vertex and visits all its immediate neighbors (nodes one edge away). Then, it visits all the neighbors of those nodes that haven't been visited yet (nodes two edges away), and continues this process, level by level. This level-order exploration strategy is fundamental to BFS's properties, most notably its ability to find the shortest path in terms of the number of edges in unweighted graphs.

The Role of the Queue Data Structure (FIFO) The key to achieving this level-by-level exploration lies in BFS's use of a queue data structure. A queue operates on the First-In, First-Out (FIFO) principle: the first element added to the queue is the first one to be removed.1 In the context of BFS, the queue stores nodes that have been discovered but whose neighbors have not yet been fully explored.

The process typically involves adding (enqueuing) the starting node to the queue. Then, while the queue is not empty, a node is removed (dequeued) from the front. When this node is processed, all its adjacent neighbors that have not yet been visited are added (enqueued) to the back of the queue. This FIFO behavior ensures that nodes are processed in the order of their distance from the source. All nodes at distance `k` are dequeued and processed before any node at distance `k+1` is dequeued, because the nodes at distance `k+1` were only added to the queue *after* the nodes at distance `k`.

This strict ordering imposed by the FIFO queue is the direct mechanism enabling BFS to guarantee finding the shortest path in unweighted graphs. Since the algorithm explores all paths of length `k` before exploring any path of length `k+1`, the first time it encounters a target node, it must have reached it via a path with the minimum possible number of edges.1

Step-by-Step BFS Algorithm Walkthrough The BFS algorithm can be implemented systematically as follows 8:

1. **Initialization:**

- Create an empty queue (e.g., using a `deque` in Python or `LinkedList` in Java) to store nodes pending exploration.
- Create a set or boolean array called `visited` to keep track of nodes already encountered. This prevents reprocessing nodes and potential infinite loops in graphs containing cycles.
- Optionally, create a map or dictionary called `predecessors` to store the path taken to reach each node (useful for reconstructing the shortest path). Initialize it as empty.
- Select a starting node `s`. Add `s` to the `visited` set. Enqueue `s` into the queue. If using `predecessors`, set `predecessors[s]` to null.

2. **Iteration:**

- While the queue is not empty:
  - Dequeue the node from the front of the queue; let this be `currentNode`.
  - Process `currentNode`. This might involve checking if it is the target node, performing an operation, or simply marking its traversal.
  - Retrieve all neighbors of `currentNode`.
  - For each `neighbor` of `currentNode` :
    - Check if `neighbor` is in the `visited` set.
    - If `neighbor` has not been visited:
      - Add `neighbor` to the `visited` set.
      - If using `predecessors`, set `predecessors[neighbor] = currentNode`.
      - Enqueue `neighbor` into the back of the queue.

3. **Termination:**

- The algorithm terminates when the queue becomes empty, signifying that all nodes reachable from the starting node `s` have been visited, or when the target node (if specified) is dequeued and processed.

The `visited` set plays a critical role in ensuring the correctness and termination of BFS, particularly when dealing with graphs that contain cycles.10 Without tracking visited nodes, the algorithm could enter an infinite loop by repeatedly enqueueing and dequeueing nodes within a cycle (e.g., A ->B ->C ->A). The `visited` set prevents a node from being enqueued and processed more than once, effectively breaking potential cycles during traversal.10

3. Depth-First Search (DFS): The Deep Dive Explorer

Fundamental Mechanism In contrast to BFS's breadth-wise exploration, Depth-First Search (DFS) employs a strategy of exploring as far as possible along a single path or branch before backtracking. The core idea is to go deeper into the graph whenever possible. Starting from a source node, DFS selects an unvisited neighbor and recursively explores from that neighbor. This continues until it reaches a node with no unvisited neighbors (a "dead end") or the target node.

A crucial component of DFS is **backtracking**. When the algorithm reaches a dead end or completes the exploration from a particular node, it returns to the node from which it arrived (the previous node in the path) and attempts to explore other unvisited neighbors from that point This process resembles navigating a complex maze: one follows a chosen path until it terminates, then retraces steps back to the last junction where an alternative path was available, and explores that new path.

The Role of the Stack (LIFO) or Recursion DFS is typically implemented using either recursion, which implicitly utilizes the program's function call stack, or an iterative approach using an explicit stack data structure.2 A stack operates on the Last-In, First-Out (LIFO) principle.

- **Recursive Implementation:** The recursive approach mirrors the DFS definition directly. A function calls itself for each unvisited neighbor, naturally diving deeper into the graph. The function call stack implicitly keeps track of the path taken and the nodes to return to upon backtracking (when a function call returns).

- **Iterative Implementation (Stack):** The iterative version uses an explicit stack. The starting node is pushed onto the stack. In each step, a node is popped from the stack. If it hasn't been visited, it's marked as visited and processed. Then, its unvisited neighbors are pushed onto the stack. The LIFO nature ensures that the most recently discovered neighbor (the one deepest along the current path) is explored next.

The choice between recursive and iterative implementations involves trade-offs. Recursion often leads to more concise and arguably more elegant code that directly reflects the algorithm's definition.21 However, for very deep graphs, the recursive approach can lead to a stack overflow error if the depth of recursion exceeds the limit imposed by the system's call stack.18 The iterative approach, using an explicit stack managed in the program's heap memory, avoids this recursion depth limitation and is only constrained by available system memory, making it potentially more robust for extremely deep traversals, though it might be slightly more complex to write.18

Step-by-Step DFS Algorithm Walkthrough Here are the step-by-step procedures for both implementations 2:

Recursive DFS:

1. **Initialization:** Create a `visited` set (or boolean array) to track visited nodes.
2. Define Recursive Function:

   Create a function, say `DFS_recursive(graph, currentNode, visited)` :

   - Mark `currentNode` as visited (add it to the `visited` set).
   - Process `currentNode` (e.g., print it, add to a list).
   - For each `neighbor` of `currentNode` in the `graph` :
     - If `neighbor` is not in the `visited` set:
       - Call `DFS_recursive(graph, neighbor, visited)`.
3. **Initial Call:** Start the traversal by calling `DFS_recursive(graph, startNode, visited)`. If the graph might be disconnected, iterate through all nodes and call `DFS_recursive` if a node hasn't been visited yet.

Iterative DFS (using Stack):

1. **Initialization:** Create an empty stack. Create a `visited` set.
2. **Start:** Push the `startNode` onto the stack.
3. Iteration:

   While the stack is not empty:

   - Pop a node from the top of the stack; let this be `currentNode`.
   - If `currentNode` is not in the `visited` set:
     - Mark `currentNode` as visited (add it to the `visited` set).
     - Process `currentNode`.
     - Retrieve all neighbors of `currentNode`.
     - For each `neighbor` of `currentNode` :
       - If `neighbor` is not in the `visited` set:
         - Push `neighbor` onto the stack. (Note: Pushing neighbors in reverse order of how they appear in the adjacency list often helps mimic the traversal order of the typical recursive implementation).
4. **Termination:** The loop finishes when the stack is empty, meaning all reachable nodes have been explored.

The backtracking mechanism is an inherent consequence of the LIFO nature of the stack (explicit or implicit). When the exploration along a certain path reaches a node with no further unvisited neighbors, the algorithm naturally reverts to the node it came from – either by returning from the recursive function call or by popping the next element from the explicit stack, which corresponds to the parent node in the traversal path. This allows the algorithm to systematically explore alternative branches from previously visited nodes.2

4. Analyzing Breadth-First Search (BFS)

Strengths BFS possesses several key strengths stemming directly from its level-by-level exploration strategy:

- **Shortest Paths (Unweighted Graphs):** BFS is guaranteed to find the shortest path, measured by the number of edges, between a source node and any other reachable node in an unweighted graph. As explained earlier, the use of a queue ensures that nodes closer to the source are always visited before nodes farther away. This makes BFS the standard algorithm for solving shortest path problems where all edge weights are uniform (or non-existent).
- **Completeness:** BFS is a complete algorithm, meaning if a path exists from the source node to a target node, BFS is guaranteed to find it. Because it systematically explores every reachable node layer by layer, it cannot get stuck in a deep path while potentially missing a solution at a shallower level. It explores the entire reachable portion of the graph exhaustively.

Weaknesses The primary drawback of BFS relates to its resource consumption:

- **Memory Consumption:** BFS can be memory-intensive, particularly for graphs with a high branching factor (nodes having many neighbors).3 The algorithm needs to store all nodes at the frontier of the search in the queue simultaneously.
- The size of the queue, and thus the memory requirement, is determined by the maximum number of nodes present at any single level or depth in the graph. For graphs that are "wide" (having many nodes at the same distance from the source) but relatively "shallow", the queue can grow very large, potentially requiring space proportional to the total number of vertices (O(V)) in the worst-case scenario.6 This can be a significant limitation when dealing with massive graphs where memory is constrained.

5. Analyzing Depth-First Search (DFS)

Strengths DFS offers a different set of advantages, often complementing the weaknesses of BFS:

- **Memory Efficiency:** DFS generally requires less memory than BFS, especially for graphs that are very deep but not necessarily wide. The space complexity is primarily determined by the maximum depth of the path being explored, as the stack (explicit or implicit) only needs to store the nodes along the current path from the source to the current node.12 For a graph with maximum depth H, the space complexity can be O(H), which can be significantly less than O(V) for deep, narrow graphs where BFS might struggle with memory.
- **Pathfinding Utility:** While not optimal for shortest paths, DFS is useful for simply determining if a path exists between two nodes, or for finding *any* path. It's also well-suited for exploring all possible paths or configurations, such as in solving puzzles or traversing decision trees.
- **Cycle Detection:** DFS is highly effective for detecting cycles in both directed and undirected graphs. By keeping track of nodes currently in the recursion path (using recursion stack status or an auxiliary set in iterative DFS), DFS can identify a "back edge" – an edge leading to an ancestor node already in the current path – which signifies a cycle.
- **Topological Sorting:** DFS is a standard algorithm for performing topological sorting on Directed Acyclic Graphs (DAGs). This process orders vertices such that for every directed edge from vertex `u` to vertex `v`, `u` comes before `v` in the ordering. This is crucial for scheduling tasks with dependencies. DFS achieves this by adding a node to the front of the sorted list only after all its descendants have been fully explored and added.

- The inherent nature of DFS, exploring full paths before backtracking, makes it particularly well-suited for problems where the structure of paths, connectivity, or dependencies is the primary concern, such as cycle detection, finding connected components, and topological sorting.

Weaknesses DFS also has notable limitations:

- **Path Optimality:** DFS does not guarantee finding the shortest path between two nodes in a graph. Because it explores deeply along one branch first, it might discover a very long path to the target before it backtracks to explore a potentially much shorter path.
- **Completeness Issues:** DFS is not complete for infinite graphs. It could potentially follow an infinitely long path and never backtrack to explore other parts of the graph. In finite graphs with cycles, if visited nodes are not properly tracked (using a `visited` set), DFS can get trapped in an infinite loop, also leading to incompleteness.16 However, when implemented correctly with visited node tracking, DFS is complete for finite graphs.
- There exists a fundamental trade-off: DFS might find *a* solution relatively quickly if the solution happens to lie deep down the first path explored. However, this potential speed advantage comes at the cost of potentially finding a non-optimal (longer) path. BFS, while potentially slower to reach deep nodes, guarantees finding the shortest path in unweighted graphs.

6. Comparative Analysis: BFS vs. DFS

Key Differences Summarized BFS and DFS represent two distinct approaches to graph traversal with contrasting characteristics :

- **Strategy:** BFS explores level by level (breadth-first); DFS explores as deep as possible before backtracking (depth-first).
- **Data Structure:** BFS uses a Queue (FIFO); DFS uses a Stack (LIFO) or recursion (implicit stack).
- **Pathfinding:** BFS finds the shortest path in unweighted graphs; DFS finds *a* path, but not necessarily the shortest.
- **Memory:** BFS memory usage depends on graph width (can be high); DFS memory usage depends on graph depth (often lower, especially for deep graphs).
- **Completeness:** BFS is complete; DFS is complete for finite graphs (with visited tracking) but not for infinite graphs.

Comparison Table The following table provides a concise side-by-side comparison:

Feature	**Breadth-First Search (BFS)**	**Depth-First Search (DFS)**
Traversal Strategy	Level-by-level	Depth-first (explore branch fully, then backtrack)
Data Structure	Queue (FIFO)	Stack (LIFO) or Recursion (Call Stack)
Path Optimality	Finds shortest path (unweighted graphs)	Does not guarantee shortest path
Completeness	Complete	Complete (finite graphs, with cycle handling); Not complete (infinite graphs)
Memory Complexity	O(V) worst case (depends on graph width)	O(H) or O(V) worst case (depends on graph depth)
Time Complexity	O(V + E) (adjacency list)	O(V + E) (adjacency list)
Key Use Cases	Shortest path (unweighted), Web Crawling, Social Network Analysis, Network Broadcast	Cycle Detection, Topological Sort, Maze Solving, Path Existence, Exploring Hierarchies

(V = number of vertices, E = number of edges, H = maximum depth of the graph)

Guidance on Choosing the Right Algorithm The choice between BFS and DFS depends heavily on the specific problem requirements and the characteristics of the graph being traversed 6:

- Use BFS when:
  - The shortest path (in terms of edges) in an unweighted graph is required.
  - Exploring nodes layer by layer or finding nodes closest to the source is important.
  - The graph is relatively shallow, or memory is not a primary constraint (as wide graphs can consume significant memory).
  - The solution is expected to be relatively close to the starting node.
- Use DFS when:
  - Finding *any* path or simply checking for path existence is sufficient.
  - Exploring the full depth of paths is necessary (e.g., for cycle detection, topological sorting, exploring all possibilities in a puzzle).
  - Memory efficiency is crucial, especially if the graph might be very deep but not excessively wide.
  - The solution might be located deep within the graph structure.

Ultimately, neither algorithm is universally superior. The optimal choice involves a careful consideration of the trade-offs between path optimality, memory usage, completeness requirements, and the specific nature of the problem being addressed.

7. Real-World Applications of Breadth-First Search (BFS)

BFS's level-by-level exploration and shortest path guarantee in unweighted graphs make it suitable for numerous practical applications:

- **Networking:**
  - **Shortest Path Routing:** Used in network protocols to find the path with the minimum number of hops between devices (routers, switches). This is fundamental for efficient data packet delivery in networks where hop count is the primary metric.
  - **Broadcasting:** Employed to efficiently disseminate information from one node to all other nodes in a network, ensuring minimal propagation delay in terms of hops. Examples include network service discovery protocols or distributing updates.
  - **Peer-to-Peer (P2P) Networks:** Used to discover nearby peers in networks like BitTorrent. Starting from a known peer, BFS can find other peers within a certain number of network hops.
- **Web Crawling:**
- **Search Engine Indexing:** Web crawlers used by search engines often employ BFS to discover and index web pages. Starting from a set of known "seed" URLs, the crawler explores links level by level. This ensures that pages closer (in terms of link distance) to the seed URLs are typically discovered and indexed before pages that are many links away, providing a broad initial coverage of the web.
- **Social Network Analysis:**
  - **Finding Degrees of Separation:** BFS is ideal for calculating the shortest connection path between two users in a social network (e.g., finding out if someone is a "friend of a friend"). This directly applies the shortest path property.
  - **Friend Recommendations:** Social platforms can use BFS to suggest potential friends by exploring the network outwards from a user. Users found within a small number of hops (e.g., friends of friends) but not yet connected are prime candidates for recommendation.
- **Other Examples:**
  - **GPS Navigation:** While complex routing uses weighted graphs and algorithms like Dijkstra's or A*, BFS can be used in simpler models to find routes with the minimum number of turns or road segments.
  - **Connected Components:** Identifying all nodes reachable from a starting node, which helps in finding connected components in an undirected graph.
  - **Puzzle Solving:** Finding the minimum number of moves required to solve certain puzzles,

like the 8-puzzle or Rubik's Cube (by exploring the state space graph).
  - **AI Pathfinding:** Used in games for pathfinding on grids or unweighted maps where the shortest path in terms of steps is desired.

A common theme across many BFS applications is the need to find something "minimal" or "closest" – the shortest path, the nearest neighbors, the minimum number of steps – which aligns perfectly with its systematic, level-by-level exploration.

8. Real-World Applications of Depth-First Search (DFS)

DFS's strategy of deep exploration and backtracking lends itself to applications focused on path discovery, structural properties, and dependencies:

- **Maze Solving:**
  - DFS is a natural fit for finding *a* path through a maze from an entrance to an exit. It explores one possible path completely until it hits a dead end, then backtracks to the last junction and tries an alternative path. This mimics a common human strategy for maze solving. The backtracking mechanism is essential for systematically exploring all possibilities from junctions.
- **Topological Sorting:**
- **Task Scheduling:** DFS is fundamental for topological sorting in Directed Acyclic Graphs (DAGs). This is used extensively in scheduling systems where tasks have prerequisites (e.g., ordering course requirements, resolving dependencies in software build systems or project management). DFS ensures that a task is added to the sorted list only after all tasks that depend on it have been processed.
- **Cycle Detection:**
- **Dependency Analysis:** Detecting cycles is crucial in systems where circular dependencies are problematic, such as in software module imports, spreadsheet formulas, or database schemas. DFS can efficiently detect such cycles by identifying back edges during traversal.
- **Hierarchical Structures (File Systems):**
  - **File System Traversal:** DFS is well-suited for navigating hierarchical structures like file systems. Operations like searching for a file within a directory and its subdirectories, calculating the total size of a directory tree, or performing recursive operations naturally follow a depth-first pattern.25
- **Other Examples:**
  - **Pathfinding:** Determining if *any* path exists between two points in a network or graph.
  - **Solving Puzzles:** Used in solving puzzles that involve exploring sequences of moves or states, like Sudoku, where exploring one possibility deeply before backtracking is effective.
  - **Finding Connected Components:** Similar to BFS, DFS can be used to identify connected components in a graph by starting a traversal from an unvisited node and marking all reachable nodes.
  - **Compiler Design:** Used in various phases of compilation, such as analyzing control flow graphs.
  - **Artificial Intelligence:** Exploring game trees or state spaces in planning problems.

The applications of DFS often revolve around exhaustive exploration of possibilities along paths, analyzing the structure of connections (cycles, connectivity), or handling hierarchical or dependent relationships.

9. Conclusion

Breadth-First Search (BFS) and Depth-First Search (DFS) are cornerstone algorithms for graph traversal, each offering a unique approach with distinct advantages and disadvantages. BFS, characterized by its level-by-level exploration using a queue (FIFO), excels at finding the shortest path in unweighted graphs and guarantees completeness. However, its memory consumption can be substantial for graphs with large branching factors.

Conversely, DFS employs a depth-first strategy, utilizing a stack (LIFO) or recursion to explore as far as possible along each branch before backtracking. This often results in lower memory usage, particularly for deep graphs, and makes it highly effective for tasks like cycle detection, topological sorting, and exploring hierarchical structures or exhaustive path possibilities (like maze solving). Its main drawbacks include the lack of a guarantee for finding optimal (shortest) paths and potential incompleteness in infinite graphs.

Both algorithms typically exhibit a time complexity of O(V + E) when implemented with adjacency lists, making them efficient for many graph sizes. The choice between BFS and DFS is not about inherent superiority but about selecting the appropriate tool for the specific problem context. Understanding their fundamental mechanisms, performance characteristics, and the trade-offs between path optimality, memory usage, and exploration strategy is crucial for computer scientists and software engineers aiming to design efficient and effective solutions for graph-related problems. Their wide-ranging applications underscore their enduring importance in diverse fields, from network engineering and web technology to artificial intelligence and bioinformatics.

0.6.2 Understanding Breadth-First Search (BFS)

What is BFS?

Imagine you're exploring a maze. BFS is like exploring it level by level. You start at the entrance (the *source node*). First, you check all the paths directly connected to the entrance (one step away). Then, you explore all the paths that are two steps away from the entrance, then three steps away, and so on. You explore *broadly* before going *deeper* into any one path.

In technical terms, BFS is an algorithm for traversing or searching tree or graph data structures. It starts at a selected node (the source) and explores all of the neighbor nodes at the present depth or level prior to moving on to the nodes at the next depth level. It uses a queue (First-In, First-Out) data structure to keep track of the next node to visit.

Why Use BFS?

BFS is particularly useful for a few key tasks:

1. **Finding the Shortest Path:** In graphs where all edges have the same "cost" or weight (like in the maze example, where each step is equal), BFS is guaranteed to find the shortest path between the starting node and any other reachable node in terms of the number of edges.
2. **Level Order Traversal:** It naturally visits nodes level by level, which can be useful in hierarchical structures.
3. **Connectivity Checking:** You can use BFS to determine if a node is reachable from a starting node or to find all reachable nodes.

Real-Life Examples BFS is used in many applications:

1. **Social Networks:** Finding friends-of-friends. If you want to see all people within 'X' connections of you, BFS is a natural fit. It explores your direct friends (level 1), then their friends (level 2), and so on.
2. **GPS Navigation:** Finding the shortest route (in terms of number of turns or segments, not necessarily distance/time if roads have different speeds/lengths) between two points on a map.
3. **Web Crawlers:** Search engines use crawlers to discover web pages. A crawler might start at a seed page and use BFS to find all pages linked from it (level 1), then all pages linked from those pages (level 2), etc.
4. **Network Broadcasting:** Sending a message to all nodes in a network efficiently.
5. **Finding Connected Components:** Identifying groups of interconnected nodes in a graph.

Code Example

Code Analysis

1. `BFS` Class: This class represents the graph itself.
   - `numVertices`: Stores the total number of nodes (vertices) in the graph.
   - `adjacencyList`: This is the core data structure for storing the graph. It's a `List` where each index corresponds to a vertex. The element at each index is a `LinkedList<Integer>` containing all the vertices directly connected to that vertex (its neighbors). This is called an *adjacency list representation.*

- `BFS(int vertices)` (Constructor): Initializes the graph with a specific number of vertices and creates empty adjacency lists for each.
- `addEdge(int source, int destination)`: Adds a connection (an edge) between two vertices. Since it adds the destination to the source's list *and* the source to the destination's list, it creates an *undirected* edge (meaning the connection goes both ways).

2. **`performBFS(int startVertex)`** Method: This is where the BFS magic happens.
   - `visited`: A boolean array to keep track of which vertices have already been visited. This prevents infinite loops in graphs with cycles.
   - `queue`: A `Queue` (implemented using `LinkedList`) is used to store the vertices to be visited. BFS relies on the FIFO (First-In, First-Out) nature of a queue.
   - **Initialization:** The `startVertex` is marked as visited and added to the queue.
   - **`Traversal Loop (while (!queue.isEmpty()))`**: The loop continues as long as there are nodes in the queue waiting to be processed.
     - `currentVertex = queue.poll()`: Removes the next vertex from the front of the queue. This is the vertex currently being visited.
     - `System.out.print(currentVertex + " ");`: Prints the visited vertex.
     - **`Neighbor Exploration (for (int neighbor : ...)):`** It iterates through all the neighbors of the `currentVertex` (obtained from the `adjacencyList`).
     - `if (!visited[neighbor])`: Checks if the neighbor has already been visited.
     - **Enqueueing:** If the neighbor hasn't been visited, it's marked as `visited` and added to the *end* of the queue (`queue.offer(neighbor)`). This ensures that nodes at the current level are fully explored before moving to the next level.

3. **`BreadthFirst_Rev2504`** Class (Main Program):
   - `main(String[] args)`: The entry point of the program.
   - Creates a `BFS` graph object with 11 vertices.
   - Calls `addEdge` multiple times to define the connections in the graph, matching the diagram in the comments.
   - Calls `performBFS(0)` to run BFS starting from vertex 0.
   - Calls `performBFS(6)` to run BFS starting from vertex 6, demonstrating how the traversal changes based on the starting point.

In essence, the code sets up a graph using adjacency lists and then implements the BFS algorithm using a queue and a visited array to explore the graph level by level, printing the nodes in the order they are visited.

“‘java /* BREADTH FIRST Developer: James Goudy */ package breadthfirst_rev2504a;
import java.util.LinkedList; import java.util.Queue; import java.util.List; import java.util.ArrayList;
// Class representing a graph using adjacency lists class BFS {

```
// Total number of vertices in the graph
private int numVertices;

// Adjacency list to store edges
private List<LinkedList<Integer>> adjacencyList;

// Constructor to initialize the graph
public BFS(int vertices) {
    this.numVertices = vertices;
    this.adjacencyList = new ArrayList<>(vertices);

    // Initialize each adjacency list for every vertex
```

```
    for (int i = 0; i < vertices; i++) {
        this.adjacencyList.add(new LinkedList<>());
    }
}

// Method to add an edge between two vertices
public void addEdge(int source, int destination) {

    this.adjacencyList.get(source).add(destination);

    // For an undirected graph
    this.adjacencyList.get(destination).add(source);
}

// Method to perform Breadth -First Search starting from a given vertex (Iterative)
public void performBFS(int startVertex) {

    // Track visited vertices
    boolean[] visited = new boolean[numVertices];

    // Queue to maintain BFS traversal order
    Queue<Integer> queue = new LinkedList<>();

    // Mark start vertex as visited
    visited[startVertex] = true;

    // Add start vertex to the queue
    queue.offer(startVertex);

    System.out.println("Breadth -First Search (Iterative) starting from vertex "
                                        + startVertex + ":");

    // Continue BFS traversal until queue becomes empty
    while (!queue.isEmpty()) {

        // Dequeue vertex from the queue
        int currentVertex = queue.poll();

        // Print the current vertex
        System.out.print(currentVertex + " ");

        // Explore all adjacent vertices of the current vertex
        for (int neighbor : this.adjacencyList.get(currentVertex)) {

            // If neighbor is not visited yet
            if (!visited[neighbor]) {

                // Mark neighbor as visited
                visited[neighbor] = true;

                // Add neighbor to the queue
                queue.offer(neighbor);
            }
```

```
        }
    }

    // Print newline after traversal completion
    System.out.println();
}

// Method to perform Breadth -First Search starting from a given vertex (Recursive)
public void performBFSRecursive(int startVertex) {
    boolean[] visited = new boolean[numVertices];
    Queue<Integer> queue = new LinkedList<>();

    System.out.println("Breadth -First Search (Recursive) starting from vertex "
                                        + startVertex + ":");

    visited[startVertex] = true;
    queue.offer(startVertex);
    recursiveBFS(queue, visited);
    System.out.println();
}

private void recursiveBFS(Queue<Integer> queue, boolean[] visited) {
    if (queue.isEmpty()) {
        return;
    }

    int currentVertex = queue.poll();
    System.out.print(currentVertex + " ");

    for (int neighbor : this.adjacencyList.get(currentVertex)) {
        if (!visited[neighbor]) {
            visited[neighbor] = true;
            queue.offer(neighbor);
        }
    }
    recursiveBFS(queue, visited);
}

   }
   public class BreadthFirst_Rev2504a {

/**
 * @param args the command line arguments
 */
public static void main(String[] args)
{
    BFS graph = new BFS(11);

    graph.addEdge(0, 1);
    graph.addEdge(0, 2);
    graph.addEdge(1, 3);
    graph.addEdge(2, 4);
    graph.addEdge(4, 5);
```

```
    graph.addEdge(0, 6);
    graph.addEdge(0, 7);
    graph.addEdge(7, 8);
    graph.addEdge(7, 9);

    graph.addEdge(9, 10);

    graph.performBFS(0);
    graph.performBFSRecursive(0);

    graph.performBFS(6);
    graph.performBFSRecursive(6);
}
```

```
   }
   /*
```

- Graph Map based on the addEdge calls:

```
    0

    / /|
    / / |
    1 2 6 7 | | /
    3 4 8 9 | | 5 10
```

Breadth-First Search (Iterative) starting from vertex 0: 0 1 2 6 7 3 4 8 9 5 10 Breadth-First Search (Recursive) starting from vertex 0: 0 1 2 6 7 3 4 8 9 5 10 Breadth-First Search (Iterative) starting from vertex 6: 6 0 1 2 7 3 4 8 9 5 10 Breadth-First Search (Recursive) starting from vertex 6: 6 0 1 2 7 3 4 8 9 5 10

```
   */
```

0.6.3 Understanding Depth-First Search (DFS)

What is DFS?

Imagine exploring the same maze as before, but this time, instead of checking all paths at the current level, you pick one path and go as deep as possible until you hit a dead end. Then, you backtrack to the last branching point and try another path from there. You prioritize going deep down a single branch before exploring other branches.

In technical terms, DFS is an algorithm for traversing or searching tree or graph data structures. It starts at a selected node (the source) and explores as far as possible along each branch before backtracking. It uses a stack (Last-In, First-Out) data structure or recursion to keep track of the next node to visit.

Why Use DFS?

DFS is particularly useful for several key tasks:

- **Path Finding:** DFS can be used to find if a path exists between two nodes. It explores each path until it finds the target or exhausts all possibilities.
- **Detecting Cycles:** DFS can detect cycles in a graph. If you encounter a visited node that is still in the current recursion stack, it indicates a cycle.
- **Topological Sorting:** For directed acyclic graphs (DAGs), DFS can be used to perform a topological sort, which orders the nodes such that for every directed edge from node A to node B, node A comes before node B in the ordering.
- **Finding Connected Components:** Similar to BFS, DFS can be used to find all reachable nodes from a starting node and identify connected components in a graph.
- **Solving Puzzles:** Many puzzles that involve exploring possibilities, like Sudoku solvers or finding a path through a maze, can be effectively solved using DFS.

Real-Life Examples

DFS is used in various applications:

- **Backtracking Algorithms:** Many algorithms that involve trying out different possibilities and undoing choices if they don't lead to a solution use DFS principles.
- **Compiler Design:** DFS can be used in compilers for tasks like syntax analysis and semantic analysis.
- **File System Traversal:** When you explore folders and subfolders on your computer, many operating systems use a form of DFS.
- **Network Analysis:** Identifying dependencies or hierarchical structures in a network.

Code Example

```
/*
DEPTH FIRST
Developer: James Goudy (Modified)
 */
package depthfirst_rev2504;

import java.util.LinkedList;
import java.util.List;
import java.util.ArrayList;
import java.util.Stack;

// Class representing a graph using adjacency lists
class DFS {

    // Total number of vertices in the graph
    private int numVertices;
```

```
// Adjacency list to store edges
private List<LinkedList<Integer>> adjacencyList;

// Constructor to initialize the graph
public DFS(int vertices) {
    this.numVertices = vertices;
    this.adjacencyList = new ArrayList<>(vertices);

    // Initialize each adjacency list for every vertex
    for (int i = 0; i < vertices; i++) {
        this.adjacencyList.add(new LinkedList<>());
    }
}

// Method to add an edge between two vertices
public void addEdge(int source, int destination) {

    this.adjacencyList.get(source).add(destination);

    // For an undirected graph
    this.adjacencyList.get(destination).add(source);
}

// Method to perform Depth -First Search starting from a given vertex (Iterative)
public void performDFSIterative(int startVertex) {
    boolean[] visited = new boolean[numVertices];
    Stack<Integer> stack = new Stack<>();

    visited[startVertex] = true;
    stack.push(startVertex);

    System.out.println("Depth -First Search (Iterative) starting from vertex "
            + startVertex + ":");

    while (!stack.isEmpty()) {
        int currentVertex = stack.pop();
        System.out.print(currentVertex + " ");

        for (int neighbor : this.adjacencyList.get(currentVertex)) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                stack.push(neighbor);
            }
        }
    }
    System.out.println();
}

// Method to perform Depth -First Search starting from a given vertex (Recursive)
public void performDFSRecursive(int startVertex) {
    boolean[] visited = new boolean[numVertices];
    System.out.println("Depth -First Search (Recursive) starting from vertex "
            + startVertex + ":");
```

```
        performDFSRecursiveUtil(startVertex, visited);
        System.out.println();
    }

    private void performDFSRecursiveUtil(int vertex, boolean[] visited) {
        visited[vertex] = true;
        System.out.print(vertex + " ");

        for (int neighbor : this.adjacencyList.get(vertex)) {
            if (!visited[neighbor]) {
                performDFSRecursiveUtil(neighbor, visited);
            }
        }
    }
}

public class DepthFirst_Rev2504 {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        DFS graph = new DFS(11);

        graph.addEdge(0, 1);
        graph.addEdge(0, 2);
        graph.addEdge(1, 3);
        graph.addEdge(2, 4);
        graph.addEdge(4, 5);

        graph.addEdge(0, 6);
        graph.addEdge(0, 7);
        graph.addEdge(7, 8);
        graph.addEdge(7, 9);

        graph.addEdge(9, 10);

        graph.performDFSIterative(0);
        graph.performDFSRecursive(0);

        graph.performDFSIterative(6);
        graph.performDFSRecursive(6);
    }

}

/*
 * Graph Map based on the addEdge calls:
 *
           0
       /  /| \
      /  / |  \
     1  2  6   7
```

```
    |  |     / \
    3  4    8   9
       |        |
       5        10

Depth -First Search (Iterative) starting from vertex 0:
0 7 9 10 8 6 2 4 5 1 3
Depth -First Search (Recursive) starting from vertex 0:
0 1 3 2 4 5 7 8 9 10 6
Depth -First Search (Iterative) starting from vertex 6:
6 0 7 9 10 8 2 4 5 1 3
Depth -First Search (Recursive) starting from vertex 6:
6 0 1 3 2 4 5 7 8 9 10
*/
```

Code Analysis

DFS Class: This class represents the graph itself, similar to the BFS class.

- **`numVertices`:** Stores the total number of vertices in the graph.
- **`adjacencyList`:** The adjacency list representation of the graph.
- **`DFS(int vertices)` (Constructor):** Initializes the graph with a specified number of vertices and creates empty adjacency lists.
- **`addEdge(int source, int destination)`:** Adds an undirected edge between two vertices.
- `performDFSIterative(int startVertex)` Method:
  This method implements the iterative DFS algorithm.
  - **`visited`:** A boolean array to keep track of visited vertices.
  - **`stack`:** A `Stack` (LIFO - Last-In, First-Out) is used to store vertices to be visited. DFS relies on the LIFO nature of a stack to explore deeper first.
  - **Initialization:** The `startVertex` is marked as visited and pushed onto the stack.
  - **Traversal Loop (`while (!stack.isEmpty())`):** The loop continues as long as there are nodes in the stack to process.
  - **`currentVertex = stack.pop()`:** Removes the top vertex from the stack. This is the vertex currently being visited.
  - **`System.out.print(currentVertex + " ");`:** Prints the visited vertex.
  - **Neighbor Exploration (`for (int neighbor : ...)`):** It iterates through all the neighbors of the `currentVertex`.
  - **`if (!visited[neighbor])`:** Checks if the neighbor has already been visited.
  - **Pushing to Stack:** If the neighbor hasn't been visited, it's marked as visited and pushed onto the stack. This ensures that the algorithm explores the neighbors of the most recently added node first, going deeper into the graph.
- `performDFSRecursive(int startVertex)` Method:
  This method initiates the recursive DFS.
  - It creates a `visited` array and calls the helper method `performDFSRecursiveUtil`.
- `performDFSRecursiveUtil(int vertex, boolean[] visited)` Method:
  This is the recursive helper function.
  - It marks the current `vertex` as visited and prints it.
  - It then iterates through the neighbors of the current `vertex`.
  - For each unvisited neighbor, it recursively calls `performDFSRecursiveUtil` on that neighbor, effectively exploring deeper down that path.

DepthFirst_Rev2504 Class (Main Program):

- **`main(String[] args)`:** The entry point of the program.
- Creates a `DFS` graph object with 11 vertices.
- Calls `addEdge` multiple times to define the same graph connections as in the BFS example.

- Calls `performDFSIterative(0)` and `performDFSRecursive(0)` to run both iterative and recursive DFS starting from vertex 0.
- Calls `performDFSIterative(6)` and `performDFSRecursive(6)` to run both iterative and recursive DFS starting from vertex 6.

Output

The output shows the order in which the nodes are visited using both the iterative and recursive approaches of DFS, starting from different vertices. Notice how the traversal goes deep along one path before backtracking and exploring other paths, which is characteristic of Depth-First Search. The iterative and recursive approaches may result in slightly different traversal orders depending on the order in which neighbors are added to the stack or processed in the adjacency list.

0.6.4 Seven Bridges of Königsberg

The **Seven Bridges of Königsberg** is a historically notable problem in mathematics that laid the foundations for graph theory and prefigured the idea of topology. Let's delve into the details:

1. **The City of Königsberg**:
   - Königsberg (now Kaliningrad, Russia) was a city set on both sides of the Pregel River.
   - It included two large islands: **Kneiphof** and **Lomse**.
   - These islands were connected to each other and to the two mainland portions of the city by **seven bridges**.

2. **The Problem**:
   - The challenge was to find a walk through the city that would **cross each bridge once and only once**.
   - The rules were strict:
     – Solutions couldn't involve reaching an island or mainland bank without using a bridge.
     – Accessing any bridge without crossing to its other end was unacceptable.

3. **Euler's Insight**:
   - **Leonhard Euler**, a brilliant mathematician, analyzed the problem.
   - He realized that the choice of route inside each landmass (island or mainland) was irrelevant.
   - The crucial aspect was the **sequence of bridges crossed**.
   - Euler reformulated the problem in abstract terms, focusing only on the list of landmasses and the bridges connecting them.

4. **Graph Theory Approach**:
   - Euler replaced each landmass with an abstract "vertex" or node.
   - Each bridge became an abstract connection or "edge" between vertices.
   - The resulting mathematical structure is a **graph**.
   - The shape of the graph could be distorted without changing its essence—only the number of edges between nodes mattered.

5. **Euler's Solution**:
   - Euler observed that whenever one enters a vertex by a bridge, one must leave it by a bridge (except at the endpoints of the walk).
   - If every bridge had been traversed exactly once, the number of bridges touching each landmass must be **even**.
   - Euler proved that the problem had **no solution** because the number of bridges touching each landmass was **odd**.

6. **Legacy**:
   - Euler's negative resolution of the Seven Bridges problem laid the groundwork for graph theory.
   - It also prefigured the concept of **topology**, which studies properties preserved under continuous deformations.

0.6.5 Depth-First Search (DFS)

Depth-First Search (DFS) is an algorithm for exploring tree or graph data structures. Imagine you're navigating a maze, and DFS is like taking a very narrow path, going as far as you can down one tunnel before checking any side passages. Here's how it works on graphs:

1. **Start at a Node:** You begin at any node (like the entrance to the maze).
2. **Explore Deepest Path:** Visit all the unvisited neighbors of that node (like going down the main tunnel).
3. **Mark and Backtrack:** Mark the current node as visited to avoid revisiting it. If you reach a dead end (a node with no unvisited neighbors), backtrack to the most recent un-explored path (like going back to a fork in the maze and taking a different tunnel).
4. **Repeat:** Keep repeating steps 2 and 3 until you've explored all possible paths from the starting node.

Data Structures Used:

- **Stack:** DFS typically uses a stack (LIFO - Last In, First Out) to keep track of the path being explored. Nodes are added to the stack as you move deeper into the graph, and removed as you backtrack.
- **Visited List:** An additional data structure, like a visited list or array, is used to keep track of the nodes already explored and avoid revisiting them.

Applications of DFS:

- Finding paths in mazes or games.
- Topological sorting (ordering a directed acyclic graph where each node has dependencies only on nodes preceding it in the order).
- Cycle detection in graphs.
- Finding connected components in graphs (groups of nodes connected by edges).

Advantages and Disadvantages:

- **Advantages:** DFS can be efficient for certain graph structures and finding specific paths quickly.
- **Disadvantages:** DFS might not be ideal for finding the shortest path between two nodes, and it can get stuck in infinite loops if there are cycles in the graph (without proper handling).

Here are some resources for further learning:

- A visual explanation of DFS on graphs: YouTube video on Depth First Search: YouTube
- A step-by-step explanation of the DFS algorithm: https://www.programiz.com/dsa/graph-dfs

Lecture Code

```
import java.util.ArrayList;
import java.util.List;

class Node {
  int data;
  boolean visited;

  public Node(int data) {
    this.data = data;
  }
}

class Graph {
  List<List<Node>> adjList;
```

```
  public Graph(int vertices) {
    adjList = new ArrayList<>(vertices);
    for (int i = 0; i < vertices; i++) {
      adjList.add(new ArrayList<>());
    }
  }

  public void addEdge(int src, int dest) {
    adjList.get(src).add(new Node(dest));
  }

  public void DFS(int startNode) {
    boolean visited[] = new boolean[adjList.size()];
    DFSUtil(startNode, visited);
  }

  private void DFSUtil(int node, boolean[] visited) {
    visited[node] = true;
    System.out.print(node + " ");

    // Visit all unvisited neighbors
    for (Node neighbor : adjList.get(node)) {
      if (!visited[neighbor.data]) {
        DFSUtil(neighbor.data, visited);
      }
    }
  }
}

public class Main {
  public static void main(String[] args) {
    int vertices = 6;
    Graph graph = new Graph(vertices);

    graph.addEdge(0, 1);
    graph.addEdge(0, 2);
    graph.addEdge(1, 2);
    graph.addEdge(1, 3);
    graph.addEdge(2, 3);
    graph.addEdge(4, 5);

    System.out.println("Depth First Traversal (starting from vertex 0)");

    for(int i= 0; i < vertices; i++)
    {
        System.out.print("V = " + i + " : ");
        graph.DFS(i);
        System.out.println();
    }
  }
}
/*
```

```
Depth First Traversal (starting from vertex 0)
V = 0 : 0 1 2 3
V = 1 : 1 2 3
V = 2 : 2 3
V = 3 : 3
V = 4 : 4 5
V = 5 : 5
*/
```

0.6.6 Breadth First

Breadth-First Search (BFS) is an algorithm used to traverse or search tree or graph data structures. It visits nodes level by level, exploring all the neighbor nodes at the current depth before moving down to the next level. Imagine exploring a maze - BFS would check every path on one floor before moving down to the floor below.

Here's a breakdown of BFS for graphs:

Concept:

1. Start at a chosen node (usually called the root node).
2. Visit all the unvisited neighbor nodes of the current node. Add them to a queue data structure (think of a line where you add people at the back).
3. Mark the current node as visited to avoid revisiting.
4. Dequeue a node from the queue (remove the first person in line). This becomes the current node.
5. Repeat steps 2-4 until the queue is empty.

Key Points:

- BFS uses a Queue to store neighbor nodes for exploration. A Queue functions like "first in, first out" - similar to a waiting line.
- It ensures we explore all nodes at a level before moving deeper into the graph.
- BFS is useful for finding shortest paths (assuming all edges have the same weight) in unweighted graphs.

Example:

Imagine a social network graph where nodes are people and edges are friendships. You can use BFS to find all your friends' friends (people two connections away). You'd start with yourself, then explore all your friends (level 1), then all their friends you haven't met yet (level 2).

Further Exploration:

- You can find visualizations of BFS on YouTube ([YouTube video on BFS]).
- For code examples, explore resources like GeeksforGeeks ([Breadth First Search for Graphs])

Lecture Code

```
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;

public class BreadthFirstSearch {

  static class Graph {
    private int V; // number of vertices
    private List<List<Integer>> adj; // Adjacency List representation

    Graph(int V) {
      this.V = V;
      adj = new ArrayList<>(V);
      for (int i = 0; i < V; i++) {
        adj.add(new LinkedList<>());
      }
    }

    public void addEdge(int v, int w) {
```

```
      adj.get(v).add(w);
    }

    public void BFS(int s) {
      boolean visited[] = new boolean[V];

      LinkedList<Integer> queue = new LinkedList<Integer>();

      visited[s] = true;
      queue.add(s);

      while (queue.size() != 0) {
        s = queue.poll();
        System.out.print(s + " ");

        for (int neighbor : adj.get(s)) {
          if (!visited[neighbor]) {
            visited[neighbor] = true;
            queue.add(neighbor);
          }
        }
      }
    }
  }

  public static void main(String args[]) {

    int vertices = 4;

    BreadthFirstSearch.Graph g = new BreadthFirstSearch.Graph (vertices);


    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 2);
    g.addEdge(2, 0);
    g.addEdge(2, 3);
    g.addEdge(3, 3);

    System.out.println("Following is Breadth First Traversal (starting from vertex 0)");

    for(int i= 0; i < vertices; i++)
    {
        System.out.print("V = " + i + " : ");
        g.BFS(i);
        System.out.println();
    }
  }
}
/*
Following is Breadth First Traversal (starting from vertex 0)
V = 0 : 0 1 2 3
V = 1 : 1 2 0 3
```

```
V = 2 : 2 0 3 1
V = 3 : 3
*/
```

This code defines a `Graph` class with an adjacency list to represent the graph. The `BFS` method takes a starting vertex `s` and performs the following steps:

1. Creates a boolean array `visited` to keep track of visited nodes.
2. Initializes a queue using `LinkedList` to store nodes for exploration.
3. Marks the starting node `s` as visited and adds it to the queue.
4. Loops as long as the queue is not empty:
    - Dequeues a node from the queue (the current node).
    - Prints the current node.
    - Iterates through the neighbors of the current node.
        - If a neighbor is not visited, mark it visited and add it to the queue.

This ensures a level-by-level exploration of the graph.

0.6.7 DFS - Stored Paths

This Java code defines a `DFS` class that implements Depth-First Search for an undirected graph represented using an adjacency list. It provides standard DFS traversal methods (both recursive and iterative) that print the visited node order, as well as specialized methods (also recursive and iterative) designed to find and store all possible paths starting from a given source vertex discovered during the DFS exploration. The `Main` class demonstrates how to create a graph, add edges, and utilize both the traversal and path-finding functionalities provided by the `DFS` class.

Java

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;
import java.util.Stack;

class DFS {

    // Num Vertices
    private final int numVertices;

    // Adjacency list to store edges
    public final List<LinkedList<Integer>> adjencyList;

    // List to store all paths found by the last DFS run
    private List<List<Integer>> allPaths;

    // Constructor
    public DFS(int vertices) {
        this.numVertices = vertices;
        this.adjencyList = new ArrayList<>(vertices);
        this.allPaths = new ArrayList<>(); // Initialize the path list

        for (int i = 0; i < vertices; i++) {
            this.adjencyList.add(new LinkedList<>());
        }
    }

    // Add edges (undirected graph)
    public void addEdge(int source, int destination) {
        // Ensure vertices are within bounds
        if (source < 0 || source >= numVertices || destination < 0 || destination >= numVertic
            System.err.println("Warning: Vertex index out of bounds. Edge not added.");
            return;
        }
        adjencyList.get(source).add(destination);
        adjencyList.get(destination).add(source); // For undirected graph
    }

    // - - - Recursive DFS Path Finding - - -

    /**
```

```
     * Finds all paths starting from startVertex using recursive DFS.
     * Stores the paths in the allPaths list.
     *
     * @param startVertex The vertex to start the search from.
     */
    public void findAllPathsRecursive(int startVertex) {
        if (startVertex < 0 || startVertex >= numVertices) {
             System.err.println("Error: Start vertex out of bounds.");
             return;
        }
        boolean[] visited = new boolean[numVertices];
        this.allPaths = new ArrayList<>(); // Reset paths for new search
        List<Integer> currentPath = new ArrayList<>();

        System.out.println("\nFinding all paths (Recursive DFS) starting from vertex " + start
        findAllPathsRecursiveUtil(startVertex, visited, currentPath);
    }

    /**
     * Recursive helper function for DFS path finding.
     *
     * @param vertex      The current vertex being visited.
     * @param visited     Array to keep track of visited nodes in the current recursion path.
     * @param currentPath The path taken to reach the current vertex.
     */
    private void findAllPathsRecursiveUtil(int vertex, boolean[] visited, List<Integer> curren
        // Mark the current node as visited and add it to the path
        visited[vertex] = true;
        currentPath.add(vertex);

        // Store a copy of the current path
        // This captures the path to the *current* vertex
        allPaths.add(new ArrayList<>(currentPath));
        // System.out.println("Path found: " + currentPath); // Optional: print paths as they

        // Recur for all adjacent vertices
        for (int neighbor : adjencyList.get(vertex)) {
            if (!visited[neighbor]) {
                findAllPathsRecursiveUtil(neighbor, visited, currentPath);
            }
        }

        // Backtrack: remove current vertex from path and mark as unvisited
        // for other potential paths.
        currentPath.remove(currentPath.size() - 1);
        visited[vertex] = false;
    }


    // - - - Iterative DFS Path Finding - - -

     /**
      * Finds paths using iterative DFS and stores them.
```

```
     * This version uses a stack storing pairs of (node, path_to_node).
     * Note: The order of path discovery might differ from the recursive version.
     *
     * @param startVertex The vertex to start the search from.
     */
    public void findAllPathsIterative(int startVertex) {
         if (startVertex < 0 || startVertex >= numVertices) {
             System.err.println("Error: Start vertex out of bounds.");
             return;
         }
         this.allPaths = new ArrayList<>(); // Reset paths
         System.out.println("\nFinding all paths (Iterative DFS) starting from vertex " + star

         // Stack stores pairs: the node and the path to reach it
         Stack<Pair<Integer, List<Integer>>> stack = new Stack<>();

         // Initial path contains only the start vertex
         List<Integer> initialPath = new ArrayList<>();
         initialPath.add(startVertex);
         stack.push(new Pair<>(startVertex, initialPath));

         // Keep track of visited nodes *globally* across different path explorations
         // This prevents infinite loops in cyclic graphs and redundant processing.
         boolean[] globallyVisited = new boolean[numVertices];


         while (!stack.isEmpty()) {
             Pair<Integer, List<Integer>> currentPair = stack.pop();
             int currentVertex = currentPair.getKey();
             List<Integer> currentPath = currentPair.getValue();

             // If we haven't processed this node *through this specific path expansion*
             // Mark it visited *globally* only after processing its path.
             // This allows finding different paths to the same node, but standard
             // iterative DFS usually uses a simpler visited check.
             // For simplicity matching the recursive intent, let's only proceed
             // if the node hasn't been the *end* of a stored path yet.
             // A better approach might be needed for complex path requirements.

             // Store the path found to this vertex
             allPaths.add(new ArrayList<>(currentPath)); // Store a copy
             // System.out.println("Path found: " + currentPath); // Optional: print


             if (!globallyVisited[currentVertex]) {
                 globallyVisited[currentVertex] = true; // Mark globally visited

                  // Explore neighbors
                 // Iterate in reverse to mimic recursive DFS neighbor order more closely (opt
                 List<Integer> neighbors = adjencyList.get(currentVertex);
                 // Collections.reverse(neighbors); // Uncomment to push neighbors in reverse

                 for (int neighbor : neighbors) {
```

```
                // Check if the neighbor is already in the *current* path to avoid cycles
                boolean visitedInCurrentPath = false;
                for(int nodeInPath : currentPath) {
                    if (nodeInPath == neighbor) {
                        visitedInCurrentPath = true;
                        break;
                    }
                }

                // If the neighbor hasn't been visited *in this specific path*
                if (!visitedInCurrentPath) {
                    List<Integer> newPath = new ArrayList<>(currentPath);
                    newPath.add(neighbor);
                    stack.push(new Pair<>(neighbor, newPath));
                }
            }
            // Collections.reverse(neighbors); // Reverse back if reversed earlier
        }
    }
}


// - - - Original Traversal Methods (for comparison/demonstration) - - -

/**
 * Performs the original iterative DFS traversal, printing nodes as visited.
 * Does not store paths.
 * @param startVertex The vertex to start from.
 */
public void performDFSTraversalIterative(int startVertex) {
    if (startVertex < 0 || startVertex >= numVertices) {
        System.err.println("Error: Start vertex out of bounds.");
        return;
    }
    boolean[] visited = new boolean[numVertices];
    Stack<Integer> stack = new Stack<>(); // Use Stack's push/pop for LIFO (DFS)

    visited[startVertex] = true;
    stack.push(startVertex); // Push onto stack

    System.out.println("\nDFS Traversal Order (Iterative): starting from " + startVertex +

    while (!stack.isEmpty()) {
        int currentVertex = stack.pop(); // Pop from stack (LIFO)
        System.out.print(currentVertex + " ");

        // Get neighbors and push unvisited ones onto the stack
        // Process neighbors in natural order (or reverse to mimic recursion)
        List<Integer> neighbors = adjencyList.get(currentVertex);
         // Collections.reverse(neighbors); // To more closely match typical recursive ord

        for (int neighbor : neighbors) {
            if (!visited[neighbor]) {
```

```
                visited[neighbor] = true;
                stack.push(neighbor);
            }
        }
         // Collections.reverse(neighbors); // Reverse back if needed
    }
    System.out.println();
}

 /**
 * Performs the original recursive DFS traversal, printing nodes as visited.
 * Does not store paths.
 * @param startVertex The vertex to start from.
 */
public void performDFSTraversalRecursive(int startVertex) {
     if (startVertex < 0 || startVertex >= numVertices) {
         System.err.println("Error: Start vertex out of bounds.");
         return;
     }
    boolean[] visited = new boolean[numVertices];
    System.out.println("\nDFS Traversal Order (Recursive) starting from vertex " + startVe
    performDFSTraversalRecursiveUtil(startVertex, visited);
    System.out.println();
}

private void performDFSTraversalRecursiveUtil(int vertex, boolean[] visited) {
    visited[vertex] = true;
    System.out.print(vertex + " ");

    for (int neighbor : adjencyList.get(vertex)) {
        if (!visited[neighbor]) {
            performDFSTraversalRecursiveUtil(neighbor, visited);
        }
    }
}

// - - - Utility - - -

/**
 * Returns the list of paths found by the last pathfinding call.
 * @return A list where each element is a list of integers representing a path.
 */
public List<List<Integer>> getAllPaths() {
    return allPaths;
}

 /**
  * Prints all stored paths.
  */
 public void printAllPaths() {
     if (allPaths == null || allPaths.isEmpty()) {
         System.out.println("No paths have been generated or stored yet.");
         return;
```

```
        }
        System.out.println("\n - - - Stored Paths - - -");
        int pathNum = 1;
        for (List<Integer> path : allPaths) {
            System.out.println("Path " + (pathNum++) + ": " + path);
        }
        System.out.println(" - - - - - - - - - - - - - - - - - - - - -");
    }

    // Simple generic Pair class (or use javafx.util.Pair if available/allowed)
    private static class Pair<K, V> {
        private final K key;
        private final V value;

        public Pair(K key, V value) {
            this.key = key;
            this.value = value;
        }

        public K getKey() { return key; }
        public V getValue() { return value; }
    }
}

class Main {

    public static void main(String[] args) {

        DFS graph = new DFS(12);

        // Build the graph
        graph.addEdge(0, 1);
        graph.addEdge(0, 2);
        graph.addEdge(1, 3);
        graph.addEdge(2, 4);
        graph.addEdge(4, 5);
        graph.addEdge(0, 6); // 0 -> 6
        graph.addEdge(0, 7); // 0 -> 7
        graph.addEdge(7, 8); // 7 -> 8
        graph.addEdge(7, 9); // 7 -> 9
        graph.addEdge(9, 10); // 9 -> 10
        graph.addEdge(5, 11); // 5 -> 11

        // Print Adjacency List (for verification)
        System.out.println(" - - - Adjacency List - - -");
        for (int i = 0; i < graph.adjencyList.size(); i++) {
            System.out.println("Vertex " + i + ": " + graph.adjencyList.get(i));
        }
         System.out.println(" - - - - - - - - - - - - - - - - - - - - - -");

        // - - - Demonstrate Original Traversals - - -
        graph.performDFSTraversalIterative(0);
        graph.performDFSTraversalRecursive(0);
```

```
        // - - - Find and Store Paths (Recursive) - - -
        graph.findAllPathsRecursive(0);
        // Print the paths found by the recursive method
        graph.printAllPaths();

        // - - - Find and Store Paths (Iterative) - - -
        graph.findAllPathsIterative(0);
         // Print the paths found by the iterative method
         // Note: The order and exact paths might differ slightly depending on implementation
         graph.printAllPaths();

         // Example: Get paths and process them
         System.out.println("\nAccessing paths manually after iterative search:");
         List<List<Integer>> storedPaths = graph.getAllPaths();
         if (storedPaths != null && !storedPaths.isEmpty()) {
             System.out.println("Total paths found: " + storedPaths.size());
             System.out.println("First path found: " + storedPaths.get(0));
             System.out.println("Last path found: " + storedPaths.get(storedPaths.size() - 1))
         }
    }
}
```

This code defines a `DFS` class that represents a graph and provides methods for performing Depth-First Search (DFS) operations on it. It specifically includes implementations for:

1. **Standard DFS Traversal (Recursive & Iterative):** These methods visit nodes in DFS order and typically print them, primarily demonstrating the traversal algorithm itself.
2. **Finding All Paths via DFS (Recursive & Iterative):** These are more complex methods designed to find and store *all* possible paths starting from a given vertex that can be discovered through a DFS exploration.

Here's a breakdown of the components:
`DFS` Class:

1. **Fields:**
   - `numVertices`: An integer storing the number of vertices in the graph.
   - `adjencyList`: A `List<LinkedList<Integer»`. This is the core graph representation using an **adjacency list**. The outer list has an index for each vertex (0 to `numVertices - 1`). The `LinkedList` at each index `i` stores the vertices directly connected (adjacent) to vertex `i`.
   - `allPaths`: A `List<List<Integer»`. This list is used to store the results of the path-finding operations (`findAllPathsRecursive` or `findAllPathsIterative`). Each element within `allPaths` is itself a `List<Integer>` representing a single path found.
2. **Constructor (`DFS(int vertices)`):**
   - Initializes the graph with a specified number of vertices.
   - Creates the `adjencyList` structure, adding an empty `LinkedList` for each vertex.
   - Initializes the `allPaths` list.
3. **`addEdge(int source, int destination)`:**
   - Adds an edge between the `source` vertex and the `destination` vertex.
   - Includes basic error checking to ensure vertex indices are valid.

- Adds `destination` to `source`'s list and `source` to `destination`'s list. This makes the graph **undirected** (an edge exists in both directions).

4. **Recursive Path Finding (`findAllPathsRecursive` and `findAllPathsRecursiveUtil`):**
   - `findAllPathsRecursive(startVertex)`: The public method to initiate the search. It sets up a `visited` array (local to the current path exploration) and resets the `allPaths` list before calling the helper function.
   - `findAllPathsRecursiveUtil(vertex, visited, currentPath)` :
     - This is the core recursive logic.
     - It marks the current `vertex` as visited *for the current path exploration.*
     - Adds the `vertex` to the `currentPath`.
     - **Crucially:** It adds a *copy* of the `currentPath` (up to the current `vertex`) to the `allPaths` list. This means every node encountered during the DFS becomes the end of a stored path.
     - It iterates through the neighbors of the current `vertex`. If a neighbor hasn't been visited *in the current recursive call stack*, it recursively calls itself for that neighbor.
     - **Backtracking:** After exploring all paths stemming from the current `vertex`, it removes the `vertex` from `currentPath` and marks it as unvisited (`visited[vertex] = false`). This is essential to allow the algorithm to explore *other* paths that might reach this `vertex` through a different sequence.

5. **Iterative Path Finding (`findAllPathsIterative`):**
   - This method attempts to find all paths using an iterative approach with a `Stack`.
   - It uses a

     `Stack`

     where each element is a

     `Pair`

     containing:
     - The current vertex (`Integer`).
     - The path (`List<Integer>`) taken to reach that vertex.
   - It also uses a

     `globallyVisited`

     array. The logic here is slightly different from the recursive version's

     `visited`

     array:
     - It adds the path to `allPaths` *when* a `Pair` is popped from the stack.
     - It checks `globallyVisited` *after* storing the path. If the node hasn't been globally visited yet, it marks it and explores its neighbors.
     - When exploring neighbors, it checks if the neighbor is already present in the *current specific path* (`visitedInCurrentPath` loop) to prevent cycles *within that single path.*
     - If a neighbor is valid (not creating a cycle in the current path), a new path is created by extending the current one, and a new `Pair` is pushed onto the stack.
   - **Note:** The use of `globallyVisited` prevents re-exploring *from* a node once *any* path ending there has been processed. This helps avoid infinite loops in cyclic graphs but might prune some paths compared to a pure recursive backtracking approach if the goal was *all simple paths.* The order of paths found might also differ from the recursive version due to stack LIFO order and neighbor processing order.

6. **Standard Traversal Methods (`performDFSTraversalIterative`, `performDFSTraversalRecursive`, `performDFSTraversalRecursiveUtil`):**
    - These implement the more "standard" DFS algorithm where the goal is just to visit each reachable node once.
    - They use a `visited` array to keep track of nodes already visited during the *entire* traversal.
    - They print the vertex when it's visited (recursive) or popped (iterative).
    - They do **not** store paths.
7. **Utility Methods:**
    - `getAllPaths()`: Returns the `allPaths` list (containing results from the last path-finding call).
    - `printAllPaths()`: Prints all the paths currently stored in the `allPaths` list in a readable format.
    - `Pair<K, V>`: A simple private static inner class to hold pairs of objects (used in the iterative path finding).

`Main` Class:

- Creates an instance of the `DFS` graph with 12 vertices.
- Adds edges to define the graph structure.
- Prints the adjacency list to show the graph's structure.
- Demonstrates the standard iterative and recursive DFS *traversals*, printing the order nodes are visited.
- Calls `findAllPathsRecursive(0)` to find paths starting from vertex 0 using recursion and prints the results.
- Calls `findAllPathsIterative(0)` to find paths starting from vertex 0 using iteration and prints those results.
- Shows how to retrieve the list of paths using `getAllPaths()` after a search.

In essence, this code provides a robust `DFS` class for undirected graphs, offering both standard traversal and more complex path-finding capabilities, implemented recursively and iteratively.

0.6.8 DSF - Maze Solving

This program defines constants for different characters in the maze and implements two methods:

- **solveMaze**: Finds the starting point and calls the recursive solveMazeDFS method.
- **solveMazeDFS**: Uses depth-first search to explore the maze. It marks visited cells, checks all four directions (up, down, left, right) for a solution, and backtracks if necessary.

The main method demonstrates how to use the program with a sample maze.

Lecture Code

```
/*
Programmer: James Goudy with JGEM
 */
package mazesolver;

import java.util.Scanner;

public class MazeSolver {

    public static final char START = 'S';
    public static final char END = 'E';
    public static final char WALL = 'W';
    public static final char OPEN = '.';
    public static final char VISITED = 'V';
    public static final char ROUTE = '*'; // New character to mark the route

    public static boolean solveMaze(char[][] maze) {
        int startRow = -1;
        int startCol = -1;

        // Find the starting point
        for (int row = 0; row < maze.length; row++) {
            for (int col = 0; col < maze[row].length; col++) {
                if (maze[row][col] == START) {
                    startRow = row;
                    startCol = col;
                    break;
                }
            }
        }

        if (startRow == -1 || startCol == -1) {
            System.out.println("Error: Starting point not found in maze");
            return false;
        }

        return solveMazeDFS(maze, startRow, startCol);
    }

    private static boolean solveMazeDFS(char[][] maze, int row, int col) {
        // Check if we reached the end or hit a wall/visited cell
        if (maze[row][col] == END) {
            return true;
        } else if (maze[row][col] == WALL || maze[row][col] == VISITED) {
```

```
            return false;
        }

        // Mark current cell as visited
        maze[row][col] = VISITED;

        // Try all four directions (up, down, left, right)
        if (row > 0 && solveMazeDFS(maze, row - 1, col)) {
            maze[row][col] = ROUTE; // Mark as part of the route on successful return
            return true; // Found solution going up
        }
        if (row < maze.length - 1 && solveMazeDFS(maze, row + 1, col)) {
            maze[row][col] = ROUTE;
            return true; // Found solution going down
        }
        if (col > 0 && solveMazeDFS(maze, row, col - 1)) {
            maze[row][col] = ROUTE;
            return true; // Found solution going left
        }
        if (col < maze[row].length - 1 && solveMazeDFS(maze, row, col + 1)) {
            maze[row][col] = ROUTE;
            return true; // Found solution going right
        }

        // Backtrack if no solution found in any direction
        maze[row][col] = OPEN; // Unmark cell as visited (backtracking)
        return false;
    }

    public static void printMaze(char[][] maze) {
        for (char[] row : maze) {
            for (char c : row) {
                System.out.print(c + " ");
            }
            System.out.println();
        }
    }

    public static void main(String[] args) {

        char[][] maze = new char[1][1];

        char[][] maze1 = {
            {'W', 'W', 'W', '.'},
            {'.', '.', 'S', '.'},
            {'W', 'W', 'W', '.'},
            {'W', '.', '.', 'E'}
        };

        char[][] maze2 = {
            {'W', 'W', 'W', 'W', 'W', 'W', 'W', 'W', 'W', 'W'},
            {'.', '.', '.', '.', 'W', '.', '.', '.', '.', '.'},
            {'W', 'W', '.', 'W', 'W', '.', 'W', 'W', 'W', 'W'},
```

```
    {'.', '.', 'S', '.', '.', '.', '.', '.', '.', '.'},
    {'W', 'W', '.', 'W', 'W', 'W', 'W', '.', 'W', 'W'},
    {'.', '.', '.', '.', '.', '.', '.', '.', '.', '.'},
    {'W', 'W', 'W', 'W', '.', 'W', 'W', 'W', 'W', 'W'},
    {'.', '.', '.', '.', '.', '.', '.', '.', '.', '.'},
    {'W', 'W', '.', 'W', '.', 'W', '.', 'W', 'W', 'W'},
    {'.', '.', '.', 'W', '.', '.', '.', '.', '.', '.'},
    {'.', 'W', 'W', '.', 'W', '.', '.', '.', '.', '.'},
    {'.', '.', '.', '.', '.', 'W', '.', '.', '.', '.'},
    {'.', '.', '.', '.', '.', 'W', 'E', 'W', '.', '.'}
};

char[][] maze3 = {
    {'W', 'W', 'W', 'W', 'W', 'W', 'W', 'W', 'W', 'W'},
    {'.', '.', '.', '.', 'W', '.', '.', '.', '.', '.'},
    {'W', 'W', '.', 'W', 'W', '.', 'W', 'W', 'W', 'W'},
    {'.', '.', 'S', '.', '.', '.', '.', 'W', '.', '.'},
    {'W', 'W', '.', 'W', 'W', 'W', 'W', 'W', 'W', 'W'},
    {'.', '.', '.', '.', '.', '.', '.', 'W', '.', 'E'},
    {'W', 'W', 'W', 'W', '.', 'W', 'W', 'W', '.', 'W'},
    {'.', '.', '.', '.', '.', '.', '.', 'W', '.', 'W'},
    {'W', 'W', '.', 'W', '.', 'W', '.', 'W', '.', 'W'},
    {'.', '.', '.', 'W', '.', '.', '.', 'W', '.', 'W'},
    {'.', 'W', 'W', '.', 'W', '.', '.', '.', '.', 'W'},
    {'.', '.', '.', '.', '.', 'W', '.', '.', '.', 'W'},
    {'.', '.', '.', '.', '.', 'W', 'W', 'W', 'W', 'W'}
};

char[][] maze4 = {
    {'W', 'W', 'W', 'W'},
    {'.', '.', 'S', '.'},
    {'W', 'W', 'W', '.'},
    {'W', 'W', '.', '.'},
    {'W', 'W', '.', '.'},
    {'W', 'W', 'W', 'W'},
    {'W', '.', '.', 'E'}
};

String choice;
Scanner myScan = new Scanner(System.in);

System.out.println("Choose maze 1,2 3 or 4");
choice = myScan.nextLine();

switch(choice){
   case "1" -> maze = maze1;
   case "2" -> maze = maze2;
   case "3" -> maze = maze3;
   case "4" -> maze = maze4;
   default ->  {System.out.println("Not a choice");}
}
```

```
        printMaze(maze);
        System.out.println("\n");

        if (solveMaze(maze)) {
            System.out.println("Maze solved!");
            printMaze(maze); // Print the maze with the solved route
        } else {
            System.out.println("No solution found for the maze");
        }
    }
}
```

Sample Output

```
Choose maze 1,2 3 or 4
3
W W W W W W W W W W
. . . . W . . . . .
W W . W W . W W W W
. . S . . . . W . .
W W . W W W W W W W
. . . . . . . W . E
W W W W . W W W . W
. . . . . . . W . W
W W . W . W . W . W
. . . W . . . W . W
. W W . W . . . . W
. . . . . W . . . W
. . . . . W W W W W


Maze solved!
W W W W W W W W W W
. . . . W . . . . .
W W . W W . W W W W
. . * . . . . W . .
W W * W W W W W W W
. . * * * . . W * E
W W W W * W W W * W
. . . . * . . W * W
W W . W * W . W * W
. . . W * * . W * W
. W W . W * * * * W
. . . . . W * * . W
. . . . . W W W W W
```

0.6.9 Dijkstra's Algorithm

Dijkstra's algorithm is a famous algorithm for finding the shortest paths between nodes in a weighted graph. Imagine you have a map like a road network, where cities are represented by nodes and roads connecting them are the edges. Each road has a weight, which could be its distance, travel time, or cost. Dijkstra's algorithm helps you find the most efficient route (shortest path) between two points on this map, considering the weights.

Here's a simplified breakdown of how it works:

1. **Start at a specific node (source):** You choose a starting point on the map (source node).
2. **Track distances:** The algorithm keeps track of the tentative shortest distance from the source node to all other nodes. Initially, these distances are set to infinity, except for the source node itself (distance 0).
3. **Explore neighbors:** The algorithm examines the neighboring nodes of the source node. For each neighbor, it calculates the distance by traveling through the source node and adds that to the source's distance to that neighbor.
4. **Update and Repeat:** If this calculated distance is shorter than the currently known distance to that neighbor, the algorithm updates the neighbor's distance. Then, it picks the unvisited node with the shortest distance (tentatively the most promising path to explore) and repeats steps 3 and 4 for that node as the new center.
5. **Shortest paths found:** The algorithm continues exploring and updating distances until all nodes have been visited. At this point, the distances from the source node to all other nodes represent the shortest paths.

Dijkstra's algorithm is widely used in various applications like GPS navigation systems, finding optimal routes in transportation networks, or planning efficient delivery paths.

Lecture Code Explanation

This code implements Dijkstra's algorithm to find the shortest paths from a single source vertex to all other vertices in a weighted graph. Here's a breakdown of the code:

1. Class Dijkstra:

- This class represents the core functionalities of Dijkstra's algorithm.
- It has properties like `totalVertexes` to store the number of vertices in the graph and `startVertex` to store the starting point for finding shortest paths.
- The constructor `Dijkstra(int totalVertexes)` initializes these properties.

2. Utility methods:

- `minimumDistance(int distance[], boolean visited[])`: This method finds the unvisited vertex with the minimum tentative distance from the source vertex. It iterates through the `distance` array and the `visited` array to find the closest unvisited vertex.
- `printSolution(int distance[], int n)`: This method prints the calculated shortest distances from the source vertex to all other vertices. It iterates through the `distance` array and prints the distance for each vertex.

3. calculate(int graph[][], int startVertex, int totalVertexes):

- This is the main method that implements Dijkstra's algorithm.
- It takes the adjacency matrix representation of the graph (`graph`), the starting vertex (`startVertex`), and the total number of vertices (`totalVertexes`) as input.
- It initializes two arrays: `distance` to store the tentative distances from the source vertex, and `visited` to keep track of visited vertices.
- It sets all distances in the `distance` array to infinity (except the source vertex which is set to 0).

- It iterates

  `totalVertexes -1`

  times (because we need to find shortest paths to all vertices):
  - In each iteration, it finds the unvisited vertex with the minimum tentative distance from the source vertex using `minimumDistance`.
  - It marks the found vertex as visited.
  - It iterates through all unvisited neighbors of the visited vertex.
    * For each neighbor, it calculates the tentative distance if going through the visited vertex is shorter than the current tentative distance.
    * If the new tentative distance is shorter, it updates the distance in the `distance` array.
- Finally, it calls `printSolution` to print the calculated shortest distances.

4. Main method:

- This method creates a sample adjacency matrix representing a weighted graph.
- It creates an instance of the `Dijkstra` class (`dgrpah`) with the number of vertices in the graph (9).
- It calls the `calculate` method of the `dgrpah` object, providing the graph, starting vertex (8), and total vertices (9).

Overall, this code demonstrates how to implement Dijkstra's algorithm to find the shortest paths in a weighted graph.

int minimumDistance(int distance[], boolean visited[]) This code snippet implements the `minimumDistance` function within the Dijkstra's algorithm class. Here's a breakdown of what it does:

1. Finding the Unvisited Vertex with Minimum Tentative Distance:

- It initializes two variables:
  - `m`: This variable stores the current minimum tentative distance found so far. It's initialized to the maximum integer value (`Integer.MAX_VALUE`) to ensure the first encountered unvisited vertex becomes the initial minimum.
  - `m_index`: This variable stores the index of the vertex with the minimum tentative distance. It's initialized to -1 to indicate no vertex found yet.
- The code iterates through all vertices in the graph using a loop (`for(int vx= 0; vx <totalVertexes; vx++)`).
- Inside the loop, it checks two conditions for each vertex (`vx`):
  - `visited[vx] == false`: This condition checks if the vertex is not yet visited. We only care about unvisited vertices because we're looking for the next closest unvisited point to explore.
  - `distance[vx] <= m`: This condition checks if the current tentative distance of the vertex (`distance[vx]`) is less than or equal to the current minimum distance (`m`).
- If both conditions are true, it means we found an unvisited vertex with a tentative distance that's either smaller than or equal to the current minimum. In this case:
  - The value of `distance[vx]` (the vertex's tentative distance) is assigned to `m`, updating the current minimum.
  - The index of the vertex (`vx`) is assigned to `m_index`, keeping track of the vertex with the current minimum tentative distance.

2. Returning the Index of the Minimum Distance Vertex:

- After iterating through all vertices, the `m_index` will hold the index of the unvisited vertex with the minimum tentative distance from the source vertex (or -1 if no unvisited vertices were found).
- Finally, the function returns the `m_index`.

In summary, this function finds the unvisited vertex in the graph with the closest (minimum tentative distance) to the source vertex among all unvisited vertices. This vertex becomes the next point to explore in Dijkstra's algorithm for finding the shortest paths.

This code snippet iterates through the unvisited neighbors of a vertex (`ux`) in Dijkstra's algorithm and updates the tentative distances of those neighbors if necessary. Here's a breakdown of what it does:

```
for(int vx = 0; vx <totalVertexes; vx++)
            {
                if(!visited[vx] && graph[ux][vx]  != -1 &&
                   distance[ux] + graph[ux][vx] != Integer.MAX_VALUE &&
                   distance[ux] + graph[ux][vx]  < distance[vx] )
                   {
                        distance[vx]= distance[ux] + graph[ux][vx];
                   }
            }
```

1. Looping through Unvisited Neighbors:

- The code uses a loop (`for(int vx = 0; vx <totalVertexes; vx++)`) to iterate through all vertices (`vx`) in the graph.

2. Checking Conditions for Neighbor Update:

- Inside the loop, it checks four conditions to determine if the tentative distance of the current neighbor (`vx`) needs to be updated:
  - `!visited[vx]`: This condition checks if the neighbor (`vx`) is not visited. We only care about unvisited neighbors because we're exploring possible paths through the graph.
  - `graph[ux][vx] != -1`: This condition checks if there's an edge (connection) between the current vertex (`ux`) and the neighbor (`vx`) in the graph. A value of -1 in the adjacency matrix typically represents no connection.
  - `distance[ux] + graph[ux][vx] != Integer.MAX_VALUE`: This condition checks if adding the tentative distance of the current vertex (`distance[ux]`) and the weight of the edge between `ux` and `vx` (`graph[ux][vx]`) doesn't overflow to infinity. This ensures we don't encounter invalid distances.
  - `distance[ux] + graph[ux][vx] <distance[vx]`: This is the most important condition. It checks if the tentative distance from the source vertex to the current neighbor (`vx`) going through the current vertex (`ux`) is shorter than the current tentative distance of the neighbor (`distance[vx]`).

3. Updating Tentative Distance:

- If all four conditions are true, it means we found a shorter path to the neighbor (`vx`) by going through the current vertex (`ux`). In this case:
  - The new tentative distance is calculated by adding the tentative distance of the current vertex (`distance[ux]`) and the weight of the edge between them (`graph[ux][vx]`).
  - This new tentative distance is assigned to `distance[vx]`, effectively updating the tentative distance of the neighbor in the `distance` array.

In summary, this code snippet iterates through the unvisited neighbors of a vertex and updates the tentative distances of those neighbors if going through the current vertex provides a shorter path. This is a crucial step in Dijkstra's algorithm for finding the shortest paths from a source vertex to all other vertices in the graph.

Lecture Code

```
/*
Project: Dijkstra's Algorithm
Programmer: James Goudy
 */
```

```
/*

 */
package ds_dijstraka_demo;


class Dijkstra {

    // Number of vertices in the graph
    int totalVertexes;
    // Starting vertex for finding shortest paths
    int startVertex;

    public Dijkstra(int totalVertexes) {
        this.totalVertexes = totalVertexes;
    }

    // Finds the unvisited vertex with the minimum tentative distance from the source vertex
    int minimumDistance(int distance[], boolean visited[]) {
        // Initialize minimum value
        int m = Integer.MAX_VALUE;
        int m_index = -1;

        for (int vx = 0; vx < totalVertexes; vx++) {
            // Check if vertex is unvisited and has a tentative distance less than or equal to
            if (!visited[vx] && distance[vx] <= m) {
                m = distance[vx];
                m_index = vx;
            }
        }

        return m_index;
    }

    // Utility function to print the calculated shortest distances
    void printSolution(int distance[], int n) {
        System.out.println(String.format("The shortest distance from source %1$sth node\nto al

        for (int i = 0; i < n; i++) {
            System.out.println(n + " To " + i + " the shortest distance is : " + distance[i]);
        }

        System.out.println("\n - - - - - - - - - - -\n\n");
    }

    // Main function to calculate Dijkstra's shortest paths
    void calculate(int graph[][], int startVertex, int totalVertexes) {
        this.totalVertexes = totalVertexes;
        this.startVertex = startVertex;

        // Arrays to store tentative distances and visited status for vertices
        int distance[] = new int[totalVertexes];
        boolean visited[] = new boolean[totalVertexes];
```

```
        // Initialize all distances as infinite (except the starting vertex which is 0)
        for (int j = 0; j < totalVertexes; j++) {
            distance[j] = Integer.MAX_VALUE;
        }
        distance[startVertex] = 0;

        // Find shortest paths for all vertices iteratively
        for (int c = 0; c < totalVertexes - 1; c++) {
            // Get the unvisited vertex with the minimum tentative distance from the source ve
            int ux = minimumDistance(distance, visited);
            visited[ux] = true; // Mark the vertex as visited

            // Iterate through unvisited neighbors of the visited vertex
            for (int vx = 0; vx < totalVertexes; vx++) {
                // Check conditions to update tentative distance of the neighbor
                if (!visited[vx] && graph[ux][vx] != -1 &&
                        distance[ux] + graph[ux][vx] != Integer.MAX_VALUE &&
                        distance[ux] + graph[ux][vx] < distance[vx]) {
                    distance[vx] = distance[ux] + graph[ux][vx]; // Update tentative distance
                }
            }
        }

        // Print the calculated shortest distances
        printSolution(distance, totalVertexes);
    }
}

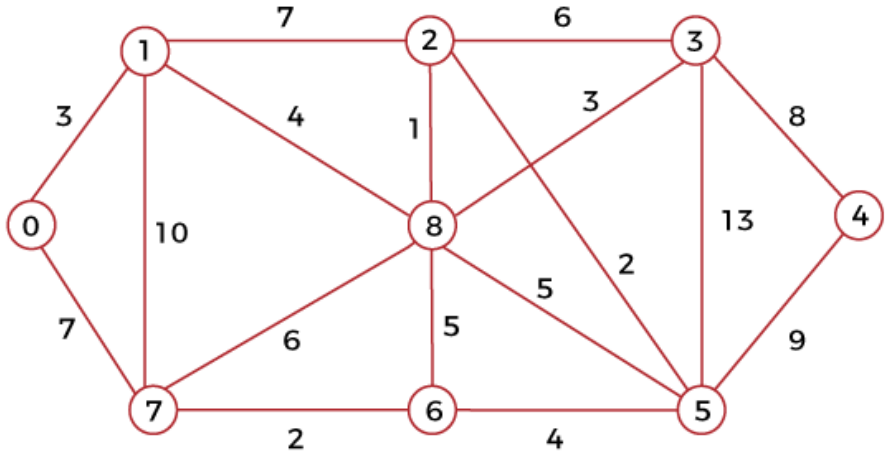
public class DS_Dijstraka_Demo {


    public static void main(String[] args) {
        int graph[][] = new int[][]{
            { -1, 3, -1, -1, -1, -1, -1, 7, -1},
            { 3, -1, 7, -1, -1, -1, -1,10, 4},
            { -1, 7, -1, 6, -1, 2, -1, -1, 1},
            { -1, -1, 6, -1, 8,13, -1, -1, 3},
            { -1, -1, -1, 8, -1, 9, -1, -1, -1},
            { -1, -1, 2,13, 9, -1, 4, -1, 5},
            { -1, -1, -1, -1, -1, 4, -1, 2, 5},
            { 7,10, -1, -1, -1, -1, 2, -1, 6},
            { -1, 4, 1, 3, -1, 5, 5, 6, -1}
        };

        Dijkstra dgrpah = new Dijkstra(9);
        dgrpah.calculate(graph, 8,9);

    }

}
}
```


0
1
2
3
4
5
6
7
8
7
3
10
4
1
6
3
8
13
2
5
5
7
6
9
2
4

0.7 Huffman Code

0.7.1 What is the Huffman Algorithm?

At its core, the **Huffman Algorithm** is a **lossless data compression algorithm**.1 The "lossless" part is key – it means that when we compress data using this method and then decompress it, we get back an exact copy of the original data.2 Nothing is lost in the process.

The genius of the Huffman Algorithm, developed by David A. Huffman while he was a Ph.D. student at MIT in 1952, lies in how it assigns codes to characters.3 Instead of using a fixed number of bits for every character (like in standard ASCII, which uses 7 or 8 bits per character), Huffman coding uses **variable-length codes**.4 The most frequently occurring characters get shorter codes, and the less frequent characters get longer codes.5 This ultimately leads to an overall reduction in the number of bits needed to represent the data.

Think of it like this: if you're sending a message and the letter 'e' appears most often, wouldn't it be more efficient to have a very short symbol for 'e', and perhaps a longer one for a rare letter like 'z'? That's the fundamental idea.

How Does It Work? The Encoding Process

The Huffman Algorithm works by building a special type of binary tree called a **Huffman Tree** (or sometimes a prefix tree).6 Here's a step-by-step breakdown:

1. **Count Frequencies:** First, we need to know how often each character (or symbol) appears in the data we want to compress. So, we scan the entire input data and count the occurrences of each unique character.

    - Example:
      Let's say our data is "ABRACADABRA".
      - A: 5 times
      - B: 2 times
      - R: 2 times
      - C: 1 time
      - D: 1 time

2. **Create Leaf Nodes:** For each unique character, we create a "leaf node" (think of the bottom-most parts of a tree). Each leaf node will store the character itself and its frequency.

3. **Build the Tree (The Greedy Part):** This is where the "greedy" nature of the algorithm comes in.7 A greedy algorithm is one that makes the locally optimal choice at each stage with the hope of finding a global optimum.89

- We treat our list of leaf nodes as a priority queue, where the priority is determined by the frequency (lowest frequency has the highest priority for being picked).
- Repeat the following steps until only one node (the root of the Huffman Tree) remains:
  - a. **Select Two Nodes:** Extract the two nodes with the lowest frequencies from the queue.
  - b. **Create a Parent Node:** Create a new internal node (a node that is not a leaf). This new node will be the parent of the two nodes selected in step (a).
  - c. **Assign Frequency to Parent:** The frequency of this new parent node is the sum of the frequencies of its two children.10
  - d. **Add to Queue:** Insert this new parent node back into the priority queue.

Let's visualize with our "ABRACADABRA" example:

- Initial nodes (Character: Frequency): (C:1), (D:1), (B:2), (R:2), (A:5)
- **Step 1:** Pick (C:1) and (D:1). Create parent (CD:2). Nodes: (CD:2), (B:2), (R:2), (A:5)
- **Step 2:** Pick (CD:2) and (B:2) (or (R:2), order doesn't matter for same frequencies here for the *selection*, though it might slightly change the tree structure but not the overall compression efficiency). Let's pick (CD:2) and (B:2). Create parent (CDB:4). Nodes: (R:2), (CDB:4), (A:5)

- **Step 3:** Pick (R:2) and (CDB:4). (Note: (R:2) has lower frequency). Create parent (RCDB:6). Nodes: (A:5), (RCDB:6)
- **Step 4:** Pick (A:5) and (RCDB:6). Create parent (ARCDB:11). This is our root node.

4. **Assign Codes:** Once the Huffman Tree is built, we assign binary codes to each character.11 We do this by traversing the tree from the root to each leaf node:

- Assign a '0' to every left branch (or edge) in the tree.
- Assign a '1' to every right branch (or edge) in the tree.
- The Huffman code for each character is the sequence of 0s and 1s encountered on the path from the root to that character's leaf node.

Continuing our example (this depends on how we arranged left/right children, let's assume one way):

A possible tree structure and codes:

```
(11)
        /    \
       A(5)  (6)
            /   \
           (4)   R(2)
          /   \
         B(2) (2)
              /  \
             C(1) D(1)
```

If we assign '0' for left and '1' for right:

- A: `0`
- R: `11`
- B: `100`
- C: `1010`
- D: `1011`

Notice how 'A' (most frequent) has the shortest code, and 'C' and 'D' (least frequent) have the longest codes. This is the magic of Huffman coding! Also, importantly, no code is a prefix of another code (e.g., '0' is A's code, no other code starts with '0'). This "prefix property" is crucial for decoding.

5. **Encode the Data:** Replace each character in the original data with its newly generated Huffman code.
    - "ABRACADABRA" becomes: `0 100 11 0 1010 0 1011 0 100 11 0`

How Does It Work? The Decoding Process

Decoding is relatively straightforward if you have the Huffman Tree (or the code table).

1. **Obtain the Huffman Tree/Codes:** The decoder needs the same Huffman Tree (or the mapping of codes to characters) that was used for encoding. This tree/table is often transmitted along with the compressed data.
2. **Read the Compressed Bit Stream:** Start reading the encoded data bit by bit.
3. **Traverse the Tree:**
    - Begin at the root of the Huffman Tree.
    - For each bit read from the compressed stream:
        - If the bit is '0', move to the left child in the tree.
        - If the bit is '1', move to the right child in the tree.

- When you reach a leaf node, you have decoded one character. Record that character.
- Return to the root of the tree and repeat the process with the next bit in the compressed stream until all bits are processed.

Example of decoding `010011...` using our codes:

1. Read `0`: Path is root ->left. It's 'A'. Output 'A'. Back to root.
2. Read `1`: Path is root ->right. Not a leaf.
3. Read `0`: Path is current node ->left. Not a leaf.
4. Read `0`: Path is current node ->left. It's 'B'. Output 'B'. Back to root.
5. Read `1`: Path is root ->right. Not a leaf.
6. Read `1`: Path is current node ->right. It's 'R'. Output 'R'. Back to root. And so on.

Because of the prefix property (no code is a prefix of another), there's no ambiguity in decoding. As soon as you reach a leaf, you know you've completed a character.

Where is Huffman Coding Used?

Huffman coding, either in its pure form or as a component of other algorithms, is used in a wide variety of applications.12 Here are some prominent examples:

1. **File Compression:**
   - **ZIP files:** Classic .zip files often use a method called "Deflate," which combines LZ77 (another compression algorithm) with Huffman coding.13
   - **GZIP files:** Used extensively on Unix-like systems and for web content, GZIP also uses Deflate.14
2. **Image Compression:**
   - **JPEG:** While JPEG primarily uses a "lossy" compression (Discrete Cosine Transform), Huffman coding is often used as one of the final, lossless steps to compress the resulting data coefficients.
   - **PNG:** This is a lossless image format, and it uses the Deflate algorithm (which includes Huffman coding) for compression.
3. **Multimedia Formats:**
   - **MP3 (Audio):** Huffman coding is used as part of the MP3 compression process to reduce the size of audio data.15
   - **MPEG (Video):** Similar to JPEGs and MP3s, various MPEG video formats (like MPEG-2 used in DVDs, MPEG-4 used in many online videos) incorporate Huffman coding for entropy encoding parts of the video and audio data.
4. **Communication Protocols:**
   - **HTTP/2 and HTTP/3 (HPACK and QPACK):** When your browser communicates with a web server using newer HTTP versions, header compression techniques like HPACK (for HTTP/2) and QPACK (for HTTP/3) use Huffman coding to reduce the size of HTTP headers, making web pages load faster.16
   - Fax machines also historically used forms of Huffman coding.17
5. **Text and Data Archiving:** Anytime there's a need to store or transmit text or other data more efficiently without losing information, Huffman coding or algorithms incorporating it can be considered.

In summary, the Huffman Algorithm is a foundational technique in data compression. By assigning shorter codes to more frequent symbols and longer codes to less frequent ones, it efficiently reduces the overall size of data for storage or transmission, all while ensuring the original data can be perfectly reconstructed.18

0.7.2 Example Code

```
package huffman_code;

import java.util.*;

// Node class for Huffman tree
class Node {

    char ch;
    int freq;
    Node left, right;

    Node(char ch, int freq)
    {
        this.ch = ch;
        this.freq = freq;
    }

    Node(int freq, Node left, Node right)
    {
        this.ch = '\0'; // internal node
        this.freq = freq;
        this.left = left;
        this.right = right;
    }

    boolean isLeaf()
    {
        return left == null && right == null;
    }
}

class HuffmanCode {

    private Node root;
    private Map<Character, String> huffmanCode;

    public HuffmanCode()
    {

    }

    public void runHuffaman(String text)
    {

        Map<Character, Integer> freqMap = buildFrequencyMap(text);
        this.root = buildTree(freqMap);
        this.huffmanCode = new HashMap<>();
```

```
        buildCodes(root, "", huffmanCode);

    }

    private Map<Character, Integer> buildFrequencyMap(String text)
    {
        Map<Character, Integer> freqMap = new HashMap<>();
        for (char ch : text.toCharArray()) {
            freqMap.put(ch, freqMap.getOrDefault(ch, 0) + 1);
        }
        return freqMap;
    }

    private Node buildTree(Map<Character, Integer> freqMap)
    {
        PriorityQueue<Node> pq = new PriorityQueue<>(Comparator.comparingInt(n -> n.freq));
        for (Map.Entry<Character, Integer> entry : freqMap.entrySet()) {
            pq.add(new Node(entry.getKey(), entry.getValue()));
        }

        while (pq.size() > 1) {
            Node left = pq.poll();
            Node right = pq.poll();
            pq.add(new Node(left.freq + right.freq, left, right));
        }

        return pq.poll(); // root
    }

    private void buildCodes(Node node, String code, Map<Character, String> map)
    {
        if (node == null) {
            return;
        }

        if (node.isLeaf()) {
            map.put(node.ch, code);
        }

        buildCodes(node.left, code + "0", map);
        buildCodes(node.right, code + "1", map);
    }

    public String encode(String text)
    {
        StringBuilder encoded = new StringBuilder();
        for (char ch : text.toCharArray()) {
            encoded.append(huffmanCode.get(ch));
        }
        return encoded.toString();
    }

    public String decode(String encodedText)
```

```
    {
        StringBuilder decoded = new StringBuilder();
        Node current = root;
        for (char bit : encodedText.toCharArray()) {
            current = (bit == '0') ? current.left : current.right;
            if (current.isLeaf()) {
                decoded.append(current.ch);
                current = root;
            }
        }
        return decoded.toString();
    }

    public Map<Character, String> getHuffmanCodes()
    {
        return this.huffmanCode;
    }

    public String stringToBinary(String text)
    {

        String newString = "";
        StringBuilder sb;

        for (int i = 0; i < text.length(); i++) {
            String myBinary = "";
            int asciVal = Integer.valueOf(text.charAt(i));

            while (asciVal > 0) {
                myBinary = myBinary + (asciVal % 2 == 1 ? "1" : "0");
                asciVal /= 2;
            }

            sb = new StringBuilder(myBinary).reverse();
            newString += sb + "";
        }

        return newString;
    }
}

public class Huffman_Code {

    public static void main(String[] args)
    {

        String quit = "n";
        Scanner myScan = new Scanner(System.in);

        String text = "huffman";
```

```
        HuffmanCode huffman = new HuffmanCode();

        System.out.println("Humman Code");

        while (quit.equals("n")) {

            System.out.print("Please enter a word or string: ");
            text = myScan.nextLine();

            huffman.runHuffaman(text);

            // Print codes
            System.out.println(text);
            for (var entry : huffman.getHuffmanCodes().entrySet()) {
                System.out.println(entry.getKey() + ": " + entry.getValue());
            }

            // Encode
            String encoded = huffman.encode(text);
            System.out.println("\nEncoded Text : " + encoded);
            System.out.println("Binary       : " + huffman.stringToBinary(text));

            // Decode
            String decoded = huffman.decode(encoded);
            System.out.println("\nDecoded Text:\n" + decoded);

            System.out.print("Would you like to quit: y/n:  ");
            quit = myScan.nextLine();

        }

        System.out.println("\nbye\n");


    }

}

/*
Please enter a word or string: Mississippi
Mississippi
p: 101
s: 0
i: 11
M: 100

Encoded Text : 100110011001110110111
Binary       : 1001101110100111100111111001111010011110011111001111010011110000111000011010011

Decoded Text:
Mississippi
Would you like to quit: y/n: n
```

```
Please enter a word or string: Bubba blows bubbles
Bubba blows bubbles
 : 010
a: 0000
B: 10111
b: 11
s: 011
u: 100
e: 1010
w: 0001
l: 001
o: 10110

Encoded Text : 10111100111100000101100110110000101101011100111100111010011
Binary       : 10000101110101110001011000101100001100000110001011011001101111111011111100111100

Decoded Text:
Bubba blows bubbles
Would you like to quit: y/n:  y

 bye

*/
```

0.7.3 Code Explanation

We have two main pieces here: a `Node` class and a `HuffmanCode` class.

1. The `Node` Class: The Building Blocks of Our Tree

Java

```
// Node class for Huffman tree
class Node {

    char ch;           // The character this node might represent (if it's a leaf)
    int freq;          // How often this character appears, or sum of frequencies of children
    Node left, right;  // The children of this node (null if it's a leaf)

    // Constructor for LEAF nodes (nodes that represent actual characters)
    Node(char ch, int freq)
    {
        this.ch = ch;
        this.freq = freq;
        // left and right will be null by default for a new leaf node
    }

    // Constructor for INTERNAL nodes (nodes that are created by combining two other nodes)
    Node(int freq, Node left, Node right)
    {
        this.ch = '\0'; // A null character, indicating it's not a character leaf
        this.freq = freq;
        this.left = left;
        this.right = right;
    }
```

```
    // Helper method to check if this node is a leaf node
    boolean isLeaf()
    {
        return left == null && right == null; // A leaf has no children
    }
}
```

Think of the `Node` class as the blueprint for every single point or junction in our Huffman Tree.

- **`char ch;`**: This variable holds the actual character (like 'A', 'B', 'C') if this node is a *leaf node* – meaning it's at the very bottom of a branch and represents a character from our input text.
- **`int freq;`** This stores the frequency.
  - If it's a leaf node, `freq` is how many times its character (`ch`) appears in the text.
  - If it's an *internal node* (a connection point within the tree, not a character itself), `freq` is the sum of the frequencies of all the leaves below it.
- **`Node left, right;`**: These are references to other `Node` objects. They are like the branches of the tree. `left` points to the child node on the left, and `right` points to the child node on the right. If a node is a leaf, both `left` and `right` will be `null` (meaning they don't point to anything).

Constructors (the Node(...) methods):
These are special methods used to create new Node objects.

- **`Node(char ch, int freq)`**: This is used to create a **leaf node**. You give it a character and its frequency.
- **`Node(int freq, Node left, Node right)`**: This is used to create an **internal node**. You give it the combined frequency of its children, and references to what its `left` child and `right` child should be. We use a special character `'\0'` (the null character) for `ch` just to signify that this internal node doesn't represent a specific character itself.

`isLeaf()` method:

- **`boolean isLeaf()`**: This is a handy little helper. It returns `true` if the node has no left or right children (meaning it's a leaf), and `false` otherwise.

2. The `HuffmanCode` Class: The Brains of the Operation

Java

```
class HuffmanCode {

    private Node root; // This will store the very top node (the root) of our Huffman tree
    private Map<Character, String> huffmanCode; // This will store the generated codes (e.g.,

    // Constructor (currently empty, could be used for setup if needed)
    public HuffmanCode()
    {

    }

    // Main method to orchestrate the Huffman encoding process
    public void runHuffaman(String text)
    {
        // Step 1: Count character frequencies
        Map<Character, Integer> freqMap = buildFrequencyMap(text);

        // Step 2: Build the Huffman Tree using these frequencies
```

```
        this.root = buildTree(freqMap);

        // Step 3: Generate the codes (like "0" or "101") for each character from the tree
        this.huffmanCode = new HashMap<>(); // Initialize the map to store codes
        buildCodes(root, "", huffmanCode);  // Start building codes from the root
    }
```

The `HuffmanCode` class contains all the logic to take a piece of text, build the tree, generate the codes, and then encode or decode text.

- **`private Node root;`**: This variable will eventually hold the single `Node` that is at the very top of our completed Huffman tree. It's our entry point into the tree.
- **`private Map<Character, String>huffmanCode;`**: A `Map` is like a dictionary. This one will store each character from our text as a "key" and its corresponding Huffman code (a string of '0's and '1's) as its "value". For example, `{'A': "0", 'B': "100", ...}`.

Let's look at its key methods:
public void runHuffaman(String text):
This is like the main manager for the Huffman process.

1. It first calls `buildFrequencyMap(text)` to count how many times each character appears in the input `text`.
2. Then, it calls `buildTree(freqMap)` using those frequencies to construct the actual Huffman tree. The result (the root of the tree) is stored in `this.root`.
3. Finally, it initializes `this.huffmanCode` as a new empty map and calls `buildCodes(root, "", huffmanCode)` to traverse the tree and fill up the `huffmanCode` map with the binary codes for each character.

`private Map<Character, Integer>buildFrequencyMap(String text):`
Java

```
private Map<Character, Integer> buildFrequencyMap(String text)
    {
        Map<Character, Integer> freqMap = new HashMap<>(); // Create an empty dictionary
        for (char ch : text.toCharArray()) { // Loop through each character in the input text
            // Get the current count of 'ch', or 0 if not seen yet, then add 1
            freqMap.put(ch, freqMap.getOrDefault(ch, 0) + 1);
        }
        return freqMap; // Return the map with all character counts
    }
```

This method does exactly what we described as the first step of the Huffman algorithm:

- It creates an empty `HashMap` called `freqMap`.
- It then loops through every single character (`ch`) in the input `text`.
- For each character, it updates its count in `freqMap`. The `getOrDefault(ch, 0)` part is clever: it tries to get the current count of `ch`. If `ch` hasn't been seen before, it defaults to `0`, and then `1` is added.
- It returns this populated `freqMap`. If `text` was "ABCA", `freqMap` would look like `{'A': 2, 'B': 1, 'C': 1}`.

`private Node buildTree(Map<Character, Integer>freqMap):`
Java

```
private Node buildTree(Map<Character, Integer> freqMap)
    {
        // Create a priority queue. Nodes with lower frequency have higher priority.
        PriorityQueue<Node> pq = new PriorityQueue<>(Comparator.comparingInt(n -> n.freq));

        // Create a leaf node for each character and add it to the priority queue.
        for (Map.Entry<Character, Integer> entry : freqMap.entrySet()) {
            pq.add(new Node(entry.getKey(), entry.getValue()));
        }

        // Loop as long as there is more than one node in the queue
        while (pq.size() > 1) {
            // Remove the two nodes with the smallest frequency
            Node left = pq.poll();  // Smallest
            Node right = pq.poll(); // Second smallest

            // Create a new internal node with these two nodes as children
            // and frequency equal to the sum of the two nodes' frequencies.
            // Add the new node to the priority queue.
            pq.add(new Node(left.freq + right.freq, left, right));
        }

        // The remaining node is the root of the Huffman Tree
        return pq.poll();
    }
```

This is where the Huffman tree is actually built, following the "greedy" approach:

- **`PriorityQueue<Node>pq = new PriorityQueue<>(Comparator.comparingInt(n ->n.freq));`**: A `PriorityQueue` is a special kind of list where items are automatically kept in a certain order based on their "priority." Here, we're telling it to prioritize `Node` objects based on their `freq` attribute – specifically, nodes with *lower* frequencies will be considered higher priority (meaning they come out first).
- **`for (Map.Entry<Character, Integer>entry : freqMap.entrySet()) { ... }`**: It loops through our `freqMap`. For each character and its frequency, it creates a new **leaf `Node`** (using the first `Node` constructor) and adds it to the `pq`. So now, `pq` is full of leaf nodes, ordered by their frequency.
- **`while (pq.size() >1) { ... }`** This loop is the heart of the tree construction. It keeps running as long as there's more than one node in the

  `pq`

  .
  - **`Node left = pq.poll();`** and **`Node right = pq.poll();`**: It takes out the two nodes with the *lowest frequencies* from the `pq`. (Remember, lowest frequency = highest priority).
  - **`pq.add(new Node(left.freq + right.freq, left, right));`** It creates a new internal `Node`

    .
    - Its frequency is the sum of the frequencies of `left` and `right`.
    - `left` becomes its left child, and `right` becomes its right child.
    - This new internal node is then added back into the `pq`. It will find its correct place in the priority order based on its new (combined) frequency.
- **`return pq.poll();`**: When the loop finishes, there's only one node left in the `pq`. This single node is the **root** of the entire Huffman Tree.

`private void buildCodes(Node node, String code, Map<Character, String>map):`
Java

```
private void buildCodes(Node node, String code, Map<Character, String> map)
    {
        if (node == null) { // Base case: if we've gone past a leaf
            return;
        }

        // If this is a leaf node, we've found a character!
        // Store the character and its accumulated code in our map.
        if (node.isLeaf()) {
            map.put(node.ch, code);
        }

        // Recursively call for the left child, appending "0" to the code
        buildCodes(node.left, code + "0", map);
        // Recursively call for the right child, appending "1" to the code
        buildCodes(node.right, code + "1", map);
    }
```

This method figures out the '0's and '1's for each character. It's a **recursive** method, meaning it calls itself to solve smaller parts of the problem.

- **`if (node == null) { return; }`**: This is a safety check or a "base case" for the recursion. If we try to go to a child that doesn't exist, we just stop that path.
- **`if (node.isLeaf()) { map.put(node.ch, code); }`**: If the current `node` we're looking at is a leaf node (it has a character!), we've found the end of a path from the root. The `code` string that we've built up by travelling down the tree is the Huffman code for `node.ch`. So, we store this pair in our `map` (which is `huffmanCode` from the `runHuffman` method).
- **`buildCodes(node.left, code + "0", map);`**: This is the recursive step. It calls `buildCodes` again, but this time for the `left` child of the current node. Crucially, it appends a `"0"` to the `code` string because we're taking a left branch.
- **`buildCodes(node.right, code + "1", map);`**: Similarly, it calls `buildCodes` for the `right` child, appending a `"1"` to the `code` string for taking a right branch.

Imagine starting at the `root` with an empty `code` string (`""`). If you go left, the code becomes "0". If you go left again, it's "00". If you then reach a leaf for character 'X', 'X' gets the code "00". The method backtracks and explores all paths.

`public String encode(String text):`
Java

```
public String encode(String text)
    {
        StringBuilder encoded = new StringBuilder();
        for (char ch : text.toCharArray()) { // Loop through each character of the input text
            encoded.append(huffmanCode.get(ch)); // Look up its Huffman code and add it
        }
        return encoded.toString(); // Return the full string of 0s and 1s
    }
```

Once `runHuffman` has been called (so the `huffmanCode` map is filled), this method takes the original `text` and converts it into the compressed string of 0s and 1s.

- It iterates through each character (`ch`) of the input `text`.
- For each `ch`, it looks up its corresponding Huffman code in the `huffmanCode` map (e.g., if `ch` is 'A', it might get "0").
- It appends this code to a `StringBuilder` (which is an efficient way to build strings).
- Finally, it returns the complete encoded string.

`public String decode(String encodedText):`
Java

```
public String decode(String encodedText)
    {
        StringBuilder decoded = new StringBuilder();
        Node current = root; // Start at the root of our Huffman tree
        for (char bit : encodedText.toCharArray()) { // Loop through each '0' or '1' in the en
            // If the bit is '0', move to the left child. Otherwise, move to the right child.
            current = (bit == '0') ? current.left : current.right;

            // If we've reached a leaf node, we've decoded a character!
            if (current.isLeaf()) {
                decoded.append(current.ch); // Add the character to our result
                current = root;             // Go back to the root for the next character
            }
        }
        return decoded.toString(); // Return the fully decoded text
    }
```

This method takes a compressed string of 0s and 1s (`encodedText`) and converts it back to the original text.

- It starts with `current` pointing to the `root` of our Huffman tree.
- It reads the `encodedText` one `bit` ('0' or '1') at a time.
  - If the `bit` is '0', it moves `current` to its `left` child.
  - If the `bit` is '1', it moves `current` to its `right` child.
- **`if (current.isLeaf()) { ... }`** After moving, it checks if

  `current`

  is now a leaf node.
  - If it is, it means we've successfully traced a complete Huffman code. The character at this leaf (`current.ch`) is appended to our `decoded` result.
  - Importantly, `current` is then reset back to `root` to start searching for the next character from the current position in the `encodedText`.
- It returns the fully `decoded` string.

`public Map<Character, String>getHuffmanCodes():`
Java

```
public Map<Character, String> getHuffmanCodes()
    {
        return this.huffmanCode; // Simply returns the map of characters to their codes
    }
```

This is a simple "getter" method. It just returns the `huffmanCode` map that was generated, in case you want to inspect the codes outside of this class.

`public String stringToBinary(String text):`
Java

```
public String stringToBinary(String text)
    {
        String newString = "";
        StringBuilder sb;

        for (int i = 0; i < text.length(); i++) {
            String myBinary = "";
            int asciVal = Integer.valueOf(text.charAt(i)); // Get ASCII value of character

            // Convert ASCII value to its binary representation
            while (asciVal > 0) {
                myBinary = myBinary + (asciVal % 2 == 1 ? "1" : "0");
                asciVal /= 2;
            }

            // The above loop generates binary in reverse, so reverse it
            sb = new StringBuilder(myBinary).reverse();
            newString += sb + ""; // Append to the result
        }
        return newString;
    }
}
```

This method is a bit different from the core Huffman logic. It appears to be a utility function to convert a given string into its **standard fixed-length binary representation** (likely 7 or 8-bit ASCII, padded implicitly).

- It iterates through each character of the input `text`.
- `int asciVal = Integer.valueOf(text.charAt(i));` gets the numerical ASCII value of the character (e.g., 'A' is 65).
- The `while (asciVal >0)` loop converts this decimal ASCII value into a binary string. It does this by repeatedly taking the number modulo 2 (to get the last bit) and then dividing by 2.
- `sb = new StringBuilder(myBinary).reverse();` is needed because the loop builds the binary string in reverse order.
- It concatenates these binary representations for all characters.

Important Note on `stringToBinary`: This method is *not* producing Huffman codes. Huffman codes are variable-length and based on frequency. `stringToBinary` produces fixed-length codes (like standard ASCII-to-binary). It might be included in this class for comparison purposes – to show how much space Huffman coding saves compared to a standard binary representation of text. For example, you could encode "AAAAA" using Huffman (might be "00000") and then using `stringToBinary` (might be "01000001010000010100000101000001010000001") to see the difference.

So, the main parts for the Huffman algorithm itself are `runHuffman` (and the methods it calls: `buildFrequencyMap`, `buildTree`, `buildCodes`), `encode`, and `decode`. The `Node` class is the essential data structure it uses.

Hopefully, this breaks down the code in a way that makes sense! The key is to see how these different code pieces work together to implement the steps of the Huffman algorithm we discussed earlier.

0.8 Serialization

0.8.1 Definition of Serialization

Serialization is the process of converting an object into a format that can be stored or transmitted and later reconstructed. This allows objects to be saved to a file, sent over a network, or stored in a database in a way that they can be reconstructed later.

0.8.2 Uses of Serialization

1. • (a) **Persistence** – Storing objects in a file or database for later retrieval.
   (b) **Communication** – Transmitting objects over a network (e.g., in distributed applications).
   (c) **Caching** – Storing objects in memory to speed up future access.
   (d) **Deep Copying** – Cloning objects by serializing and deserializing them.
   (e) **Interoperability** – Sharing data between different programming languages or systems.

0.8.3 Example in Java

In Java, serialization is implemented using the `Serializable` interface.

```
import java.io.*;

// Class that implements Serializable
class Person implements Serializable {
    private static final long serialVersionUID = 1L;  // Ensure versioning compatibil
    String name;
    int age;

    // Constructor
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Method to display object data
    public void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}

public class SerializationExample {
    public static void main(String[] args) {
        Person person = new Person("John", 30);

        // Serialize the object
        try (FileOutputStream fileOut = new FileOutputStream("person.ser");
             ObjectOutputStream out = new ObjectOutputStream(fileOut)) {
            out.writeObject(person);
            System.out.println("Object serialized successfully.");
        } catch (IOException e) {
            e.printStackTrace();
        }
```

```
        // Deserialize the object
        try (FileInputStream fileIn = new FileInputStream("person.ser");
             ObjectInputStream in = new ObjectInputStream(fileIn)) {
            Person deserializedPerson = (Person) in.readObject();
            System.out.println("Object deserialized successfully.");
            deserializedPerson.display();
        } catch (IOException | ClassNotFoundException e) {
            e.printStackTrace();
        }
    }
}
```

Explanation:

(a) The `Person` class implements `Serializable`.

(b) `ObjectOutputStream` writes the object to a file (`person.ser`).

(c) `ObjectInputStream` reads the object back and restores it.

0.8.4 Equivalent Example in Python

In Python, `pickle` is commonly used for serialization.

```
import pickle

# Class definition
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print(f"Name: {self.name}, Age: {self.age}")

# Serialize object
person = Person("John", 30)
with open("person.pkl", "wb") as file:
    pickle.dump(person, file)
    print("Object serialized successfully.")

# Deserialize object
with open("person.pkl", "rb") as file:
    deserialized_person = pickle.load(file)
    print("Object deserialized successfully.")
    deserialized_person.display()
```

Key Differences:

- `pickle.dump()` serializes the object.
- `pickle.load()` deserializes the object.

0.8.5 Equivalent Example in C#

In C#, `System.Runtime.Serialization` and `System.Xml.Serialization` are used.

```
using System;
using System.IO;
using System.Runtime.Serialization;
using System.Runtime.Serialization.Formatters.Binary;

[Serializable]
class Person
{
    public string Name { get; set; }
    public int Age { get; set; }

    public Person(string name, int age)
    {
        Name = name;
        Age = age;
    }

    public void Display()
    {
        Console.WriteLine($"Name: {Name}, Age: {Age}");
    }
}

class SerializationExample
{
    static void Main()
    {
        Person person = new Person("John", 30);
        string filePath = "person.dat";

        // Serialize object
        IFormatter formatter = new BinaryFormatter();
        using (Stream stream = new FileStream(filePath, FileMode.Create, FileAccess.W
        {
            formatter.Serialize(stream, person);
            Console.WriteLine("Object serialized successfully.");
        }

        // Deserialize object
        using (Stream stream = new FileStream(filePath, FileMode.Open, FileAccess.Rea
        {
            Person deserializedPerson = (Person)formatter.Deserialize(stream);
            Console.WriteLine("Object deserialized successfully.");
            deserializedPerson.Display();
        }
    }
}
```

Key Differences:

- Uses `[Serializable]` attribute.
- `BinaryFormatter` is used for serialization and deserialization.

– `FileStream` handles writing and reading from a file.

0.8.6 Summary

– **Java:** Uses `Serializable` interface and `ObjectOutputStream`/`ObjectInputStream`.
– **Python:** Uses `pickle.dump()` and `pickle.load()`.
– **C#:** Uses `[Serializable]` attribute and `BinaryFormatter`.

Each language has different serialization approaches but achieves the same goal—converting objects into a storable/transmittable format and reconstructing them later.

0.9 JAVA Techniques

These are techniques specific to JAVA.

0.9.1 Examples of Java's HashMap, TreeMap, and LinkedHashMap

1. `HashMap`

Characteristics:

- Unordered collection (no guarantees on order).
- Fastest performance for basic operations (`get`, `put`) with O(1) average complexity.
- Allows `null` as key/value.

Example:

```
import java.util.HashMap;
import java.util.Map;

public class HashMapExample {
    public static void main(String[] args) {
        Map<String, Integer> map = new HashMap<>();

        map.put("apple", 3);
        map.put("banana", 2);
        map.put("orange", 5);

        System.out.println("Value of apple: " + map.get("apple"));

        System.out.println("Iterating HashMap:");
        for (String key : map.keySet()) {
            System.out.println("Key: " + key + ", Value: " + map.get(key));
        }
    }
}
```

Best Uses:

- When you need fast lookup and insertion.
- Order doesn't matter.

---

2. `TreeMap`

Characteristics:

- Sorted according to natural ordering or a specified comparator.
- `O(log n)` complexity for `put`, `get`, `remove` operations.
- Cannot contain `null` keys (but can have null values).

Example:

```
import java.util.Map;
import java.util.TreeMap;

public class TreeMapExample {
    public static void main(String[] args) {
        Map<String, Integer> map = new TreeMap<>();

        map.put("apple", 3);
        map.put("banana", 2);
```

```
        map.put("orange", 5);

        System.out.println("Value of banana: " + map.get("banana"));

        System.out.println("Iterating TreeMap (sorted):");
        for (String key : map.keySet()) {
            System.out.println("Key: " + key + ", Value: " + map.get(key));
        }
    }
}
```

Best Uses:

- Maintaining sorted order (such as alphabetical order).
- Range queries (`subMap()`, `headMap()`, `tailMap()`).

---

3. `LinkedHashMap`

Characteristics:

- Maintains insertion order.
- Slightly slower than `HashMap`, with O(1) average complexity.
- Allows `null` key/value.

Example:

```
import java.util.LinkedHashMap;
import java.util.Map;

public class LinkedHashMapExample {
    public static void main(String[] args) {
        Map<String, Integer> map = new LinkedHashMap<>();

        map.put("apple", 3);
        map.put("banana", 2);
        map.put("orange", 5);

        System.out.println("Value of orange: " + map.get("orange"));

        System.out.println("Iterating LinkedHashMap (insertion order):");
        for (String key : map.keySet()) {
            System.out.println("Key: " + key + ", Value: " + map.get(key));
        }
    }
}
```

Best Uses:

- Caching implementations, where insertion/access order is important.
- Preserving the order of inserted keys (e.g., configurations, ordered tasks).

Summary (Quick Reference):

Feature	`HashMap`	`TreeMap`	`LinkedHashMap`
Order	No order guarantee	Sorted	Insertion order
Performance (`get`, `put`)	O(1) avg.	O(log n)	O(1) avg.
Null Keys Allowed	Yes	No	Yes

References:

- Oracle Documentation – HashMap
- Oracle Documentation – TreeMap
- Oracle Documentation – LinkedHashMap

0.9.2 HashSet, LinkedHashSet, and TreeSet Comparision

Comparison of `HashSet`, `LinkedHashSet`, and `TreeSet` in Java. These are all implementations of the `Set` interface, meaning they store unique elements. Their main differences lie in their internal implementation, element ordering, performance characteristics, and handling of null values.

Core Similarity:

- **Implement `Set` Interface:** All three classes implement the `java.util.Set` interface.
- **Store Unique Elements:** They do not allow duplicate elements. Adding an element that is already present (according to the `equals()` method) has no effect.

Key Differences:

Here's a table summarizing the main distinctions:

Feature	HashSet	LinkedHashSet	TreeSet
Underlying Structure	`HashMap`	`LinkedHashMap`	`TreeMap` (Red-Black Tree)
Ordering	Unordered	Insertion Order	Sorted Order (Natural or Custom)
Performance (Avg)	O(1) for `add`, `remove`, `contains`	O(1) for `add`, `remove`, `contains` (slightly higher constant factor than HashSet)	O(logn) for `add`, `remove`, `contains`
Iteration Performance	Proportional to capacity + size	Proportional to size	Proportional to size (O(n) total)
Null Elements	Allows one `null` element	Allows one `null` element	Does *not* allow `null` elements (by default)[1]
Interfaces Implemented	`Set`, `Cloneable`, `Serializable`	`Set`, `Cloneable`, `Serializable`	`Set`, `NavigableSet`, `SortedSet`, `Cloneable`, `Serializable`
Key Requirement	`hashCode()` and `equals()` must be consistent	`hashCode()` and `equals()` must be consistent	Elements must be mutually comparable (implement `Comparable` or use a `Comparator`)[2]

Detailed Explanations:

1. `HashSet`
   - **Implementation:** Uses a `HashMap` internally. The elements you add to the `HashSet` become the keys in the underlying `HashMap`, and a dummy `Object` is used as the value.
   - **Ordering:** It makes no guarantees about the iteration order of the elements. The order might even change over time as more elements are added and the internal hash table is resized.
   - **Performance:** Offers the best average-case performance (O(1)) for basic operations (`add`, `remove`, `contains`), assuming the hash function disperses elements properly among the buckets. However, iteration performance depends on both the number of elements (size) and the hash table's capacity.
   - **Nulls:** Allows one `null` element.
   - **Use Case:** Best choice when you need a general-purpose `Set`, don't care about the order of elements, and want the fastest performance for adding, removing, and checking for presence.
2. `LinkedHashSet`
   - **Implementation:** It's a subclass of `HashSet`. Internally, it uses a `LinkedHashMap` (which maintains both a hash table like `HashMap` and a doubly-linked list connecting the elements).

- **Ordering:** Maintains the insertion order of elements. When you iterate over a `LinkedHashSet`, the elements appear in the order they were added.
- **Performance:** Offers the same O(1) average-case performance for `add`, `remove`, and `contains` as `HashSet`, although with a slightly higher constant overhead due to maintaining the linked list. Iteration performance is better than `HashSet` in typical scenarios because it only depends on the number of elements (size), not the capacity.
- **Nulls:** Allows one `null` element.
- **Use Case:** Ideal when you need the uniqueness of a `Set` but also need to maintain the order in which elements were inserted. Useful for creating predictable sequences or caches where insertion order matters.

3. `TreeSet`
   - **Implementation:** Uses a `TreeMap` internally, which is based on a Red-Black Tree data structure.
   - **Ordering:** Keeps elements sorted according to their *natural ordering* (if the elements implement the `Comparable` interface) or according to a `Comparator` provided when the `TreeSet` is created.
   - **Performance:** Offers O(logn) performance for `add`, `remove`, and `contains` operations due to the balanced tree structure. Iteration is efficient (O(n) for the whole set).
   - **Nulls:** [1] Does **not** allow `null` elements by default because `null` cannot be compared to other elements using natural ordering (it would cause a `NullPointerException`). If you use a `Comparator` that explicitly handles `null`, you *might* be able to store one, but it's generally not recommended.
   - **Interfaces:** Implements `NavigableSet` and `SortedSet`, providing additional methods for navigation (e.g., `first()`, `last()`, `headSet()`, `tailSet()`, `ceiling()`, `floor()`).
   - **Requirements:** [2] Elements added must be *mutually comparable*. This usually means they must all implement the `Comparable` interface, or you must supply a suitable `Comparator`. Adding non-comparable elements (or elements of incompatible types) will result in a `ClassCastException`.
   - **Use Case:** The go-to choice when you need a `Set` that automatically keeps elements sorted, or when you need the navigational capabilities provided by the `SortedSet` and `NavigableSet` interfaces.

When to Use Which:

- Use `HashSet` if: You need maximum speed and don't care about element order.
- Use `LinkedHashSet` if: You need the performance benefits of hashing but also need to maintain the order of insertion.
- Use `TreeSet` if: You need elements to be kept in a sorted order (either natural or custom) or require navigational methods.

By understanding these differences, you can choose the most appropriate `Set` implementation for your specific needs in Java.

```
/**
Developer: James Goudy
Project HashSet, LinkedHashSet, TreeSet - Demo

Sets - can only contain unique values


 */
package java_sets; // Declares the package name for this Java file

/**
 *
```

```
 * @author jgoudy
 */
// Import statements for necessary Java utility classes


// Imports the HashSet class, which implements the Set interface
// using a hash table (no guaranteed order)
import java.util.HashSet;

// Imports the LinkedHashSet class, which maintains insertion order
import java.util.LinkedHashSet;

// Imports the Set interface, the base interface for all set collections
import java.util.Set;

// Imports the TreeSet class, which stores elements
// in a sorted order (natural ordering or by Comparator)
import java.util.TreeSet;

// Public class definition named Java_Sets
public class Java_Sets {


    /*
    Notice how easy it is to switch between the different sets.
    Difference being is how order is being stored/not stored pending
    the algorithm

    */


    // The main method, the entry point of the Java application
    public static void main(String[] args)
    {

        // Declare a variable 'mySet' of type Set that can hold String objects.
        // This uses the interface type, allowing different
        // implementations to be assigned later.
        Set<String> mySet;


        // Initialize a String array 'myData' with some sample data,
        // including duplicate values.
        String[] myData = {"AA","AA.","BB", "CC", "DD", "BB", "EE.",
                           "CC", "AA", "FF", "AA","BB", "GG","EE"};

        // - - - - - - - - - - - - - - - - HashSet Demonstration - - - - - - - - - - - - - - - -

        // Instantiate 'mySet' as a new HashSet.
        // HashSet does not guarantee any specific order of elements.
        mySet = new HashSet<>();

        // Loop through each 'value' in the 'myData' array
```

```
for (String value : myData) {

    // Add the 'value' to the HashSet. If the value is already present,
    // add() returns false and
    // the set remains unchanged (ensuring uniqueness).

    mySet.add(value);
}

// Print a header indicating the output is from the HashSet
System.out.println("\nHashset - no guarantee of order");

// Loop through each 'value' currently in the HashSet
for (String value : mySet) {
    // Print the value followed by a space. The order is unpredictable.
    System.out.print(value + " ");
}

// Print newlines for better formatting in the console output
System.out.println("\n");


// - - - - - - - - - - - - - - - - LinkedHashSet Demonstration - - - - - - - - - - - - -
// Re -instantiate 'mySet' as a new LinkedHashSet.
// LinkedHashSet maintains the order in which elements were inserted.
mySet = new LinkedHashSet<>();

// Loop through each 'value' in the 'myData' array again
for (String value : myData) {
    // Add the 'value' to the LinkedHashSet.
    // Duplicates are ignored. Insertion order is preserved.
    mySet.add(value);
}

// Print a header indicating the output is from the LinkedHashSet and
// its characteristic (Insertion Order)
System.out.println("\nLinkedSet - Insertion Order");


// Loop through each 'value' currently in the LinkedHashSet
for (String value : mySet) {

    // Print the value followed by a space.
    // The order will match the insertion sequence
    // of unique elements from myData.
    System.out.print(value + " ");
}
// Print newlines for formatting
System.out.println("\n");


// - - - - - - - - - - - - - - - - - - - - TreeSet Demonstration - - - - - - - - - - - - -
// Re -instantiate 'mySet' as a new TreeSet.
```

```
        // TreeSet stores elements in a sorted order
        // (natural alphabetical order for Strings).
        mySet = new TreeSet<>();

        // Loop through each 'value' in the 'myData' array once more
        for (String value : myData) {

            // Add the 'value' to the TreeSet.
            // Duplicates are ignored.
            //Elements are automatically sorted upon insertion.
            mySet.add(value);
        }

        // Print a header indicating the output is from the TreeSet and
        // its sorting mechanism (based on Red -Black Tree,
        // resulting in sorted order)
        System.out.println("\nTreeSet - Items put in order based "
                           + "on Red Black Tree");


        // Loop through each 'value' currently in the TreeSet
        for (String value : mySet) {
            // Print the value followed by a space.
            // The order will be alphabetical.
            System.out.print(value + " ");
        }


        // Print newlines for formatting
        System.out.println("\n");

    } // End of the main method

} // End of the Java_Sets class
```

0.9.3 Programming To The Interface, Not The Implementation"

"Programming to the interface, not the implementation," including the definitions of Interface and Abstract

Background Information

1. Definitions

- **Abstract Class:**
  - An abstract class is a class declared with the `abstract` keyword.
  - It **cannot be instantiated** directly using `new`.
  - It **can contain** both **abstract methods** (methods without implementation) and **concrete methods** (methods with implementation).
  -
  - It1 can also have instance variables (state) and constructors, just like a regular class.
  - A (concrete) class `extends` an abstract class and *must* provide implementations for any inherited abstract methods.
  - Abstract classes are often used as a **base class** for a group of related subclasses, providing some common implementation details or state while leaving other parts for the subclasses to define. They represent an "is-a" relationship with some shared structure or partial implementation.
- **Interface:**
  - In Java, an interface is a **reference type** that acts as a **contract** or a blueprint for classes.
  - It defines a set of **abstract methods** (methods without implementations, prior to Java 8) and/or **constants** (`static final` fields).
  - Since Java 8, interfaces can also contain **`default` methods** (methods with an implementation that implementing classes inherit) and **`static` methods**. Since Java 9, they can also include `private` methods.
  - A class `implements` an interface, promising to provide concrete implementations for all the abstract methods declared in that interface (unless the class itself is abstract).
  - Interfaces **cannot be instantiated** directly using `new`.
  - Their primary purpose is to define a common set of behaviors (a contract) that multiple, potentially unrelated classes can adhere to. They enable polymorphism and achieve abstraction.

Key Difference Summary: Interfaces primarily define a *contract* (what methods must exist), focusing on *what* a class can do. Abstract classes can provide a *partial implementation* and common *state*, often serving as a structural base for related classes. A class can implement multiple interfaces but can only extend one class (abstract or concrete).

Programming to the Interface, Not the Implementation

This is a fundamental design principle in object-oriented programming. It means:

When you declare variables, write method parameters, or specify return types, you should use the most general type possible that provides the necessary functionality – usually an interface type – rather than a specific concrete class type.

- **"Programming to the Interface":** Your code depends on the *contract* defined by the interface (e.g., `Set`, `List`, `Map`). It interacts with objects based *only* on the methods and behaviors guaranteed by that interface.
  Java

```
// GOOD: Programming to the 'Set' interface
Set<String> names = new HashSet<>();
processSet(names);
```

```
public void processSet(Set<String> data) { // Accepts any kind of Set
    // Code here only uses methods defined in the Set interface
    if (data.contains("Alice")) {
         data.remove("Alice");
    }
    // ...
}
```

- **"Not the Implementation":** You avoid tying your code directly to a specific concrete class (e.g., `HashSet`, `ArrayList`, `HashMap`) unless you absolutely need functionality unique to that specific class (which is rare for general use).
  Java

```
// LESS FLEXIBLE: Programming to the 'HashSet' implementation
HashSet<String> names = new HashSet<>();
processSpecificSet(names);

public void processSpecificSet(HashSet<String> data) { // Only accepts HashSets
   // Code might potentially use HashSet -specific methods (if any existed beyond Set)
   // or implicitly rely on HashSet's behavior (like lack of order).
   // ...
}
```

Why follow this principle?

1. **Flexibility/Maintainability:** Your code becomes much easier to change. If you decide you need a different implementation later (e.g., you need sorted elements), you only change the object creation part (`new ...()`), not the rest of the code that uses the object via its interface.
2. **Loose Coupling:** Components of your system depend on abstract contracts rather than concrete details of other components. This reduces dependencies and makes the system more modular. Changes in one part are less likely to break others.
3. **Testability:** It's easier to substitute mock objects (for testing) that implement the same interface.
4. **Abstraction:** It hides unnecessary implementation details, making the code cleaner and focusing on the essential behavior.

0.10 End

End Of Content